
DOKUMENTACJA TECHNICZNA

Fraction Guild Clans

*Kompletna dokumentacja systemu gildii
dla serwerow Minecraft*

Wersja 2.0 — Modułarna · Samowystarczalna · Wielojęzyczna

Kompatybilność: Minecraft 1.8 – 1.21.11

Silniki: Paper / Spigot / Purpur

Zależności: BRAK (zero zależności zewnętrznych)

Data: Czerwiec 2026

Spis treści

1. Wprowadzenie
2. Instalacja i Struktura Plików
3. Lang Files — System Plików Językowych
4. System Gildii
5. Jajo Gildii — System Obrony
6. Cuboidy i Własny System Ochrony
7. Wiesniacy Handlowi
8. System Dołączania do Gildii
9. GUI — Interfejs Graficzny
10. Ranking Gildii
11. Mapa Serwera HTML
12. Custom Tabulator
13. System Modułów i Konfiguracja
14. Komendy i Uprawnienia
15. Troubleshooting i FAQ

1. Wprowadzenie

1.1 Opis Pluginu i Filozofia

1.1.1 Fraction Guild Clans jako kompleksowy system gildii

Fraction Guild Clans (w skrócie FGC) to zaawansowany plugin serwerowy przeznaczony dla platform Paper, Spigot oraz Purpur, który łączy najlepsze cechy klasycznych systemów gildii z nowoczesnymi rozwiązaniami architektonicznymi. Plugin tworzy pełny ekosystem rozgrywki gildijnej — od momentu założenia gildii przez grupę graczy, przez rozwój terytorialny, system rankingowy, aż po mechaniki rajdowania i obrony baz. W odróżnieniu od wielu rozwiązań dostępnych na rynku, FGC nie jest prostym wrapperem ani fork istniejącego projektu. Stanowi autorską implementację, zaprojektowaną od podstaw z myślą o modularności, samowystarczalności i pełnej konfigurowalności.

Filozofia projektowa FGC zakłada, że każdy serwer Minecraft ma unikalną wizję rozgrywki. Jednym administratorowi zależy na hardkorowym modelu rajdów z pełnym PvP i niszczeniem jaj gildii, podczas gdy inny buduje serwer roleplay z gildiami działającymi jako domy handlowe. FGC akomoduje oba te scenariusze — i wszystkie pośrednie — bez konieczności ingerencji w kod źródłowy pluginu. Cała logika rozgrywkowa, ekonomiczna i ochronna jest definiowana deklaratywnie poprzez pliki konfiguracyjne YAML, co czyni z FGC narzędzie nie tylko dla serwerów survival-PvP, ale także dla projektów RPG, economy oraz minigames.

Plugin w wersji 2.0 został zaprojektowany z myślą o najnowszych wersjach Minecraft (1.21.11), ale zachowuje kompatybilność wstecz do 1.8.x. Oznacza to, że mechaniki takie jak serce gildii w formie jaja (Egg System), system rankingowy ELO, czy cuboidy z flagami ochronnymi działają niezależnie od wersji serwera, a administrator może korzystać z nowoczesnych bloków (np. Creaking Heart, Trial Spawner, Heavy Core) jako elementów swoich konstrukcji gildijnych.

1.1.2 Filozofia "Everything is Configurable"

FGC opiera się na zasadzie "**Everything is Configurable**" — dosłownie każdy aspekt działania pluginu można dostosować do potrzeb serwera. Termin ten oznacza coś więcej niż zwykłą konfigurację parametrów numerycznych. W kontekście FGC oznacza on:

- **Każdą funkcjonalność** można włączyć lub wyłączyć bez wpływu na pozostałe komponenty (system modułów).

- **Każdy koszt** (itemy, waluta, doświadczenie) wymagany do założenia lub rozbudowy gildii jest definiowalny przez administratora.
- **Każda wiadomość** wyświetlana graczom może zostać przetłumaczona, zmodyfikowana lub rozszerzona o własne placeholdery.
- **Każda mechanika ochronna** (zakres cuboidu, czas tarczy, odporność na TNT, zasady PvP) podlega szczegółowej konfiguracji per-gildia lub globalnie.
- **Każdy element GUI** (ikony, nazwy, opisy, układ ekranów) można zmienić bez znajomości programowania.

Filozofia ta eliminuje najczęstszy problem pluginów gildijnych: sytuację, w której administrator musi dostosować wizję serwera do narzuconych przez plugin ograniczeń. W FGC to plugin dostosowuje się do wizji administratora. Wszystkie parametry posiadają sensowne wartości domyślne, co pozwala na uruchomienie serwera natychmiast po instalacji, jednocześnie oferując głęboką personalizację dla zaawansowanych użytkowników.

1.1.3 Architektura modułowa — system modułów

W wersji 2.0 FGC wprowadza **architekturę modułową**, w której każda funkcjonalność stanowi osobny, niezależny moduł. Moduły to pliki YAML umieszczone w katalogu `plugins/FractionGuildClans/modules/` i każdy z nich można aktywować lub dezaktywować poprzez zmianę wartości parametru `enabled`.

Dostępne moduły obejmują m.in.:

Moduł	Nazwa pliku	Funkcjonalność
Jajo Gildii	<code>egg.yml</code>	Serce gildii do zniszczenia, system żyć (3 stopnie), hologramy, naprawa
Cuboidy	<code>cuboid.yml</code>	System ochrony terenów z flagami, promień cuboidu, strefy (spawn/budowla/PvP)
Ranking	<code>ranking.yml</code>	System ELO, asysty, K/D/A, top gildii i graczy, obliczenia rankingowe
GUI	<code>gui.yml</code>	Wszystkie ekrany interfejsu graficznego — główne menu, panel gildii, administracja
TAB	<code>tab.yml</code>	Niestandardowa lista graczy (TAB) z rankingiem, prefixami gildii, animacjami
Mapa	<code>map.yml</code>	Mapa gildii (HTML oraz ASCII w czacie), oznaczenia terytoriów

Wieśniacy	<code>villagers.yml</code>	Handlarze gildii (customowe encje), system wymiany i handlu
Ekonomia	<code>economy.yml</code>	Integracja z EcoCore, waluty, banki gildii, transakcje, odsetki
Pay-to-Join	<code>join_items.yml</code>	System opłat za dołączenie do gildii (itemy lub waluta)

Taka struktura zapewnia kilka kluczowych korzyści. Po pierwsze, **niezawodność** — wyłączenie modułu, który nie jest potrzebny na danym serwerze, eliminuje cały zbiór potencjalnych błędów z nim związanych. Po drugie, **wydajność** — moduł, który jest dezaktywowany, nie rejestruje listenerów (nasłuchiwalcy zdarzeń), nie zajmuje pamięci operacyjnej ani nie obciąża głównego wątku serwera (main thread). Po trzecie, **czytelność konfiguracji** — administrator modyfikuje tylko te pliki, które odpowiadają za funkcje faktycznie używane na serwerze, co znacząco redukuje złożoność zarządzania.

Moduły mogą mieć zależności między sobą (np. moduł `economy.yml` musi być aktywny, jeśli `cuboid.yml` wymaga opłat za rozbudowę), ale system ładowania automatycznie wykrywa i raportuje takie zależności podczas startu serwera, informując administratora o konieczności aktywacji powiązanego modułu.

1.2 Wymagania Systemowe

1.2.1 Wspierane wersje silnika

Fraction Guild Clans współpracuje z trzema głównymi rodzinami silników serwerowych Minecraft, obejmującymi szeroki zakres wersji gry od klasycznych wydań po najnowsze aktualizacje:

Silnik	Minimalna wersja	Maksymalna wersja	Rekomendacja
Paper	1.8.x	1.21.11	Wersja 1.20+ dla pełni funkcji
Spigot	1.8.x	1.21.11	Kompatybilność podstawowa
Purpur	1.16.x	1.21.11	Dla zaawansowanych konfiguracji

Paper jest zalecanym silnikiem, ponieważ zapewnia najlepszą równowagę między wydajnością a kompatybilnością z pluginami Bukkit/Spigot. Purpur oferuje dodatkowe opcje konfiguracyjne (np. customowe reguły gry per-świat), które mogą wzbogacić rozgrywkę gildijną, ale wymaga świadomego zarządzania tymi rozszerzeniami. Spigot jest w pełni obsługiwany, choć niektóre optymalizacje wydajnościowe dostępne w Paper mogą wpłynąć pozytywnie na stabilność przy dużej liczbie aktywnych gildii (100+).

Wersje 1.8.x–1.12.x są obsługiwane w trybie kompatybilności wstecznej, co oznacza, że wszystkie podstawowe mechaniki (gildie, cuboidy, ranking, jajo) działają poprawnie, ale niektóre funkcje zależne od

nowszych API Minecraft (np. komponenty MiniMessage, nowe typy bloków jako jaja gildii, zaawansowane particle) mogą być niedostępne lub działać w trybie fallback (zastępczym).

1.2.2 Brak wymaganych zależności zewnętrznych

Najważniejsza cecha techniczna FGC v2.0: plugin jest **całkowicie samowystarczalny** (self-contained). Oznacza to, że po pobraniu pliku JAR i umieszczeniu go w katalogu `plugins/` serwer jest gotowy do działania — bez żadnych dodatkowych pluginów, bibliotek ani zależności.

W praktyce oznacza to eliminację typowej dla pluginów gildijnych listy wymagań wstępnych:

- **NIE jest wymagany Vault** — FGC v2.0 posiada wbudowany system ekonomiczny oparty na EcoCore, który jest własnym, opcjonalnym rozszerzeniem tego samego autora.
- **NIE są wymagane WorldGuard ani WorldEdit** — FGC posiada własny system ochrony terenów (cuboidów) z pełnym zestawem flag, który nie koliduje z innymi systemami ochrony i nie wymaga zewnętrznych narzędzi do zarządzania regionami.
- **NIE jest wymagany żaden plugin ekonomiczny** (EssentialsX, CMI, itp.) — system walutowy EcoCore jest wbudowany i działa niezależnie.
- **NIE jest wymagany żaden plugin do bazy danych** — FGC posiada wbudowane wsparcie dla SQLite (domyślnie) oraz MySQL, zarządzane własnym systemem persystencji danych.

Taka architektura znacząco upraszcza proces instalacji i aktualizacji. Administrator nie musi monitorować kompatybilności między wieloma pluginami, rozwiązywać konfliktów wersji ani martwić się o to, że aktualizacja jednego zależnego komponentu (np. WorldEdit) spowoduje awarię systemu gildii. Każda aktualizacja FGC jest samodzielnym pakietem, który zawiera wszystkie niezbędne komponenty.

1.2.3 Opcjonalne integracje

Mimo że FGC nie wymaga żadnych zewnętrznych pluginów, oferuje opcjonalne integracje z najpopularniejszymi narzędziami ekosystemu Minecraft, które mogą wzbogacić funkcjonalność serwera:

Integracja	Typ	Co dostarcza
PlaceholderAPI	Opcjonalna	Placeholdery (<code>%fgc_guild%</code> , <code>%fgc_rank%</code> , <code>%fgc_points%</code>) używane w innych pluginach (TAB, scoreboard, chat)
EcoCore	Opcjonalna	Rozszerzony system ekonomiczny autora — wielowalutowość, banki gildii z odsetkami, zaawansowane transakcje

Integracja z PlaceholderAPI jest czysto informacyjna — FGC rejestruje swoje placeholdery, które mogą być odczytywane przez inne pluginy. Nie wpływa to na stabilność ani funkcjonalność FGC; w przypadku braku

PlaceholderAPI plugin działa w pełni normalnie, a placeholders są dostępne wewnętrznie (np. w systemie TAB i wiadomościach).

Integracja z EcoCore aktywuje zaawansowane funkcje ekonomiczne. Bez EcoCore FKC używa wbudowanego, uproszczonego systemu walutowego (pojedyncza waluta serwerowa). Po wykryciu EcoCore plugin automatycznie deleguje operacje ekonomiczne do tego systemu, umożliwiając m.in. prowadzenie banków gildii z naliczaniem odsetek, transakcje między gildiami, waluty gildijne oraz integrację z szerszym ekosystemem ekonomicznym serwera. EcoCore jest własnością tego samego autora co FGC, co gwarantuje pełną kompatybilność i synchroniczny rozwój obu projektów.

1.3 Analiza Konkurencyjna

1.3.1 Porównanie z FactionsUUID, FunnyGuilds, Guilds

Rynek pluginów gildijnych jest zdominowany przez kilka ugruntowanych rozwiązań, z których każde ma swoje mocne strony i ograniczenia. Poniższa tabela systematyzuje kluczowe różnice między Fraction Guild Clans v2.0 a trzema najpopularniejszymi alternatywami: FactionsUUID (spuścizna klasycznego systemu Factions), FunnyGuilds (najpopularniejszy polski plugin gildii) oraz Guilds by Glare (nowoczesne rozwiązanie z pełnym GUI).

Kryterium	FactionsUUID	FunnyGuilds	Guilds (Glare)	Fraction Guild Clans v2.0
Wymagane zależności	Vault, opcjonalnie WG/WE	Brak (samowystarczalny)	Vault	Brak — całkowicie samowystarczalny
System ekonomii	Wymaga Vault + plugin ekonomiczny	Własny (prosty)	Wymaga Vault	EcoCore — własny system ekonomiczny autora (opcjonalny)
Ochrona terenów	Brak wbudowanej (zależy od WG/WE)	Własny RegionManager	Własny claim system	Własny system cuboidów z flagami — bez WG/WE
System językowy (i18n)	lang.yml + translations.conf (dwa systemy równoległe)	messages.yml (własny format)	Katalog lang/ z plikami per-język	Katalog lang/ z MiniMessage + legacy, hot-reload, fallback

Architektura modułów	Monolityczny — wszystko w jednym configu	Monolityczny — częściowa konfiguracja	Monolityczny — włączanie/wyłączanie funkcji	Pełna modularność — każda funkcja to osobny plik YAML w <code>modules/</code>
Serce / Jajo gildii	Brak (system Power)	Tak — blok do zniszczenia, 3 życia	Brak (system tierów)	Tak — jajo z 3-stopniową ochroną, hologramy, naprawa
System rankingowy	Brak wbudowanego	ELO (top i graczy), asysty, K/D/A	Podstawowy leaderboard	ELO z asystami, K/D/A, szczegółowe statystyki
GUI (interfejs graficzny)	Brak — tylko komendy	Brak — tylko komendy	Pełne GUI dla wszystkich funkcji	Pełne GUI — konfigurowalne per-moduł
Custom TAB	Brak	Tak — animowana tablista	Brak	Tak — konfigurowalny, per-moduł
Mapa gildii	ASCII w czacie (<code>/f map</code>)	Brak	Brak	ASCII w czacie + mapa HTML
Wieśniacy / Handlarze	Brak	Brak	Brak	Tak — customowe encje, system wymiany
Pay-to-Join	Brak	Brak	System invite codes	Tak — opłaty w itemach lub walucie
TNT Bank	Tak (deposit/withdraw/siphon/fill)	Brak	Brak	Tak — pełny system zarządzania
Wsparcie wersji	1.8–1.20+	1.8–1.20+	1.16+	1.8.x – 1.21.11

Baza danych	Brak (pliki flat)	SQLite, MySQL	SQLite, MySQL, MongoDB	SQLite MySQL (domyślnie),
System tarczy (Shield)	Tak	Czasowa ochrona przed TNT	Brak	Tak — konfigurowalna per-gildia
Format wiadomości	MiniMessage + legacy	Własne placeholdery	MiniMessage	MiniMessage + legacy &k , własne placeholdery

Analiza powyższej tabeli ukazuje wyraźnie cztery obszary, w których FGC v2.0 definiuje nowy standard na rynku pluginów gildijnych.

Po pierwsze, samowystarczalność. FactionsUUID od dekad pozostaje wierny koncepcji zależności od Vault i zewnętrznych systemów ochrony terenów. Oznacza to, że administrator musi zarządzać ekosystemem 3–5 pluginów, z których każdy wymaga osobnej konfiguracji, aktualizacji i rozwiązywania konfliktów. FunnyGuilds, choć samowystarczalny, nie oferuje wbudowanego systemu ekonomicznego — jego system rankingowy i ekonomiczny są uproszczone i nieintegrują się z szerszym rynkiem walutowym serwera. FGC v2.0 eliminuje ten problem całkowicie: wbudowany system cuboidów zastępuje WorldGuard/WorldEdit, a EcoCore zastępuje Vault i dowolny plugin ekonomiczny, przy czym oba są opcjonalne i nieinwazyjne.

Po drugie, modularność. Żaden z porównywanych pluginów nie oferuje architektury modułowej na poziomie FGC. FactionsUUID i FunnyGuilds opierają się na monolitycznej strukturze konfiguracyjnej, gdzie włączenie lub wyłączenie funkcji wymaga edycji rozproszonych sekcji w jednym, często ogromnym pliku. W FGC każdy moduł to osobny, zamknięty plik YAML w katalogu `modules/`, który można aktywować jednym przełącznikiem `enabled: true/false`. Taka struktura jest bardziej czytelna dla administratora, łatwiejsza w utrzymaniu i bardziej odporna na błędy — problem w module `villagers.yml` nigdy nie wpłynie na działanie `ranking.yml`.

Po trzecie, system plików językowych. FactionsUUID boryka się z problemem dualnego systemu tłumaczeń (równoległe `lang.yml` i `translations.conf` w różnych formatach), co wprowadza dezorientację. FunnyGuilds używa własnego, nieznormalizowanego formatu `messages.yml`, który nie wspiera MiniMessage ani hierarchicznych kluczy. FGC v2.0 implementuje nowoczesny system i18n z katalogiem `lang/`, wsparciem dla formatu MiniMessage (`<color>`, `<bold>`, `<gradient>`) z jednoczesnym fallbackiem do legacy (`&a`, `&l`), hot-reloadem bez restartu serwera oraz automatycznym fallbackiem do języka angielskiego w przypadku brakujących kluczy w lokalnym tłumaczeniu.

Po czwarte, zakres funkcjonalny. FGC v2.0 łączy elementy, które w innych pluginach występują rozdzielone: serce gildii w formie jajo (inspiracja FunnyGuilds), pełne GUI (inspiracja Guilds), TNT Bank (inspiracja FactionsUUID), system wieśniaków-handlarzy (unikalna funkcja FGC), custom TAB, mapę HTML oraz system pay-to-join. Modularna architektura pozwala administratorowi skomponować własny

zestaw funkcji — od minimalnego systemu gildii z jednym cuboidem, po rozbudowany ekosystem z pełną ekonomią, rankingiem i handlem.

2. Instalacja i Struktura Plikow

Rozdział ten opisuje kompletny proces instalacji pluginu **Fraction Guild Clans v2.0**, strukture generowanych plikow oraz architekture systemow, ktore w wersji 2.0 zostaly calkowicie przebudowane z mysla o niezaleznosci od zewnetrznych zaleznosci. W przeciwienstwie do wiekszosci pluginow gildii dostepnych na rynku, FGC v2.0 nie wymaga instalacji Vault, WorldGuard, WorldEdit ani zadnego innego pluginu towarzyszacego — wszystkie mechaniki sa realizowane przez wbudowane moduly.

2.1 Instalacja i Pierwsze Uruchomienie

2.1.1 Instalacja JAR — filozofia zero zaleznosci

Proces instalacji pluginu Fraction Guild Clans v2.0 sprowadza sie do pojedynczego kroku: umieszczenia pliku `FractionGuildClans-2.0.jar` w katalogu `plugins/` serwera Minecraft. Po restarcie serwera lub uzyciu komendy zarzadzajacej pluginami (takiej jak `/reload confirm` na silnikach Paper/Spigot, lub `/folia reload` na Folia), plugin automatycznie generuje cala niezbedna strukture katalogow i plikow konfiguracyjnych.

Taka filozofia "zero zaleznosci" (ang. *zero-dependency architecture*) oznacza, ze administrator serwera nie musi pobierac ani konfigurowac zadnych dodatkowych pluginow. W tradycyjnym ekosystemie serwerowym plugin gildii wymagalby co najmniej: **Vault** jako API ekonomicznego, **EssentialsX** lub **CMI** jako backendu ekonomii, **WorldGuard** do ochrony terenow oraz **WorldEdit** jako zaleznosci WorldGuarda. W przypadku FGC v2.0 caly ten stos technologiczny zostaje zastapiony jednym, samowystarczalnym plikiem JAR. Plugin wykrywa automatycznie, czy na serwerze zainstalowane sa opcjonalne integracje (takie jak **PlaceholderAPI**), i aktywuje je tylko wtedy, gdy sa dostepne — nigdy nie sa wymagane do podstawowego dzialania.

Pierwsze uruchomienie tworzy domyslne pliki konfiguracyjne z komentarzami dokumentujacymi kazdy parametr. Plugin zapisuje rowniez w logach serwera (plik `latest.log`) potwierdzenie aktywacji kazdego modulu oraz informacje o trybie pracy systemu ekonomicznego (EcoCore lub tryb standalone z itemami jako waluta).

2.1.2 Struktura katalogow

Po pierwszym uruchomieniu plugin generuje hierarchie katalogow, ktora organizuje wszystkie dane, konfiguracje i zasoby. Struktura ta zostala zaprojektowana z mysla o latwej nawigacji dla administratorow

oraz o separacji danych statycznych (konfiguracja, tłumaczenia) od danych dynamicznych (dane gildii, graczy, kopie zapasowe).

Ponizsza tabela przedstawia kompletna strukture katalogow generowana automatycznie przez plugin:

Katalog / Plik	Przeznaczenie	Zawartosc	Modyfikowalny
<code>plugins/FractionGuildClans/</code>	Katalog glowny pluginu	Wszystkie pliki i podkatalogi	Nie (sciezka stal)
<code>└─ config.yml</code>	Glowna konfiguracja	Parametry globalne, prefixy, formaty wiadomosci, ustawienia TPS	Tak
<code>└─ lang/</code>	Pliki jezykowe	Tlumaczenia: <code>pl_PL.yml</code> , <code>en_US.yml</code> , <code>de_DE.yml</code> , <code>es_ES.yml</code> i inne	Tak (hot-reload)
<code>└─ modules/</code>	Konfiguracje modulow	14 plikow YAML, kazdy odpowiada za jeden modul funkcjonalny	Tak (wymaga restartu modulu)
<code>└─ data/</code>	Baza danych	Podkatalogi <code>guilds/</code> i <code>players/</code> z danymi w formacie SQLite lub plikow JSON	Nie (edytowac przez komendy)
<code>└─ backups/</code>	Kopie zapasowe	Automatyczne snapshoty bazy danych z datownikiem (<code>backup_YYYY-MM-DD_HH-MM.zip</code>)	Mozna pobierac
<code>└─ logs/</code>	Logi operacyjne	Logi transakcji ekonomicznych, zmian w gildiach, audyt bezpieczenstwa	Tylko do odczytu

Analiza struktury katalogow ujawnia kilka kluczowych decyzji architektonicznych. Po pierwsze, separacja katalogu `lang/` jako samodzielnego poddrzewa umozliwia dodawanie nowych tłumaczen bez ingerencji w kod pluginu — wystarczy umiescic nowy plik YAML z odpowiednim kodem jezykowym (np. `fr_FR.yml` dla francuskiego), a plugin automatycznie go wykryje przy kolejnym uruchomieniu. Po drugie, katalog `modules/` realizuje filozofie modularnej architektury: kazdy plik YAML w tym katalogu odpowiada za jeden, niezalezny aspekt funkcjonalny pluginu. Administrator moze wytlaczyc dowolny modul zmieniajac wartosc `enabled: false` w odpowiednim pliku, co spowoduje calkowite wyrejestrowanie mechanik zwiazanych z tym modulem wraz z jego komendami, listenerami eventow i placeholderami. Po trzecie, katalog `data/` przechowuje wszystkie dane dynamiczne w formatach latwych do migracji — domyslnie SQLite dla lokalnych instalacji, z opcja przełaczenia na MySQL lub PostgreSQL w pliku `config.yml`. Katalog `backups/` zawiera automatyczne kopie zapasowe generowane co skonfigurowany interval

(domyslnie co 6 godzin) oraz przed kazda dezaktywacja pluginu, co zabezpiecza przed utrata danych przy awarii serwera.

2.1.3 System modutow — włączanie i wyłączanie funkcjonalności

Architektura modularna stanowi fundament wersji 2.0 i odpowiada na jedno z najczestszych zadan administratorow serwerow: mozliwosc dostosowania funkcjonalnosc pluginu do specyfiki swojego serwera bez koniecznosc edycji kodu zrodlowego. W FGC v2.0 kazdy modul to samodzielna jednostka funkcjonalna zawierajaca wlasne komendy, listenery eventow, placeholders oraz plik konfiguracyjny. Moduly mozna włączac i wyłączac niezaleznie od siebie, a zmiana ta wymaga jedynie uzycia komendy `/guild admin module reload <nazwa>` — bez restartu serwera.

Ponizsza tabela przedstawia 14 modutow dostepnych w FGC v2.0 wraz z ich przeznaczeniem i zaleznosciami miedzy modulami:

Modul	Plik konfiguracyjny	Opis funkcjonalny	Zalezy od	Domyslnie
guild	modules/guild.yml	Podstawy systemu gildii: tworzenie, usuwanie, zaproszenia, ranga, zarzadzanie czlonkami	—	Wlaczony
egg	modules/egg.yml	System jaja gildii — 3 poziomy ochrony, uszkodzenia, naprawa, destrukcja gildii	guild, cuboid	Wlaczony
cuboid	modules/cuboid.yml	Wlasny system ochrony terenow — cuboidy, 7 flag, 3 strefy, trusted players	guild	Wlaczony
ranking	modules/ranking.yml	System rankingu ELO graczy i gildii, topki, asysty, K/D/KDA	guild	Wlaczony
villagers	modules/villagers.yml	Wiesniacy gildii — handel, questy, customowe oferty per-gildia	guild, economy	Wyłączony
gui	modules/gui.yml	System graficznych interfejsow — menu gildii, ekrany konfiguracji	guild	Wlaczony
tab	modules/tab.yml	Customowa tablista z rankingiem, prefixami gildii, animacjami	guild, ranking	Wlaczony
map	modules/map.yml	Interaktywna mapa gildii w przegladarce (HTML) oraz mapa ASCII w czacie	guild, cuboid	Wlaczony
economy	modules/economy.yml	Integracja z EcoCore lub tryb standalone (itemy jako waluta)	—	Wlaczony

<code>join_items</code>	<code>modules/join_items.yml</code>	System itemow startowych dla nowych graczy i nagrod za zaproszenia	—	Wylaczony
<code>lang</code>	<code>modules/lang.yml</code>	Zarzadzanie plikami jezykowymi, hot-reload, fallback jezykowy	—	Wlaczony
<code>database</code>	<code>modules/database.yml</code>	Warstwa persystencji danych — SQLite, MySQL, PostgreSQL, migracje	—	Wlaczony
<code>backup</code>	<code>modules/backup.yml</code>	Automatyczne i reczne kopie zapasowe, przywracanie danych	<code>database</code>	Wlaczony
<code>webhook</code>	<code>modules/webhook.yml</code>	Integracje z Discordem przez webhooki powiadomienia o wydarzeniach gildii	— <code>guild</code>	Wylaczony

Analiza tabeli modutow ukazuje kilka istotnych wlasciwosci architektury. Moduly `guild`, `economy`, `lang`, `database` i `backup` stanowia rdzen systemu (core modules) — sa one domyslnie wlaczone i nie zaleza od innych modutow opcjonalnych. Pozostale moduly rozszerzaja funkcjonalnosc rdzenia i deklaruja zaleznosci, ktore musza byc spelnione, aby modul zostal aktywowany. Przyklad: proba wlaczenia modulu `egg` bez aktywnego modulu `cuboid` skutkuje bledem w logach i automatycznym pozostawieniem modulu `egg` w stanie nieaktywnym. Taki system zaleznosci zapobiega bledom wynikajacym z braku wymaganych komponentow. Z kolei moduly `villagers`, `join_items` i `webhook` sa domyslnie wylaczone, poniewaz odnosza sie do funkcjonalnosci specjalistycznych, ktore nie kazdy administrator serwera bedzie chcial wykorzystywac. Kazdy modul posiada wlasna sekcje `settings` w pliku YAML, gdzie mozna konfigurowac szczegolowe parametry jego dzialania — na przyklad w module `cuboid` definiuje sie promien cuboidu gildii, a w module `ranking` — wspolczynniki algorytmu ELO.

2.2 EcoCore — Wlasny System Ekonomii

2.2.1 EcoCore jako dedykowany plugin ekonomiczny

W wersji 2.0 Fraction Guild Clans wprowadza **EcoCore** — autorski system ekonomiczny, ktory zastepuje tradycyjny stos Vault + EssentialsX/CMI. EcoCore to niezalezny plugin ekonomiczny stworzony przez tego samego autora co FGC, zaprojektowany specjalnie z mysla o potrzebach serwerow z systemem gildii. Decyzja o rezygnacji z Vault podyktowana jest kilkoma czynnikami technicznymi: Vault to wylaczenie API-proxy, ktore samo w sobie nie zapewnia zadnej funkcjonalnosc ekonomicznej — wymaga zewnetrznego backendu (EssentialsX, CMI, PlayerPoints). Taki model wprowadza dodatkowa warstwe abstrakcji, ogranicza mozliwosci konfiguracji i uzaleznia plugin gildii od aktualizacji innych projektow.

EcoCore eliminuje te problemy poprzez bezposrednia integracje z FGC. Komunikacja miedzy pluginami odbywa sie przez wewnetrzne API (shared interfaces), co zapewnia pelna kontrole nad walutami,

transakcjami i bankami gildii. EcoCore obsługuje wielowalutowosc — pozwala na zdefiniowanie waluty głównej serwera (np. monety) oraz walut pomocniczych specyficznych dla gildii (np. punkty gildii, tokeny PvP). Każda transakcja jest logowana w pliku `data/transaction_log.yml`, co umożliwia audyt ekonomiczny i wykrywanie anomalii.

2.2.2 Konfiguracja walut, banków i transakcji

Plik `modules/economy.yml` zawiera wszystkie parametry systemu ekonomicznego. Administrator może zdefiniować dowolną liczbę walut, każda z osobnymi ustawieniami formatowania wyświetlania, symbolu oraz precyzji dziesiętnej. Bank gildii w EcoCore funkcjonuje jako wspólne konto, do którego członkowie z odpowiednimi uprawnieniami mogą wpłacać i wypłacać środki. System odsetek pozwala na konfigurowanie procentowego przyrostu środków w banku gildii w określonym interwale czasowym, co zachęca do gromadzenia zasobów.

Przykład konfiguracji waluty w pliku `economy.yml`:

```
currencies:
  primary:
    name: "Moneta"
    symbol: "&e☹"
    format: "{amount} {symbol}"
    decimals: 2
    starting_balance: 100.0
  guild_points:
    name: "Punkt Gildii"
    symbol: "&b☆"
    format: "{symbol} {amount}"
    decimals: 0
    starting_balance: 0

bank:
  enabled: true
  interest_rate: 0.5
  interest_interval: "24h"
  min_balance_for_interest: 1000.0

transactions:
  log_enabled: true
  log_retention_days: 30
```

2.2.3 Tryb standalone — system itemów jako waluta

W sytuacji, gdy administrator nie chce używać EcoCore (lub gdy plugin EcoCore nie jest zainstalowany na serwerze), FGC v2.0 automatycznie przełącza się w **tryb standalone**. W tym trybie funkcja waluty pełniona jest przez fizyczne itemy w ekwipunku gracza — domyślnie diamenty, ale można skonfigurować dowolny

material dostępny w Minecraft (np. szmaragdy, złoto, netherite ingoty, a nawet custom itemy z innych pluginów za pomocą identyfikatorów NamespacedKey).

System itemowy działa identycznie jak system monetarny pod względem interfejsu komend — gracz używa tych samych komend (`/guild deposit` , `/guild withdraw` , `/guild pay`), ale zamiast modyfikować saldo w bazie danych, plugin dodaje lub usuwa itemy z ekwipunku. Mechanizm ten sprawdza się szczególnie na serwerach typu *survival* i *semi-vanilla*, gdzie administratorzy celowo unikają abstrakcyjnych systemów monetarnych na rzecz barteru opartego na surowcach. Przełączanie między trybem EcoCore a trybem standalone wymaga jedynie zmiany parametru `mode: "ecocore"` lub `mode: "items"` w pliku `economy.yml` i przeładowania modułu — bez konieczności restartu serwera.

2.3 Własny System Ochrony Terenów

2.3.1 Dlaczego NIE WorldGuard/WorldEdit

Tradycyjne pluginy gildii polegają na **WorldGuard** do ochrony terenów gildii, co wymaga również zainstalowania **WorldEdit** jako zależności. Taki model wprowadza szereg problemów operacyjnych. WorldGuard to potężne narzędzie do zarządzania całym światem, zawierające dziesiątki flag i mechanik nie mających zastosowania w kontekście pojedynczych gildii — konflikty priorytetów regionów, skomplikowana hierarchia obowiązków (ownership, membership, groups) oraz narzut wydajnościowy wynikający z ogólnej natury pluginu. Ponadto, każda aktualizacja WorldGuarda lub WorldEdita może wymuszać aktualizację pluginu gildii, a w przypadku opóźnień w rozwoju — prowadzić do niekompatybilności.

Fraction Guild Clans v2.0 całkowicie eliminuje te zależności. Wbudowany system ochrony oparty jest na cuboidach (prostopadłościanach przestrzennych) definiowanych wokół środka gildii, z pełnym wsparciem dla 7 flag ochronnych, 3 stref wewnętrznych i systemu zaufanych graczy. Rozwiązanie to jest nie tylko niezależne, ale również wydajniejsze — listenery eventów są rejestrowane selektywnie, tylko dla aktywnych cuboidów, co minimalizuje narzut na główny wątek serwera (main thread) i pozytywnie wpływa na wartość TPS (Ticks Per Second).

2.3.2 System ochrony oparty na cuboidach — 3 strefy

Cuboid gildii w FGC v2.0 dzieli się na trzy strefy o różnej funkcjonalności ochronnej. **Strefa spawnu** (centralny obszar wokół jaja gildii) to najbardziej chroniona przestrzeń — tutaj tylko lider i zastępcy mogą budować, a PvP jest domyślnie wyłączone. **Strefa budowlana** (środkowy pierścień cuboidu) pozwala członkom gildii na swobodną budowę i niszczenie, z możliwością dodania zaufanych graczy spoza gildii. **Strefa PvP** (zewewnętrzny pierścień) to obszar, gdzie ochrona przed PvP jest wyłączona — stanowi to strefę buforową, w której wrogowie mogą atakować członków gildii, ale nadal nie mogą niszczyć bloków.

Każda z 7 flag ochronnych (`build` , `destroy` , `pvp` , `use` , `bucket` , `entry` , `interact`) może być skonfigurowana indywidualnie dla każdej strefy. Flaga `build` kontroluje możliwość stawiania bloków,

`destroy` — ich niszczenia, `pvp` — walke między graczami, `use` — używanie przedmiotów (np. lozek, stołu zaklinania), `bucket` — nalewanie i zbieranie płynów (wody, lawy), `entry` — wchodzenie na teren cuboidu, a `interact` — interakcje z blokami (drzwi, skrzynie, przyciski, dźwignie). Administrator może globalnie zdefiniować wartości domyślne dla każdej flagi w pliku `modules/cuboid.yml`, a lider gildii może je nadpisać dla swojego cuboidu za pomocą komendy `/guild flag <nazwa> <wartosc>`.

2.3.3 Integracja z systemem jaja gildii jako główna obrona

System ochrony cuboidów ściśle współpracuje z modułem `egg` (jajo gildii). Jako gildii pełni funkcje centralnego punktu obronnego — jego zniszczenie prowadzi do upadku całej gildii. W związku z tym, strefa spawnu (w której znajduje się jajo) posiada najwyższy poziom ochrony: wszystkie flagi są domyślnie ustawione na `deny` dla osób niebędących członkami gildii, a samo jajo otrzymuje ochronę przeciwko wybuchom (TNT, wzmacniacze, respawn anchor) przez pierwsze 24 godziny od założenia gildii.

Cuboid gildii aktywowany jest automatycznie w momencie założenia gildii i dezaktywowany po zniszczeniu jaja. Promień cuboidu jest konfigurowalny i może różnić się w zależności od poziomu (tieru) gildii — wyższy tier oznacza większy obszar chroniony. System obsługuje również sytuacje nakładania się cuboidów sąsiednich gildii, stosując reguły priorytetowe oparte na poziomie rankingowym i czasie założenia.

3. Lang Files — System Plików Językowych

System plików językowych (internationalization, w skrócie i18n) w **Fraction Guild Clans v2.0** stanowi centralny mechanizm komunikacji między pluginem a graczem. Wszystkie wiadomości — od potwierdzenia utworzenia gildii po ostrzeżenie o uszkodzeniu jaja — są w pełni konfigurowalne, wielojęzyczne i wspierają nowoczesne formatowanie tekstu. Architektura oparta na plikach YAML z hierarchicznymi kluczami eliminuje konieczność rekompilacji pluginu przy zmianie dowolnego komunikatu, a mechanizm hot-reload pozwala na natychmiastowe zastosowanie modyfikacji bez restartu serwera. Plugin dostarcza domyślne pakiety językowe (polski i angielski), ale administrator może dowolnie rozszerzać bazę o kolejne tłumaczenia lub nadpisywać poszczególne klucze dla wybranych gildii.

3.1 Architektura i18n

3.1.1 Katalog `lang/` — struktura plików językowych

Wszystkie pliki językowe pluginu przechowywane są w katalogu `plugins/FractionGuildClans/lang/`. Przy pierwszym uruchomieniu plugin automatycznie generuje domyślne pakiety językowe: `pl_PL.yml` (polski) oraz `en_US.yml` (angielski). Administrator może dodawać kolejne pliki zgodnie ze standardem nazewnictwa `LL_CC.yml`, gdzie `LL` to dwuliterowy kod języka zgodny z ISO 639-1, a `CC` to dwuliterowy kod kraju zgodny z ISO 3166-1 alfa-2.

Przykłady poprawnych nazw plików to: `de_DE.yml` (niemiecki), `es_ES.yml` (hiszpański), `fr_FR.yml` (francuski), `cs_CZ.yml` (czeski) czy `ru_RU.yml` (rosyjski). Plugin wczytuje wszystkie pliki `.yml` z katalogu `lang/` podczas startu serwera oraz przy każdym wywołaniu komendy hot-reload. Nazwa pliku (bez rozszerzenia) stanowi identyfikator języka używany w konfiguracji oraz w komendach administracyjnych.

3.1.2 Format YAML z hierarchicznymi kluczami — przykład struktury

Każdy plik językowy to dokument YAML z hierarchiczną strukturą kluczy. Schemat klucza przyjmuje postać `kategoria.podkategoria.klucz`, co zapewnia logiczne grupowanie powiązanych komunikatów i eliminuje konflikty nazw między różnymi modułami. Na przykład wszystkie komunikaty związane z tworzeniem gildii znajdują się pod ścieżką `guild.create.*`, a komunikaty dotyczące jaja gildii pod `egg.*`.

Poniższy przedstawia fragment kompletnego pliku `pl_PL.yml` demonstrujący strukturę hierarchiczną, placeholder, prefixy oraz wsparcie dla formatowania MiniMessage i legacy:

```

# plugins/FractionGuildClans/lang/pl_PL.yml
# Plik jezykowy – Polski (Polska)
# Format: MiniMessage (<color>) + legacy (&k) – wspierane jednocześnie

prefix:
  info: "<dark_gray>[<aqua>Gildia<dark_gray>] <gray>"
  error: "<dark_gray>[<red>Gildia<dark_gray>] <red>"
  success: "<dark_gray>[<green>Gildia<dark_gray>] <green>"
  warning: "<dark_gray>[<yellow>Gildia<dark_gray>] <yellow>"

guild:
  create:
    success: "<green>Gratulacje!</green> Stworzyles gildie <aqua>{guild}</aqua> z tagiem <yellow>[{tag}]</yellow>!"
    error_name_taken: "Gildia o nazwie <red>{guild}</red> juz istnieje! Wybierz inna nazwe."
    error_tag_taken: "Tag <red>[{tag}]</red> jest juz zajety!"
    error_name_invalid: "Nazwa gildii zawiera niedozwolone znaki. Uzyj tylko liter i cyfr."
    error_items: "<red>Brakuje ci itemow do stworzenia gildii!</red> Potrzebujesz: <yellow>{items}</yellow>"
    error_location: "<red>Nie mozesz stworzyc gildii w tym miejscu.</red> Zbyt blisko innej gildii (min. <yellow>{radius}</yellow> blokow)."
    error_world: "<red>Tworzenie gildii jest zabronione w tym swiecie.</red>"
  invite:
    sent: "<green>Zaproszenie wyslane do gracza <yellow>{player}</yellow>!"
    received: "<green>Otrzymales zaproszenie do gildii <aqua>{guild}</aqua> od <yellow>{sender}</yellow>."
    accept_hover: "Kliknij aby dolaczyc do gildii {guild}!"
    error_already_member: "<red>Gracz {player} jest juz czlonkiem twojej gildii!</red>"
    error_has_guild: "<red>Gracz {player} nalezy juz do innej gildii.</red>"
    error_self: "<red>Nie mozesz zaprosic samego siebie!</red>"
  join:
    success: "<green>Dolaczyles do gildii <aqua>{guild}</aqua>!"
    broadcast: "<green>Gracz <yellow>{player}</yellow> dolaczyl do gildii!"
  leave:
    success: "<yellow>Opuściles gilde <aqua>{guild}</aqua>."
    broadcast: "<yellow>Gracz <red>{player}</red> opuscil gildie."
  disband:
    success: "<red>Gildia <aqua>{guild}</aqua> zostala rozwiazana."
    broadcast: "<dark_red>Gildia <red>{guild}</red> przestala istniec!"

egg:
  damaged: "<red><b>UWAGA!</b></red> Jajo twojej gildii zostalo uszkodzone! Pozostalo <yellow>{lives}/{max_lives}</yellow> zyc."
  destroyed: "<dark_red><b>OSTRZEZENIE!</b></dark_red> Jajo gildii <aqua>{guild}</aqua> zostalo zniszczone! Gildia upadnie za <yellow>{time}</yellow>."
  repaired: "<green>Jajo gildii zostalo naprawione!</green> Aktualny stan: <green>{lives}/{max_lives}</green>."

```

Struktura pliku odzwierciedla podział funkcjonalny pluginu. Klucz najwyższego poziomu `prefix` definiuje cztery kategorie prefiksów wiadomości (`info`, `error`, `success`, `warning`), które plugin automatycznie dołącza do komunikatów na podstawie kontekstu wywołania. Każdy podklucz (`guild.create` , `guild.invite` , `egg`) grupuje komunikaty powiązane tematycznie, co ułatwia nawigację podczas edycji i redukuje ryzyko przypadkowej modyfikacji niepowiązanych komunikatów.

3.1.3 System fallback — brak klucza = fallback do en_US, potem do raw key

Mechanizm fallback (awaryjnego zastępstwa) gwarantuje, że plugin nigdy nie zwróci pustej wiadomości, nawet gdy w aktywnym pliku językowym brakuje zdefiniowanego klucza. Proces rozwiązywania klucza przebiega w trzech etapach:

1. **Wyszukiwanie w aktywnym języku** — plugin sprawdza obecność klucza w pliku językowym skonfigurowanym dla serwera (np. `pl_PL.yml`). Jeśli klucz istnieje, zwracana jest jego wartość.
2. **Fallback do języka angielskiego** — jeśli klucz nie zostanie odnaleziony w aktywnym tłumaczeniu, plugin przeszukuje plik `en_US.yml` . Mechanizm ten zapewnia kompatybilność wsteczną i zapobiega "znikaniu" komunikatów po dodaniu nowych funkcji w aktualizacji pluginu.
3. **Raw key jako ostatnia deska ratunku** — gdy klucz nie występuje ani w aktywnym języku, ani w angielskim fallbacku, plugin zwraca sam identyfikator klucza (np. `guild.create.error_items`). Dzięki temu administrator lub deweloper może natychmiast zidentyfikować brakujący wpis i uzupełnić tłumaczenie.

Taka architektura eliminuje ryzyko awarii komunikacyjnej po aktualizacji pluginu. Dodanie nowego klucza językowego w kodzie nie wymaga natychmiastowego aktualizowania wszystkich plików tłumaczeń — wystarczy zdefiniować go w `en_US.yml` , a pozostałe języki przejmą wartość fallbacku do momentu przetłumaczenia.

3.2 Formatowanie Wiadomości

3.2.1 Wsparcie MiniMessage (`<color>` , `<bold>` , `<gradient>`) i legacy (`&a` , `&l`)

Plugin w wersji 2.0 wspiera dwa równoległe systemy formatowania tekstu, co zapewnia kompatybilność zarówno z nowoczesnymi bibliotekami Adventure, jak i ze starszymi konwencjami formatowania znanemu z silnika Bukkit.

MiniMessage to format oparty na tagach XML-styl, obsługiwany natywnie przez bibliotekę Adventure (używaną od Minecraft 1.16+). Tagi MiniMessage oferują zaawansowane możliwości formatowania, w tym gradienty kolorów, przejścia tonalne, klikalne elementy tekstowe (click events), akcje najechania (hover events) oraz efekty typu rainbow czy reset. Przykład użycia: `<gradient:green:aqua>Witaj w gildii!` `</gradient>` .

Legacy format (`&k` , `&a` , `&l`) to tradycyjny system kodów kolorów oparty na znaku `&` (lub `$` w formacie silnika). Format ten zachowany jest dla kompatybilności wstecznej i ułatwia migrację konfiguracji ze starszych wersji pluginu lub innych pluginów gildyjnych.

Poniższa tabela przedstawia porównanie obu systemów formatowania wraz z przykładami zastosowania:

Cecha	MiniMessage (<code><tag></code>)	Legacy (<code>&kod</code>)	Komentarz
Kolory podstawowe	<code><red></code> , <code><aqua></code> , <code><green></code>	<code>&c</code> , <code>&b</code> , <code>&a</code>	MiniMessage używa pełnych nazw kolorów, legacy — jednoliterowych kodów.
Pogrubienie	<code><bold>tekst</bold></code>	<code>&ltekst</code>	MiniMessage wymaga zamknięcia tagu, legacy stosuje przełącznik.
Kursywa	<code><italic>tekst</italic></code>	<code>&otekst</code>	Obie notacje osiągają identyczny efekt wizualny.
Podkreślenie	<code><underlined>tekst</underlined></code>	<code>&ntekst</code>	Wsparcie dla obu systemów w każdym komunikacie.
Gradient	<code><gradient:red:blue>tekst</gradient></code>	Niedostępne	Gradienty to unikalna cecha MiniMessage — legacy nie oferuje tej funkcji.
Rainbow	<code><rainbow>tekst</rainbow></code>	Niedostępne	Efekt tęczy automatycznie przypisuje spektrum kolorów do tekstu.
Click event	<code><click:run_command:/cmd>tekst</click></code>	Niedostępne	Klikalne elementy tekstowe działają tylko w MiniMessage.
Hover event	<code><hover:show_text:'tresc'>tekst</hover></code>	Niedostępne	Tekst wyświetlany po najechaniu kursorem — tylko MiniMessage.
Reset	<code><reset></code>	<code>&r</code>	Przywraca domyślny styl tekstu.
Czytelność	Wysoka (znaczniki samoopisujące)	Niska (kody wymagają zapamiętania)	MiniMessage jest znacznie czytelniejszy dla nowych użytkowników.
Zagnieżdżanie	Pełne wsparcie	Ograniczone	MiniMessage pozwala na dowolne zagnieżdżanie tagów.

Plugin analizuje każdą wiadomość i automatycznie wykrywa użyty system formatowania. Jeśli w tekście występują znaki `<` otaczające poprawne tagi MiniMessage, tekst przetwarzany jest przez parser Adventure. W przeciwnym razie plugin stosuje konwersję legacy do komponentów Adventure. Nie jest konieczne jawne deklarowanie systemu formatowania — plugin rozpoznaje go dynamicznie dla każdej wiadomości osobno. Dzięki temu administrator może mieszać obie notacje w ramach jednego pliku językowego, choć w praktyce zaleca się stosowanie jednego systemu w obrębie pojedynczego komunikatu dla zachowania spójności.

3.2.2 Placeholdery w wiadomościach

System plików językowych udostępnia zestaw placeholderów, które plugin dynamicznie podmienia na wartości środowiskowe w momencie wyświetlania komunikatu. Placeholdery zapisywane są w nawiasach klamrowych: `{placeholder}`. Poniższa tabela zawiera kompletny spis dostępnych placeholderów wraz z typem danych i kontekstem użycia:

Placeholder	Typ danych	Kontekst użycia	Przykład wartości
<code>{player}</code>	String	Nazwa gracza będącego podmiotem akcji.	<code>Notch</code> , <code>Herobrine</code>
<code>{guild}</code>	String	Pełna nazwa gildii.	<code>Srebrne Lwy</code> , <code>Shadow Clan</code>
<code>{tag}</code>	String	Tag gildii (skrótowa nazwa w nawiasach).	<code>[SL]</code> , <code>[SC]</code>
<code>{tag_color}</code>	String	Tag gildii z zastosowanym kolorem z konfiguracji.	<code><dark_red>[SC]</code> <code></dark_red></code>
<code>{points}</code>	Integer	Aktualna liczba punktów rankingowych gildii.	<code>15420</code> , <code>890</code>
<code>{level}</code>	Integer	Poziom rozwoju gildii.	<code>5</code> , <code>12</code>
<code>{lives}</code>	Integer	Aktualna liczba żyć jaja gildii.	<code>2</code> , <code>0</code>
<code>{max_lives}</code>	Integer	Maksymalna liczba żyć jaja gildii (domyślnie 3).	<code>3</code>
<code>{radius}</code>	Integer	Promień cuboidu gildii w blokach.	<code>50</code> , <code>100</code>
<code>{members}</code>	Integer	Aktualna liczba członków gildii.	<code>8</code> , <code>15</code>
<code>{max_members}</code> <code>}</code>	Integer	Maksymalna liczba członków dla danego poziomu.	<code>10</code> , <code>20</code>

<code>{balance}</code>	Double	Stan konta banku gildii (waluta EcoCore).	15420.50 , 0.00
<code>{time}</code>	String	Sformatowany czas (np. do końca wygaśnięcia gildii).	14d 3h 22m
<code>{sender}</code>	String	Nazwa gracza inicjującego akcję (zaproszenie, wyrzucenie).	Steve
<code>{target}</code>	String	Nazwa gracza będącego celem akcji.	Alex
<code>{amount}</code>	Integer/Double	Wartość liczbową (np. kwota transakcji).	500 , 1000.0
<code>{item}</code>	String	Nazwa przedmiotu (polska lub angielska w zależności od języka).	64x Diament , 32x Złoto
<code>{price}</code>	Double	Cena operacji w walucie serwera.	10000.0
<code>{rank}</code>	Integer	Pozycja gildii w rankingu globalnym.	3 , 15
<code>{position}</code>	String	Współrzędne lokalizacji sformatowane jako X Y Z.	150 64 -220

Placeholdery podlegają walidacji w czasie rzeczywistym. Jeśli plugin nie może rozwiązać wartości danego placeholdera (np. `{guild}` w kontekście gracza bez gildii), zwraca ciąg "N/A" lub wartość skonfigurowaną w kluczu `placeholders.not_available`. Nieprawidłowe placeholdery (nieistniejące nazwy) pozostawiane są w oryginalnej formie `{placeholder}` jako sygnał dla administratora o błędzie w definicji komunikatu.

3.2.3 Kategorie wiadomości: `prefix.info`, `prefix.error`, `prefix.success`, `prefix.warning`

System prefiksów kategoryzowanych automatycznie dołącza odpowiedni nagłówek do każdej wiadomości na podstawie jej typu semantycznego. Prefiksy definiowane są w sekcji `prefix:` pliku językowego i mogą zawierać dowolne formatowanie MiniMessage oraz legacy. Podczas wysyłania komunikatu plugin wywołuje metodę `getMessage(key, type)`, gdzie parametr `type` określa, który prefiks zostanie dołączony przed treścią główną.

`prefix.info` — neutralne komunikaty informacyjne (np. lista członków gildii, status konta). Domyślnie używa kolorystyki szaro-niebieskiej, która nie przyciąga nadmiernej uwagi i jest czytelna w różnych warunkach oświetlenia w grze.

prefix.error — komunikaty błędów krytycznych uniemożliwiających wykonanie akcji (np. brak uprawnień, niewystarczające środki, zablokowana lokalizacja). Kolorystyka czerwona z mocnym kontrastem względem tła czatu. Prefiks ten używany jest również przy walidacji argumentów komend.

prefix.success — potwierdzenia pomyślnego wykonania operacji (np. utworzenie gildii, dołączenie do gildii, naprawa jaja). Zielona kolorystyka zapewnia natychmiastową pozytywną informację zwrotną dla gracza.

prefix.warning — ostrzeżenia o sytuacjach nietypowych lub niebezpiecznych (np. zbliżanie się do granicy cuboidu, niskie życie jaja, wygaśnięcie gildii w ciągu 24 godzin). Żółta kolorystyka sygnalizuje konieczność zwrócenia uwagi bez anulowania akcji.

3.3 Konfiguracja i Hot-Reload

3.3.1 Tworzenie własnego tłumaczenia — krok po kroku z przykładem

Proces tworzenia niestandardowego pakietu językowego składa się z czterech etapów, które nie wymagają restarty serwera dzięki mechanizmowi hot-reload opisanemu w sekcji 3.3.2.

Krok 1: Utworzenie kopii bazowego pliku. Skopiuj domyślny plik `en_US.yml` do nowego pliku o nazwie zgodnej ze standardem `LL_CC.yml`, np. `cs_CZ.yml` dla czeskiego tłumaczenia. Użycie `en_US.yml` jako bazy zapewnia kompletność wszystkich kluczy — nie istnieje ryzyko pominięcia klucza podczas tłumaczenia.

Krok 2: Przetłumaczenie wartości. Otwórz nowy plik w edytorze tekstowym z wsparciem dla UTF-8 (zalecane: VS Code, Notepad++, IntelliJ IDEA). Przetłumacz wartości po dwukropku, zachowując nienaruszoną strukturę kluczy. Nie modyfikuj nazw kluczy — zmiana `guild.create.success` na `guild.create.uspesny` spowoduje nieodnalezienie komunikatu przez plugin.

Krok 3: Weryfikacja YAML. Upewnij się, że struktura YAML pozostaje poprawna: wcięcia muszą używać spacji (nie tabulatorów), dwukropki oddzielają klucz od wartości, a cudzysłowy opcjonalnie otaczają wartości zawierające znaki specjalne. Nieprawidłowa składnia YAML spowoduje błąd parsowania i fallback do angielskiego pliku językowego.

Krok 4: Aktywacja tłumaczenia. Zapisz plik w katalogu `plugins/FractionGuildClans/lang/` i wykonaj komendę `/guild admin lang reload` (patrz sekcja 3.3.2). Następnie ustaw nowy język jako domyślny w `config.yml` (`lang.default: cs_CZ`) lub pozwól graczom wybrać go indywidualnie komendą `/guild lang <kod>`.

Poniższy przykład przedstawia kompletny, przetłumaczony fragment pliku `cs_CZ.yml` (czeski) demonstrujący zachowanie struktury kluczy, formatowania i placeholderów:

```

# plugins/FractionGuildClans/lang/cs_CZ.yml
# Uživatelský překlad – Cestina (Česká republika)
# Zaklad: en_US.yml | Přeloženo: guild.create a egg

prefix:
  info: "<dark_gray>[<aqua>Gildie<dark_gray>] <gray>"
  error: "<dark_gray>[<red>Gildie<dark_gray>] <red>"
  success: "<dark_gray>[<green>Gildie<dark_gray>] <green>"
  warning: "<dark_gray>[<yellow>Gildie<dark_gray>] <yellow>"

guild:
  create:
    success: "<green>Gratulujeme!</green> Vytvoril jsi gildii <aqua>{guild}</aqua> s tagem <yellow>[{tag}]</yellow>!"
    error_name_taken: "Gildie s názvem <red>{guild}</red> již existuje! Vyber si jiný název."
    error_tag_taken: "Tag <red>[{tag}]</red> je již obsazený!"
    error_name_invalid: "Název gildie obsahuje nepovolene znaky. Použij pouze písmena a čísla."
    error_items: "<red>Nemáš dostatek předmětů pro vytvoření gildie!</red> Potřebuješ: <yellow>{items}</yellow>"
    error_location: "<red>Zde nemůžeš vytvořit gildii.</red> Právě jsi blízko jiné gildie (min. <yellow>{radius}</yellow> bloku)."
    error_world: "<red>Vytváření gildii je v tomto světě zakázáno.</red>"
  invite:
    sent: "<green>Pozvánka odeslána hráči <yellow>{player}</yellow>!"
    received: "<green>Obdržel jsi pozvánku do gildie <aqua>{guild}</aqua> od <yellow>{sender}</yellow>."
    accept_hover: "Klikni pro připojení do gildie {guild}!"
    error_already_member: "<red>Hráč {player} je již členem tvé gildie!</red>"
    error_has_guild: "<red>Hráč {player} již patří do jiné gildie.</red>"
    error_self: "<red>Nemůžeš pozvat sám sebe!</red>"
  join:
    success: "<green>Připojil jsi se do gildie <aqua>{guild}</aqua>!"
    broadcast: "<green>Hráč <yellow>{player}</yellow> se připojil do gildie!"
  leave:
    success: "<yellow>Opustil jsi gildii <aqua>{guild}</aqua>."
    broadcast: "<yellow>Hráč <red>{player}</red> opustil gildii."
  disband:
    success: "<red>Gildie <aqua>{guild}</aqua> byla rozpuštěna."
    broadcast: "<dark_red>Gildie <red>{guild}</red> přestala existovat!"

egg:
  damaged: "<red><b>POZOR!</b></red> Vajíčko tvé gildie bylo poškozeno! Zbývá <yellow>{lives}/{max_lives}</yellow> života."
  destroyed: "<dark_red><b>VAROVANI!</b></dark_red> Vajíčko gildie <aqua>{guild}</aqua> bylo zničeno! Gildie padne za <yellow>{time}</yellow>."
  repaired: "<green>Vajíčko gildie bylo opraveno!</green> Aktuální stav: <green>{lives}/{max_lives}</green>."

```


Tłumaczenie zachowuje identyczną hierarchię kluczy co plik źródłowy `en_US.yml`, co gwarantuje poprawne działanie fallbacku i eliminuje błędy związane z literówkami w nazwach kluczy. Placeholderzy (`{guild}`, `{tag}`, `{lives}` itd.) pozostają nieprzetłumaczone — plugin podmienia je na wartości środowiskowe niezależnie od języka docelowego. Formatowanie MiniMessage (`<green>`, `<bold>`) również pozostaje identyczne, co zapewnia spójny wygląd komunikatów we wszystkich językach.

3.3.2 Hot-reload języka bez restartu — `/guild admin lang reload`

Mechanizm hot-reload pozwala na dynamiczne przeładowanie wszystkich plików językowych bez konieczności restartu serwera Minecraft. Funkcja ta jest niezbędna w środowiskach produkcyjnych, gdzie restart oznacza utratę graczy i przerwę w dostępności serwera.

Komenda `/guild admin lang reload` (wymagana permisja: `fgc.admin.lang.reload`) wykonuje następujący ciąg operacji:

1. **Unieważnienie cache'u** — plugin czyści wewnętrzną pamięć podręczną tłumaczeń, zmuszając system do ponownego wczytania plików z dysku.
2. **Ponowne parsowanie YAML** — wszystkie pliki `.yml` w katalogu `lang/` są ponownie analizowane pod kątem poprawności składniowej. Błędy parsowania raportowane są do konsoli z dokładną lokalizacją (nazwa pliku i numer linii).
3. **Walidacja kluczy** — system porównuje klucze w każdym pliku z bazą `en_US.yml` i generuje ostrzeżenia o brakujących kluczach, które zostaną obsłużone mechanizmem fallback.
4. **Aktywacja zmian** — nowe tłumaczenia natychmiast dostępne są dla wszystkich graczy na serwerze. Wiadomości wysyłane po przeładowaniu używają już zaktualizowanych definicji.

Komenda zwraca podsumowanie operacji do nadawcy: liczbę wczytanych plików językowych, liczbę wykrytych błędów YAML oraz liczbę brakujących kluczy per plik. W przypadku krytycznego błędu parsowania danego pliku (np. nieprawidłowe wcięcia) plugin pomija ten plik i kontynuuje wczytywanie pozostałych, co zapobiega całkowitej utracie komunikacji.

3.3.3 Tłumaczenia per-gildia — `custom prefix`, powitanie, komunikaty

System plików językowych w wersji 2.0 wprowadza możliwość nadpisywania wybranych komunikatów na poziomie pojedynczej gildii. Lider gildii (lub gracz z uprawnieniem `fgc.guild.lang.custom`) może dostosować prefix gildii, powitanie dla nowych członków oraz komunikaty wewnętrzne bez wpływu na globalne tłumaczenia serwera. Mechanizm ten realizowany jest poprzez sekcję `custom_lang` w danych gildii, która przechowywana jest w bazie danych (SQLite/MySQL) i ładowana do pamięci podczas startu serwera.

Aby ustawić customowe tłumaczenie, lider gildii używa komendy `/guild setlang <klucz> <wartosc>`, np. `/guild setlang prefix "<dark_red>[WATAHA]</dark_red> <gray>"`. System

waliduje poprawność formatowania MiniMessage przed zapisem — nieprawidłowe tagi są odrzucane z komunikatem błędu. Następująca tabela zawiera pełen spis kluczy dostępnych do nadpisania na poziomie gildii:

Klucz	Domyślna wartość (fallback)	Opis	Uprawnienie
<code>prefix</code>	Globalny <code>prefix.info</code>	Prefix wyświetlany przed wiadomościami gildii w czacie wewnętrznym.	<code>fgc.guild.lang.prefix</code>
<code>welcome</code>	<code>guild.join.welcome</code>	Wiadomość powitalna wysyłana do nowo przyjętego członka.	<code>fgc.guild.lang.welcome</code>
<code>motd</code>	<code>""</code> (puste)	Wiadomość dnia wyświetlana członkom po zalogowaniu (opcjonalna).	<code>fgc.guild.lang.motd</code>
<code>rank_name</code>	<code>""</code> (puste)	Nazwa rangi wyświetlana zamiast domyślnej nazwy gildii w rankingu.	<code>fgc.guild.lang.rank_name</code>
<code>chat_format</code>	<code>global.chat.format</code>	Format wiadomości na czacie gildii; obsługuje placeholderzy <code>{player}</code> , <code>{message}</code> .	<code>fgc.guild.lang.chat_format</code>
<code>join_broadcast</code>	<code>guild.join.broadcast</code>	Wiadomość broadcastowana przy dołączeniu nowego członka (pusta = brak broadcastu).	<code>fgc.guild.lang.broadcasts</code>
<code>leave_broadcast</code>	<code>guild.leave.broadcast</code>	Wiadomość broadcastowana przy opuszczeniu gildii przez członka (pusta = brak broadcastu).	<code>fgc.guild.lang.broadcasts</code>
<code>egg_damaged</code>	<code>egg.damaged</code>	Ostrzeżenie o uszkodzeniu jaja — można dodać własne wezwanie do broni.	<code>fgc.guild.lang.egg</code>
<code>egg_destroyed</code>	<code>egg.destroyed</code>	Komunikat o zniszczeniu jaja — pozwala na np. wezwanie sojuszników.	<code>fgc.guild.lang.egg</code>
<code>war_declaration</code>	<code>war.declare.default</code>	Formuła deklaracji wojny wysyłana do przeciwnej gildii.	<code>fgc.guild.lang.war</code>
<code>ally_request</code>	<code>ally.request.default</code>	Formuła zaproszenia sojuszniczego wysyłana do innej gildii.	<code>fgc.guild.lang.ally</code>
<code>enemy_announce</code>	<code>enemy.announce.default</code>	Komunikat ogłaszający wrogość względem innej gildii.	<code>fgc.guild.lang.enemy</code>

<code>guild_description</code>	<code>""</code>	Opis gildii wyświetlany w GUI oraz komendzie <code>/guild info</code> .	<code>fgc.guild.lang.description</code>
<code>cuboid_enter</code>	<code>cuboid.enter.default</code>	Wiadomość wyświetlana graczowi przy wejściu na teren gildii.	<code>fgc.guild.lang.cuboid</code>
<code>cuboid_leave</code>	<code>cuboid.leave.default</code>	Wiadomość wyświetlana przy opuszczeniu terenu gildii.	<code>fgc.guild.lang.cuboid</code>
<code>death_message</code>	<code>player.death.default</code>	Niestandardowa wiadomość śmierci członka gildii (widoczna dla sojuszników).	<code>fgc.guild.lang.death</code>
<code>kill_message</code>	<code>player.kill.default</code>	Niestandardowa wiadomość o zabiciu przeciwnika przez członka gildii.	<code>fgc.guild.lang.kill</code>
<code>levelup_announce</code>	<code>guild.levelup.default</code>	Komunikat ogłaszający awans gildii na wyższy poziom.	<code>fgc.guild.lang.levelup</code>
<code>treasury_status</code>	<code>guild.treasury.default</code>	Wiadomość podsumowująca stan skarbca gildii po transakcji.	<code>fgc.guild.lang.treasury</code>

Mechanizm tłumaczeń per-gildia oparty jest na warstwowym systemie rozwiązywania kluczy. Przy wywołaniu komunikatu plugin sprawdza najpierw, czy gildia (jeśli kontekst wiadomości jest związany z gildią) posiada nadpisanie dla danego klucza. Jeśli tak — używa wartości customowej. Jeśli nie — przechodzi do standardowego fallbacku: aktywny język gracza, domyślny język serwera, `en_US.yml`, raw key. Taka architektura zapewnia pełną elastyczność przy jednoczesnym zachowaniu spójności — nieprzetłumaczone klucze zawsze mają wartość zastępczą, a nadpisanie per-gildia są całkowicie opcjonalne.

Nadpisanie per-gildia przechowywane są w tabeli `guild_custom_lang` bazy danych (pole `lang_overrides` typu TEXT w formacie JSON). Administrator serwera może przeglądać i edytować nadpisanie dowolnej gildii komendą `/guild admin lang guild <tag> <klucz> <wartosc>`, co pozwala na moderację treści customowych i zapobieganie nadużyciom (np. nieodpowiednich treści w prefixach). Przy usunięciu gildii wszystkie jej nadpisanie językowe automatycznie usuwane są z bazy danych przez mechanizm cascade delete.

4. System Gildii

System Gildii stanowi centralny moduł rozgrywki serwera FactionsGuildCraft v2.0. Wbudowany jako samodzielny komponent w katalogu `modules/`, nie wymaga żadnych zewnętrznych pluginów typu Factions, SimpleClans ani FunnyGuilds. Architektura opiera się na własnej implementacji zarządzania terenami (cuboidami), wewnętrznym systemie rang oraz modularnym silniku relacji międzygildyjnych.

Wersja 2.0 wprowadza pięć poziomów rang hierarchicznych (wraz z rangą **Co-Lidera**, inspirowaną rozwiązaniem z `FactionsUUID`), konfigurowalne wymagania założycielskie oraz pełną internacjonalizację komunikatów przez pliki językowe w katalogu `lang/`.

Moduł gildii komunikuje się opcjonalnie z **EcoCore** (własnym systemem ekonomicznym autora) w celu obsługi kosztów walutowych, lecz podstawowa funkcjonalność działa niezależnie od jakiegokolwiek zewnętrznego pluginu ekonomicznego. Wszystkie komunikaty wyświetlane graczom przechodzą przez warstwę **LangManager**, co umożliwia hot-reload tłumaczeń bez restartu serwera oraz pełne wsparcie składni MiniMessage.

4.1 Zakładanie Gildii

4.1.1 Wymagane Itemy — Konfigurowalna Lista

Koszt założenia gildii definiuje się wyłącznie w pliku `modules/guild.yml`. Plugin nie zawiera wartości hardcoded; w przypadku braku pliku konfiguracyjnego ładowane są wartości domyślne z klasy `GuildModuleConfig`. Sekcja `foundation-cost` pozwala zdefiniować listę itemów, ich ilości, metadane (Custom Model Data, nazwy wyświetlane, lore) oraz opcjonalny koszt walutowy.

```
# modules/guild.yml – sekcja foundation-cost
foundation-cost:
  items:
    - material: DIAMOND_BLOCK
      amount: 16
      display-name: "&bBlok Diamentu"
    - material: EMERALD_BLOCK
      amount: 8
    - material: NETHERITE_INGOT
      amount: 4
      custom-model-data: 1
  # Koszt walutowy – aktywny tylko gdy moduł economy jest włączony
  # i EcoCore jest dostępny.
  currency-cost: 5000.0
```

Tabela poniżej przedstawia predefiniowaną listę itemów wymaganych do założenia gildii. Administrator serwera może dowolnie modyfikować tę listę — zmieniać ilości, dodawać itemy z tagami NBT lub usunąć wymagania całkowicie (pusta lista `items: []` oznacza brak kosztu materialnego).

Item	Domyślna Ilość	Przeznaczenie w Ekosystemie Serwera	Konfigurowalność
------	----------------	-------------------------------------	------------------

Diamond Block	16	Surowiec defensywny — symbol trwałości bazy, wymagany do craftingu beaconów i enchantów	Pełna: material, amount, display-name, lore, custom-model-data, enchanty
Emerald Block	8	Surowiec handlowy — reprezentuje kapitał gospodarczy, nawiązanie do villager trading	Pełna: jak powyżej
Netherite Ingot	4	Surowiec endgame'owy — wymóg progresji gracza do późnego etapu gry (Nether)	Pełna: jak powyżej; można zastąpić np. Mace (Heavy Core) w wersji 1.21
Gold Block	0 (opcjonalny)	Surowiec rezerwowy — używany jako dodatkowy wymóg rankingowy na niektórych serwerach	Pełna: domyślnie wyłączony, do włączenia przez admina

Filozofia kosztu założycielskiego wymaga od gracza udowodnienia zaangażowania na trzech płaszczyznach progresji — eksploracji (diamanty z głębin), ekonomii (emeraldy z handlu) oraz endgame'u (netherite z Netheru). Taka struktura zapewnia, że gildie zakładają wyłącznie gracze z doświadczeniem, a nie konta alternatywne (alts). Opcjonalne pole `currency-cost` działa jako dodatkowy filtr: gdy moduł economy jest aktywny i EcoCore zgłasza dostępność, gracz musi posiadać określoną ilość waluty na koncie. Jeśli EcoCore nie jest podłączony lub moduł economy jest wyłączony, pole `currency-cost` jest ignorowane przez parser — nie powoduje błędu, a jedynie loguje komunikat na poziomie `DEBUG`.

Weryfikacja itemów odbywa się przez klasę `FoundationCostValidator`, która iteruje po ekwipunku gracza (łącznie z shulker boxami, jeśli opcja `search-shulkers: true` jest włączona) i sprawdza zgodność materiału, ilości oraz metadanych. System obsługuje również itemy z pluginów customowych (np. `ItemsAdder`, `Oraxen`) poprzez identyfikację po `material` z prefiksem namespace.

4.1.2 Wymagania Gracza

Przed wykonaniem `/guild create` plugin weryfikuje warunki kwalifikacyjne z sekcji `foundation-requirements` w `modules/guild.yml`. Spełnienie wszystkich wymagań jest obligatoryjne; w przypadku niespełnienia któregośkolwiek gracz otrzymuje komunikat z pliku językowego.

```
# modules/guild.yml – sekcja foundation-requirements
foundation-requirements:
  min-playtime-hours: 24
  min-player-level: 15
  cooldown-hours-after-leave: 48
  max-guilds-per-player: 1
  blocked-worlds:
    - "world_spawn"
    - "world_arena"
```

Proces weryfikacji przebiega sekwencyjnie (short-circuit evaluation):

1. **Czas gry** — odczyt z wewnętrznego trackera lub statystyk Minecraft (`STAT_PLAY_ONE_MINUTE`). Domyślnie wymagane **24 godziny**.
2. **Poziom gracza** — poziom XP postaci; domyślnie **15**, co gwarantuje przejście przez podstawowy etap progresji.
3. **Cooldown po opuszczeniu gildii** — odczekanie **48 godzin** przed założeniem nowej, co zapobiega tworzeniu gildii-fantomów.
4. **Limit gildii** — domyślnie **1 gildia** na gracza; zwiększenie włącza tryb multi-gildyjny.
5. **Blokada światów** — komenda zablokowana na światach z listy `blocked-worlds` (spawn, areny).

Dzięki short-circuit evaluation gracz otrzymuje informację o pierwszym napotkanym niespełnionym warunku, co zapewnia precyzyjny komunikat bez przytłaczania listą wszystkich błędów.

4.1.3 Proces Zakładania

Założenie gildii odbywa się przez `/guild create <nazwa> <tag> [kolor]` :

```
/guild create MroczneWilki MW &8
```

Krok 1 — Walidacja parametrów. Nazwa: 3–24 znaki (litera, cyfry, spacje, podkreślenia). Tag: 2–6 znaków alfanumerycznych. Kolor (opcjonalny): kod formatowania Minecraft lub tag MiniMessage; domyślnie wartość z `default-guild-color` .

Krok 2 — Pobranie kosztu. Itemy usuwane są z ekwipunku. Jeśli aktywny jest moduł economy z EcoCore, z konta odejmowana jest kwota `currency-cost` . Operacja jest atomowa — błąd podczas pobierania waluty anuluje całą transakcję.

Krok 3 — Tworzenie rekordu gildii. W bazie danych (SQLite/MySQL) tworzony jest rekord z UUID gildii, nazwą, tagiem, kolorem, timestamp'em oraz referencją do założyciela.

Krok 4 — Przypisanie rangi Lidera. Założyciel otrzymuje rangę **LIDER** z pełnym zestawem uprawnień. Jest to jedyna ranga, której nie można zmienić przez `/guild promote` — przekazanie wymaga `/guild leader <gracz>` .

Krok 5 — Claimowanie startowego cuboida. Gildia otrzymuje startowy cuboid o promieniu `starting-radius` (domyślnie 16 bloków). System używa wbudowanego `CuboidManager` , niezależnego od WorldGuard/WorldEdit. Współrzędne zapisywane są jako para punktów z identyfikatorem świata.

Krok 6 — Powiadomienie. Gracz otrzymuje komunikat z klucza `guild.create.success` z pliku językowego. Przy włączonej opcji `announce-globally: true` na czacie pojawia się ogłoszenie o powstaniu nowej gildii.

Opcjonalny schemat startowy. Jeśli w katalogu `schematics/` znajduje się plik zdefiniowany w `starting-schematic`, plugin może go automatycznie wkleić w środku claimu (wymaga `paste-starting-schematic: true`). Wklejanie zachowuje ograniczenia Y-level (`schematic-min-y`, `schematic-max-y`), co zapobiega niszczeniu naturalnego terenu.

4.2 System Rang

4.2.1 Hierarchia Rang i Upewnienia

System rang składa się z **pięciu poziomów hierarchicznych**, co stanowi rozszerzenie klasycznego modelu o pozycję **Co-Lidera** z `FactionsUUID`. Każda ranga posiada zestaw uprawnień w sekcji `ranks` pliku `modules/guild.yml`, które administrator może dowolnie modyfikować oraz dodawać nowe rangi.

Ranga	Poziom	Zakres Upewnienia	Ograniczenia
Lider	5 (najwyższy)	Pełna kontrola: rozwiązanie gildii, przekazanie przywództwa, modyfikacja rang, zarządzanie relacjami, edycja cuboidów, dostęp do vault, zmiana ustawień (opis, MOTD, flagi), banowanie, zarządzanie economy.	Brak ograniczeń; jedyna ranga nieusuwalna i niepodlegająca degradacji.
Co-Lider	4	Zapraszanie/wyrzucanie graczy (nie Lidera), zarządzanie relacjami, edycja cuboidów (claim/unclaim, rozszerzanie), dostęp do vault, zmiana ustawień gildii, banowanie niższych rang, nadawanie tytułów, kickowanie do Członka włącznie.	Nie może: rozwiązać gildii, przekazać przywództwa, degradować Lidera, zmieniać struktury rang w configu.
Moderator	3	Zapraszanie nowych graczy (automatycznie Rekrut), wyrzucanie Członka/Rekruta, nadawanie tytułów niższym rangom, podstawowe claimowanie, dostęp do vault (odczyt/zapis zależnie od konfiguracji), podgląd logu aktywności.	Nie może: zarządzać relacjami, rozszerzać cuboidów, banować, zmieniać ustawień gildii.
Członek	2	Budowanie i niszczenie na terenie gildii (przy fladze <code>build</code>), korzystanie z vault (jeśli dozwolone), udział w wojnach i sojuszach, widoczność tagu, czat gildii (<code>/guild chat</code>), użycie warpów.	Nie może: zapraszać/wyrzucać graczy, claimować terenu, modyfikować ustawień, nadawać tytułów.

Rekrut	1 (najniższy)	Budowanie na terenie gildii (przy fladze <code>build-recruit</code>), widoczność tagu, czat gildii (ograniczony), korzystanie z warpów.	Nie może: niszczyć bloków krytycznych (obsidian, ender chest, skrzynie), otwierać vault, brać udziału w głosowaniach, wyrzucać innych.
---------------	------------------	--	--

Hierarchia 5-rangowa odwzorowuje realne struktury dowodzenia: Lider jako właściciel strategiczny, Co-Lider jako zastępca operacyjny, Moderator jako "oficer polowy", Członek jako rdzeń zespołu oraz Rekrut jako okres próbny. **Dziedziczenie uprawnień** jest domyślnie włączone: wyższe rangi automatycznie posiadają kompetencje niższych, chyba że ustawiono `inherit: false`. Pozwala to tworzyć modele specjalizacyjne, np. rangę "Skarbnik" z dostępem tylko do vault bez prawa budowania.

Zmiana rang odbywa się przez `/guild promote <gracz>` oraz `/guild demote <gracz>`, z wbudowanymi zabezpieczeniami: brak degradacji poniżej Rekruta i brak awansu powyżej Co-Lidera. Przekazanie przywództwa wymaga osobnej komendy.

4.2.2 System Banowania Graczy z Gildii

Mechanizm banowania (inspirowany FactionsUUID) umożliwia trwałe wykluczenie gracza z gildii. Zbanowany gracz nie może dołączyć ponownie, dopóki ban nie zostanie cofnięty.

Komendy systemu banowania:

```
/guild ban <gracz> [powod]      – Zbanowanie gracza
/guild unban <gracz>             – Zdjęcie bana
/guild banlist                   – Lista zbanowanych graczy
```

Proces banowania przebiega następująco:

1. **Weryfikacja uprawnień** — komenda sprawdza `guild.ban` (domyślnie: Lider, Co-Lider). Moderator może otrzymać to uprawnienie przez konfigurację.
2. **Weryfikacja celu** — nie można zbanować gracza z wyższą lub równą rangą.
3. **Wykonanie bana** — UUID zbanowanego gracza trafia do tabeli `guild_bans` z timestamp'em, powodem i UUID banującego. Jeśli zbanowany jest obecnym członkiem, automatycznie zostaje wyrzucony.
4. **Powiadomienie** — zbanowany gracz (jeśli online) otrzymuje komunikat z klucza `guild.ban.target-message`; pozostali członkowie widzą `guild.ban.announcement`.
5. **Cooldown dołączenia** — po zdjęciu bana gracz musi odczekać `unban-cooldown-hours` (domyślnie 24h).

Lista zbanowanych (`/guild banlist`) wyświetla się w formacie paginowanego czatu: nick, data bana, powód oraz banujący. Formatowanie obsługuje MiniMessage, w tym oznaczenie banów czasowych (jeśli

włączone).

4.2.3 Titles — Indywidualne Tytuły dla Członków

System tytułów pozwala Liderowi, Co-Liderowi i Moderatorowi nadać każdemu członkowi indywidualny tytuł wyświetlany przy tagu gildii. Tytuły wspierają pełną składnię MiniMessage — gradienty, efekty `<bold>`, `<italic>`, komponenty interaktywne.

```
/guild settitle <gracz> <tytuł>
```

Przykład:

```
/guild settitle Krolik <gradient:green:yellow>Mistrz Budowniczy
```

Tytuł przechowywany jest w bazie jako string MiniMessage i renderowany dynamicznie. W czacie gildii tytuł wstawiany jest przez placeholder `%title%` w szablonie zdefiniowanym w pliku językowym:

```
# lang/pl.yml
guild:
  chat:
    format: "<dark_gray>[<gold>%guild_tag%<dark_gray>] <gray>%title%
<white>%player_name%<gray>: %message%"
```

Brak tytułu zastępowany jest pustym stringiem lub wartością z `guild.title.default`. Maksymalna długość tytułu kontrolowana jest przez `title-max-length` (domyślnie 32 znaki).

4.3 Zarządzanie Gildią

4.3.1 Ustawienia Gildii

Każda gildia posiada zestaw konfigurowalnych atrybutów, modyfikowanych przez członków z odpowiednimi uprawnieniami. Wszystkie ustawienia przechowywane są w bazie danych.

Dostępne ustawienia:

- **Opis gildii** (`/guild description <tekst>`) — do 256 znaków, wspiera MiniMessage.
- **Powitanie** (`/guild welcome <tekst>`) — wiadomość wyświetlana nowym członkom przy dołączeniu.
- **MOTD** (`/guild motd <tekst>`) — wiadomość wyświetlana członkom przy logowaniu.
- **Flagi publiczne/prywatne** — boolean'y kontrolujące widoczność:
 - `is-public` — widoczność na liście publicznej (`/guild list`)

- `allow-join-requests` — możliwość wysyłania próśb o dołączenie bez zaproszenia
- `show-home` — widoczność lokalizacji bazy dla sojuszników

Flagi gildii działają w tandemie z wbudowanym systemem ochrony cuboidów (Rozdział 5). Każda gildia definiuje własne flagi terenowe: `build`, `destroy`, `use`, `pvp`, `mob-spawning`, `fire-spread`, `tnt-damage`, zarządzane natywnie przez `CuboidManager` bez zależności od WorldGuard.

4.3.2 Sojusze, Wojny, Zawieszenie Broni — System Relacji

System relacji implementuje cztery stany między gildiami, modelowane na podstawie FactionsUUID. Relacje determinują zachowanie PvP, widoczność na mapie oraz dostęp do terenów.

Relacja	Tag	Kolor Domyślny	Efekt PvP	Dostęp do Terenów	Widoczność na Mapie
Sojusz (Ally)	[AL LY]	Zielony (<green>)	Wyłączony — gracze sojuszniczych gildii nie mogą się zranić	Ograniczony: budowanie i używanie mechanizmów dozwolone; niszczenie i skrzynie zabronione	Pełna: pozycje baz i cuboidów sojuszników widoczne
Wróg (Enemy)	[EN EMY]	Czerwony (<red>)	Wymuszony włączony — obrażenia zawsze aktywne	Brak — traktowani jak outsiderzy	Ograniczona: widoczne tylko pozycje głównych baz, nie pełne cuboidy
Neutralny (Neutral)	[NE UTR AL]	Szary (<gray>)	Zgodny z ustawieniami świata i flagą pvp	Brak — brak dostępu do cudzych cuboidów	Brak — nie pojawiają się na mapie gildii
Zawieszenie Broni (Truce)	[TR UCE]	Żółty (<yellow>)	Wyłączony — jak w sojuszu, ale bez uprawnień terytorialnych	Brak — gracze traktowani jak outsiderzy, chronione PvP	Ograniczona: widoczna tylko główna baza, bez cuboidów; służy do negocjacji

Każda relacja wymaga **obustronnej akceptacji**, z wyjątkiem **Wroga**, którą jedna gildia może jednostronnie narzucić drugiej. Sojusz wymaga wysłania propozycji (`/guild ally <tag>`) i akceptacji przez drugą stronę. W stanie sojuszu obie gildie uzyskują ograniczony dostęp do swoich terenów, co ułatwia koordynację obrony i raidów.

Zawieszenie Broni (Truce) to stan przejściowy do negocjacji — PvP wyłączone (jak w sojuszu), ale brak uprawnień terytorialnych. Domyślnie każda nowa gildia zaczyna jako **Neutralna** względem wszystkich innych.

Zmiana relacji inicjuje **cooldown dyplomatyczny** (domyślnie 24h), zapobiegający szybkiemu przełączaniu relacji w celu obejścia ograniczeń PvP. Status relacji wyświetlany jest w czacie jako prefiks w kolorze relacji.

Wojny deklarowane stanowią rozszerzenie systemu: `/guild war declare <tag>` rozpoczyna stan wojenny po akceptacji lub upływie czasu na odpowiedź. Obie gildie automatycznie otrzymują relację Wróg, a na terenie wrogiej gildii obowiązują zasady raidowania (TNT, itemy raidingowe — szczegóły w Rozdziale 6). Wojna trwa domyślnie 48 godzin lub do spełnienia warunków zakończenia (np. zniszczenie serca gildii).

4.3.3 Rozwiązanie Gildii i Przekazanie Przywództwa

Rozwiązanie gildii (`/guild disband`) to proces nieodwracalny, dostępny wyłącznie dla Lidera. Plugin wymusza potwierdzenie (podwójne wpisanie komendy lub kliknięcie w czacie, zależnie od `disband-confirmation-type`).

Sekwencja rozwiązania:

1. **Potwierdzenie** w ciągu 60 sekund.
2. **Zwrot kosztu** (opcjonalnie) — jeśli `refund-on-disband: true`, część kosztu zwracana jest wg współczynnika `refund-percentage`.
3. **Usunięcie cuboidów** — claimi zwalniane w `CuboidManager` i stają się dostępne do ponownego zajęcia.
4. **Rozpuszczenie relacji** — wszystkie sojusze i wrogowie automatycznie zerwani.
5. **Wyrzucenie członków** — wszyscy tracą członkostwo i tag.
6. **Archiwacja** — rekord oznaczany jako `disbanded: true` z timestamp'em; dane pozostają dostępne dla administratorów.
7. **Powiadomienie** — komunikat na czat serwera (jeśli włączono) oraz do byłych członków.

Przekazanie przywództwa (`/guild leader <gracz>`), dostępne wyłącznie dla obecnego Lidera, wymaga by wskazany gracz był członkiem gildii. Po przekazaniu poprzedni Lider automatycznie otrzymuje rangę **Co-Lidera** (konfigurowalne przez `former-leader-rank`), a nowy Lider pełne uprawnienia. Operacja jest natychmiastowa, bez potrzeby potwierdzenia.

4.3.4 Log Aktywności i System Powiadomień

Każda operacja administracyjna jest automatycznie logowana jako narzędzie audytu wewnętrznego.

Zapisywane zdarzenia: dołączenie/wyjście/wyrzucenie gracza, zmiany rang, zmiany relacji dyplomatycznych, operacje na cuboidach, transakcje ekonomiczne, zmiany ustawień gildii, nadawanie/usuwanie tytułów, bany/unbany, przekazanie przywództwa, rozwiązanie gildii.

Log przechowywany jest **równolegle w dwóch miejscach**:

1. **Plik tekstowy** — `logs/guilds/<guild-uuid>.log`, rotacja dzienna, archiwum po 30 dniach.
2. **Baza danych** — tabela `guild_activity_log` z kolumnami: `timestamp`, `guild_id`, `actor_uuid`, `action_type`, `target_uuid`, `details` (JSON).

Dostęp do logu: `/guild log [strona]` (uprawnienie `guild.log.view`, domyślnie: Lider, Co-Lider, Moderator). Format definiowany w pliku językowym pod `guild.log.format` z placeholderami `%timestamp%`, `%actor%`, `%action%`, `%target%`.

System powiadomień działa w dwóch trybach:

- **Synchroniczne** (domyślne) — wyświetlane natychmiast członkom online w czacie lub na action barze (`notification-style`).
- **Offline** — dla członków offline powiadomienia kolejgowane są w `guild_notifications` i wyświetlane po zalogowaniu. Przeglądanie zaległych: `/guild notifications`.

Wszystkie komunikaty systemu gildii pobierane są dynamicznie z plików w katalogu `lang/` (domyślnie `pl.yml` lub `en.yml`). Użycie MiniMessage zapewnia pełną kontrolę nad wyglądem tekstu, wsparcie kolorów heksadecymalnych, gradientów oraz efektów hover i click events — znacząco podnosząc jakość UX względem klasycznych kodów `&` z Factions.

4. Jajo Gildii — System Obrony

Jajo gildii stanowi centralny element mechaniczną całego systemu Fraction Guild Clans. Jest to fizyczny blok umieszczony wewnątrz cuboidu gildii, którego zniszczenie w trzech kolejnych etapach prowadzi do ostatecznego rozwiązania gildii. Mechanika ta łączy w sobie elementy PvP (Player versus Player — walka gracza z graczem), zarządzania zasobami, planowania obrony oraz koordynacji zespołowej członków gildii. Celem systemu jest stworzenie wrażenia realnego zagrożenia i konsekwencji strategicznych decyzji — każda gildia musi aktywnie dbać o swoje jajo, bowiem jego utrata oznacza nieodwracalny koniec istnienia organizacji na serwerze. Jajo generowane jest automatycznie w momencie założenia gildii i nie może zostać przeniesione ręcznie przez graczy, co gwarantuje spójność rozgrywki i eliminuje próby exploitowania systemu przez ukrywanie jaja w trudnodostępnych lokalizacjach.

4.1 Mechanika Jaja Gildii

4.1.1 Umiejscowienie i Generowanie

Jajo gildii generowane jest automatycznie przez plugin w ściśle zdefiniowanej lokalizacji, co eliminuje dowolność gracza w kwestii jego pozycjonowania i zapewnia uczciwe warunki dla potencjalnych atakujących. Proces generowania inicjowany jest bezpośrednio po wykonaniu komendy `/guild create` i pomyślnym zakończeniu procesu zakładania gildii.

Lokalizacja jaja określana jest na podstawie centrum geometrycznego cuboidu gildii. Plugin oblicza środek powierzchniowej projekcji cuboidu (średnia arytmetyczna współrzędnych X i Z granic obszaru), a następnie ustala pozycję bloku na osi Y. Poziom Y jest wartością konfigurowalną w sekcji `egg.generation-y-level` pliku `config.yml` i domyślnie wynosi 64, co odpowiada standardowemu poziomowi terenu w typowym świecie Minecraft Overworld. Administrator może zmienić tę wartość na przykład na 70, aby jajo generowało się nieco powyżej poziomu gruntu i było łatwiej dostępne dla atakujących, lub na 55, jeśli chce, aby jajo znajdowało się pod ziemią i wymagało od agresorów dodatkowego wysiłku eksploracyjnego.

Blok jaja umieszczany jest w świecie, w którym zostaje założona gildia. Jeśli powierzchnia centrum cuboidu nie jest odpowiednia (np. woda, lava, powietrze nad przepaścią), plugin wykonuje iteracyjne poszukiwanie najbliższego stałego bloku w dół (maksymalnie 10 bloków), a następnie w górę (maksymalnie 10 bloków), aby zapewnić stabilne osadzenie. Jeśli żadna odpowiednia powierzchnia nie zostanie znaleziona, generowanie jaja zostaje odroczone, a lider gildii otrzymuje komunikat z prośbą o przygotowanie terenu.

Poniższy fragment konfiguracji `config.yml` ilustruje sekcję odpowiedzialną za generowanie jaja:

```
egg:
  # Typ bloku używanego jako jajo gildii
  block-type: "DRAGON_EGG"

  # Poziom Y, na którym generowane jest jajo (względem światowej osi Y)
  # -1 oznacza automatyczne dopasowanie do terenu
  generation-y-level: 64

  # Promień w blokach od centrum cuboidu, w którym jajo może zostać wygenerowane
  # 0.0 = dokładne centrum
  spawn-offset: 0.0

  # Czy jajo ma generować się na najbliższym stałym bloku, jeśli poziom Y jest
  # powietrzem
  snap-to-ground: true

  # Maksymalna liczba prób znalezienia odpowiedniej powierzchni
  max-placement-attempts: 20
```

4.1.2 Wygląd i Typ Bloku

Domyślnie jajo gildii reprezentowane jest przez blok `DRAGON_EGG` — unikalny blok z Minecraft, który występuje w grze od wersji 1.9 i jest naturalnie kojarzony z końcowym bossem gry (Ender Dragon). Ten wybór ma uzasadnienie zarówno estetyczne, jak i symboliczne — jajo smoka odzwierciedla kruchość i wartość obronnego punktu gildii.

Plugin oferuje jednak pełną konfigurowalność typu bloku, co pozwala administratorom serwera na dopasowanie wyglądu jaja do estetyki serwera lub do wykorzystania nowych bloków dostępnych w najnowszych wersjach Minecraft. W wersjach od 1.21.x administrator może zastosować bloki takie jak `CREAKING_HEART` (blok pochodzący z biomu pale garden, wprowadzony w wersji 1.21.2) lub `TRIAL_SPAWNER` (blok z trial chambers, wprowadzony w wersji 1.21), które posiadają unikalne animacje i efekty wizualne, dodatkowo podkreślając wyjątkowość obiektu.

Przy wyborze niestandardowego bloku należy uwzględnić następujące ograniczenia techniczne. Blok musi być pełnym blokiem solidnym (`isSolid() == true`) lub półblokiem (`isBlock()`), aby plugin mógł prawidłowo obsługiwać zdarzenia niszczenia. Bloki nietrwałe takie jak `TALL_GRASS`, `WATER` czy `TORCH` nie mogą zostać użyte jako jajo, ponieważ plugin odrzuci je podczas walidacji przy starcie serwera, logując odpowiedni błąd w konsoli. Jeśli podano nieprawidłowy typ bloku, plugin przywraca wartość domyślną `DRAGON_EGG` i kontynuuje inicjalizację.

Warto zwrócić uwagę, że niektóre bloki (np. `CREAKING_HEART`) posiadają własne zachowania w grze — w przypadku tego konkretnego bloku oznacza to okresową generację mobów w pobliżu. Plugin aktywnie tłumie te domyślne zachowania dla bloku jaja, aby nie zakłócały one przebiegu rozgrywki. Tłumienie realizowane jest poprzez nasłuchiwanie zdarzeń spawnu mobów w promieniu 16 bloków od jaja i anulowanie tych, które są wynikiem działania samego bloku, a nie celowej mechaniki ataku na gildię.

4.1.3 System Powiadomień o Statusie Jaja

Komunikacja statusu jaja do członków gildii realizowana jest przez trójwarstwowy system powiadomień, który zapewnia, że żadne istotne zdarzenie nie zostanie przeoczone przez zespół. Warstwa pierwsza obejmuje standardowe komunikaty chatowe — tekstowe wiadomości wyświetlane w czacie gildijnym (dostępnym poprzez alias `/g` lub `/guild chat`). Komunikaty te zawierają precyzyjne informacje o zdarzeniu, jego skutkach oraz sugerowanych działaniach. Przykładowy komunikat po pierwszym uszkodzeniu jaja brzmi: `[GILDIA] Twoje jajo zostało uszkodzone! Pozostałe wytrzymałość: 2/3. Zgromadź zespół na obronę!`

Warstwa druga to tytuły ekranowe (Title API), które pojawiają się na środku ekranu każdego członka gildii online. Tytuły używane są dla zdarzeń o najwyższym priorytecie: uszkodzenie jaja, zniszczenie jaja oraz udana naprawa. Tytuł składa się z dwóch linii — nagłówek (duża czcionka, efektowny kolor) oraz podtytułu (mniejsza czcionka z opisem sytuacji). Przykład: nagłówek `OSTRZEŻENIE!` w kolorze czerwonym, podtytuł

Jajo gildii zostało uszkodzone w kolorze białym. Czas wyświetlania tytułu domyślnie wynosi 3 sekundy (fade-in: 10 ticków, display: 60 ticków, fade-out: 10 ticków) i jest konfigurowalny w `config.yml`.

Warstwa trzecia obejmuje dźwięki (Sound API), które współdziałają z powiadomieniami wizualnymi, tworząc spójne doświadczenie multimedialne. Każdy typ zdarzenia przypisany jest do konkretnego dźwięku z biblioteki Minecraft:

- Uszkodzenie jaja: `ENTITY_WITHER_SPAWN` — głęboki, niski dźwięk zwiastujący zagrożenie.
- Zniszczenie etapowe: `ENTITY_GENERIC_EXPLODE` — efekt eksplozji, synchronizowany z wizualnym particlem.
- Udana naprawa: `BLOCK_BEACON_ACTIVATE` — pozytywny, wznoszący się dźwięk potwierdzający sukces.
- Pełne odrodzenie jaja: `UI_TOAST_CHALLENGE_COMPLETE` — uroczysty fanfarrowy dźwięk nagradzający odzyskanie pełnej wytrzymałości.

Dźwięki odtwarzane są z głośnością 1.0 i pitch zależnym od etapu zniszczenia — im bliżej finalnego zniszczenia, tym niższy pitch, co podświadomie buduje napięcie u słuchacza.

4.2 System Zniszczeń

4.2.1 Zasada Trójetapowego Zniszczenia

Rdzeniem systemu obronnego jest mechanika wymagająca od atakującej gildii trzykrotnego zniszczenia jawa, przy czym każde kolejne zniszczenie wymaga spełnienia coraz bardziej rygorystycznych warunków i prowadzi do głębszego osłabienia obrony gildii docelowej. System ten zaprojektowano celowo, aby zapobiec sytuacjom, w których gildia traci całą infrastrukturę w wyniku jednego, krótkiego ataku — jednocześnie jednak każde uszkodzenie jawa ma realne, odczuwalne konsekwencje, co zachęca obie strony do aktywnego zaangażowania w proces wojenny.

Każdy etap zniszczenia posiada unikalne warunki aktywacji, efekty na cuboid gildii, konsekwencje dla jej członków oraz czas, po którym następuje automatyczna próba regeneracji. Poniższa tabela zestawia parametry trzech etapów zniszczenia w formie umożliwiającej szybką referencję podczas konfiguracji i administracji serwera.

Parametr	Etap 1: Uszkodzenie	Etap 2: Pęknięcie	Etap 3: Rozpad
Nazwa systemowa	DAMAGED	CRACKED	DESTROYED

Wizualny stan jaja	Mikropęknięcia, lekka zmiana barwy	Widoczne pęknięcia, częściowe uszkodzenie kruszywa	Blok zniszczony, pozostaje particle field
Warunek aktywacji	Zadanie obrażeń blokowi przez dowolny mechanizm (kopanie, TNT, projectile)	Zadanie obrażeń blokowi w stanie DAMAGED w oknie czasowym 30 minut od regeneracji	Zadanie obrażeń blokowi w stanie CRACKED w oknie czasowym 30 minut od regeneracji
Cooldown między atakami	Brak	60 minut (konfigurowalne)	120 minut (konfigurowalne)
Efekt na cuboid	Brak zmian w ochronie	Częściowe wyłączenie ochrony budowlanej na krawędziach (promień -2)	Całkowite wyłączenie ochrony PvP i budowlanej, otwarcie drzwi i skrzynek
Efekt defensywny	Fala 5 mobów obronnych (typ konfigurowalny)	Fala 10 mobów obronnych + efekt Slowness I dla intruzów na krawędzi	Brak fali obronnej — jajo dezaktywowane
Czas auto-regeneracji	4 godziny	8 godzin	Brak auto-regeneracji — wymagana ręczna odbudowa
Powiadomienie globalne	Tylko członkowie gildii	Członkowie gildii + gracze w promieniu 100 bloków	Wszyscy gracze na serwerze

Analiza tabeli ujawnia celowy wzorzec projektowy progresywnego eskalacji. Pierwsze uszkodzenie jest stosunkowo łatwe do przeprowadzenia i służy głównie jako ostrzeżenie — aktywuje falę obronną, ale nie narusza integralności cuboidu. To zachęta dla obrońców do zareagowania, jednocześnie dając atakującym punkt zaczepienia narracyjny ("pierwsza krew"). Drugie zniszczenie stanowi punkt zwrotny — po raz pierwszy ochrona cuboidu zostaje faktycznie naruszona, co otwiera przestrzeń dla dalszych działań ofensywnych. Trzecie zniszczenie jest punktem bez powrotu — jajo przestaje istnieć jako blok fizyczny, a gildia wchodzi w 24-godzinny okres próbny, podczas którego musi zebrać zasoby na odbudowę. Kluczowym elementem balansu jest cooldown między atakami na wyższych etapach — atakujący nie mogą po prostu czekać przy jaju i natychmiast uderzać po każdej regeneracji; muszą zaplanować kolejny szturm, co daje obrońcom czas na przygotowanie.

4.2.2 Pierwsze Zniszczenie — Uszkodzenie Jaja

Pierwsze zniszczenie inicjowane jest, gdy blok jaja otrzyma wystarczającą ilość obrażeń, aby zredukować jego wewnętrzny licznik wytrzymałości do zera. Mechanika obrażeń bloku działa analogicznie do standardowego kopania w Minecraft — każde uderzenie narzędziem lub zadanie obrażeń przez eksplozję zmniejsza wytrzymałość o wartość zależną od typu obrażeń. Jajo posiada wbudowany mnożnik odporności — domyślnie 5x więcej punktów wytrzymałości niż standardowy blok w grze — co oznacza, że nawet z

diamentowym kilofem (`Efficiency V`) gracz potrzebuje kilkunastu sekund ciągłego kopania, aby przebić się przez blok. Ten czas jest kluczowy dla obrońców, ponieważ zazwyczaj pozwala na dotarcie do jaja i przerwanie ataku.

W momencie osiągnięcia zera wytrzymałości blok nie zostaje fizycznie zniszczony — zamiast tego zmienia swój stan wewnętrzny na `DAMAGED` , co wizualnie objawia się pojawieniem się mikropęknięć na teksturze (realizowane poprzez `resource pack` lub `particle effect` w zależności od użytego typu bloku) i emisją cząsteczek typu `BLOCK_CRACK` wokół bloku. Jednocześnie aktywowana jest fala obronna mobów — grupa skonfigurowanych stworzeń generowana jest w promieniu 10 bloków od jaja, z preferencją spawnu za plecami atakujących.

Domyślnie fala pierwszego etapu składa się z 5 szkieletów (`SKELETON`) uzbrojonych w łuki (Power I, Punch I), wyposażonych w diamentowe hełmy z enchantem Protection II. Typy mobów, ich wyposażenie, enchanty oraz liczba są w pełni konfigurowalne w sekcji `egg.defense-waves.stage1` pliku `config.yml` . Moby te posiadają predefiniowany target — atakują graczy, którzy zadali obrażenia jaju w ciągu ostatnich 60 sekund, oraz każdego gracza niebędącego członkiem gildii, który znajduje się wewnątrz cuboidu. Moby obronne despawnowane są automatycznie po 5 minutach lub po śmierci wszystkich atakujących intruzów — zależnie od tego, co nastąpi wcześniej.

Po uszkodzeniu jajo rozpoczyna wewnętrzny licznik auto-regeneracji. Domyślnie wynosi on 4 godziny (14400 sekund). Po upływie tego czasu jajo automatycznie przywraca stan `HEALTHY` , a wszystkie wizualne efekty uszkodzenia znikają. Proces regeneracji jest wizualnie sygnalizowany przez spiralny efekt `particle HAPPY_VILLAGER` wokół bloku przez 10 sekund przed finalnym przywróceniem. Jeśli w trakcie trwania licznika regeneracji jajo zostanie ponownie zaatakowane, regeneracja zostaje przerwana, a jajo przechodzi do etapu 2.

4.2.3 Drugie Zniszczenie — Pęknięcie Ochrony

Drugi etap zniszczenia aktywowany jest wyłącznie, gdy jajo w stanie `DAMAGED` zostanie ponownie zredukowane do zera wytrzymałości. Ponowne zadanie obrażeń możliwe jest dopiero po upływie 60-minutowego cooldownu od momentu pierwszego uszkodzenia — ten okres jest konfigurowalny w `config.yml` pod kluczem `egg.damage-cooldown.stage1-to-stage2` . Cooldown ten zapobiega scenariuszowi, w którym atakujący "campią" przy jaju i natychmiast po jego regeneracji ponownie je atakują. Obrońcy mają zagwarantowany czas na reakcję i przygotowanie obrony.

Gdy drugie zniszczenie zostaje osiągnięte, jajo przechodzi w stan `CRACKED` . Wizualnie objawia się to znacznie bardziej widocznymi pęknięciami na bloku oraz ciągłą emisją cząsteczek `SMOKE_NORMAL` i `LAVA` z powierzchni bloku. Stan `CRACKED` niesie ze sobą pierwszą realną, mechaniczną konsekwencję dla obrony gildii — częściowe wyłączenie ochrony cuboidu na jego zewnętrznej warstwie.

Mechanizm częściowego wyłączenia ochrony działa następująco: z zewnętrznego promienia cuboidu odejmowane są 2 bloki (wartość konfigurowalna). Wszystkie bloki znajdujące się wewnątrz tego nowego,

zmniejszonego obszaru zachowują pełną ochronę — nie mogą być niszczone ani budowane przez osoby spoza gildii. Jednak obszar pomiędzy oryginalnym a zmniejszonym promieniem (tzw. "pierścień obronny") traci ochronę budowlaną. Oznacza to, że atakujący mogą w tym obszarze stawiać i niszczyć bloki, co pozwala im na budowanie wież oblężniczych, kopanie tuneli pod ziemią oraz tworzenie innych struktur ułatwiających dalszy atak. Ochrona PvP w pierścieniu pozostaje aktywna — gracze spoza gildii nadal nie mogą atakować członków wewnątrz cuboidu, chyba że obaj znajdują się w strefie PvP skonfigurowanej przez lidera.

Dodatkowo w całym pierścieniu aktywowany jest efekt `SLOWNESS I` (Spowolnienie I) dla wszystkich graczy niebędących członkami gildii. Efekt ten utrzymuje się przez cały czas przebywania w pierścieniu i jest odświeżany co 5 sekund. Powoduje to, że atakujący poruszają się w strefie zagrożenia znacznie wolniej, co daje obrońcom przewagę taktyczną podczas ostrzału z murów cuboidu.

Fala obronna drugiego etapu jest znacznie silniejsza — domyślnie generuje 10 mobów, w tym 5 szkieletów i 5 żołnierzy (`ZOMBIE`) uzbrojonych w żelazne miecze (`Sharpness I`). Co istotne, moby te posiadają enchant `FIRE_ASPECT I` na broni, co dodatkowo komplikuje sytuację atakującym. Auto-regeneracja po drugim etapie wynosi domyślnie 8 godzin (28800 sekund) — dwukrotnie dłużej niż po pierwszym uszkodzeniu, co odzwierciedla powagę sytuacji.

4.2.4 Trzecie Zniszczenie — Dezaktywacja i Rozwiązanie Gildii

Trzecie, ostatnie zniszczenie jest punktem bez powrotu w istnieniu gildii. Aktywowane jest po zadaniu obrażeń jaju w stanie `CRACKED`, z zachowaniem 120-minutowego cooldownu od momentu drugiego zniszczenia. Gdy wytrzymałość spada do zera, blok jaja fizycznie zostaje zniszczony — znika z świata, a w jego miejscu pojawia się efekt eksplozji (particle `EXPLOSION_HUGE` i dźwięk `ENTITY_GENERIC_EXPLODE`) bez jednak zadawania obrażeń otoczeniu, aby uniknąć niekontrolowanego niszczenia otaczającej infrastruktury.

Po zniszczeniu bloka jajo przechodzi w stan `DESTROYED`. W tym stanie nie istnieje już jako fizyczny obiekt w świecie — pozostaje jednak jego "dusza" w postaci nieruchomego pola particle `PORTAL`, które unosi się w miejscu, gdzie znajdował się blok. Pole to służy jako punkt odniesienia dla systemu odbudowy i jako wizualny znak upadku gildii.

Dezaktywacja jaja powoduje natychmiastowe i drastyczne konsekwencje:

1. **Całkowite wyłączenie ochrony cuboidu** — wszystkie flagi ochronne (budowanie, niszczenie, interakcja z skrzynkami, drzwiami, przełącznikami) zostają zawieszone. Każdy gracz na serwerze może dowolnie modyfikować teren w obrębie cuboidu.
2. **Uaktywnienie PvP** — pełna walka gracz z graczem staje się możliwa na całym obszarze cuboidu, niezależnie od wcześniejszych ustawień flag PvP.

3. **Brak fali obronnej** — w przeciwieństwie do poprzednich etapów, trzecie zniszczenie nie generuje mobów obronnych. Gildia jest całkowicie bezbronna.
4. **Globalne powiadomienie** — komunikat o zniszczeniu jaja wyświetlany jest wszystkim graczom na serwerze w formacie: `[SERWER] Gildia [TAG] NazwaGildii straciła swoje jajo! Ma 24 godziny na odbudowę.`

Kluczowym elementem tego etapu jest 24-godzinny okres na odbudowę (konfigurowalny w `config.yml` pod kluczem `egg.rebuild-timer.hours: 24`). W tym czasie członkowie gildii mogą zebrać wymagane surowce i ręcznie odbudować jajo w dokładnie tym samym miejscu, gdzie znajdowało się oryginalne. Proces odbudowy wymaga użycia komendy `/guild egg rebuild` przez lidera lub moderatora gildii, posiadającego w ekwipunku odpowiednią liczbę skonfigurowanych surowców (domyślnie: 64 obsidianu, 16 ender pearl, 1 nether star oraz 10000 jednostek waluty serwera). Po pomyślnej odbudowie jajo wraca do stanu `HEALTHY`, a ochrona cuboidu zostaje przywrócona w pełnym zakresie.

Jeśli jednak gildia nie dokona odbudowy w wyznaczonym czasie, plugin automatycznie inicjuje procedurę rozwiązania gildii (`DISBAND`). Proces ten jest nieodwracalny — wszystkie dane gildii usuwane są z bazy, członkowie tracą przynależność, cuboid zostaje usunięty z systemu ochrony (teren staje się terenem niczym), a na mapie HTML pojawia się historyczny marker upadłej gildii. Zanim jednak do tego dojdzie, system wysyła serię ostrzegawczych powiadomień: na 12 godzin, 6 godzin, 1 godzinę, 30 minut i 10 minut przed rozwiązaniem — co daje członkom maksymalną możliwość reakcji.

4.3 Ochrona i Naprawa

4.3.1 Naturalna Regeneracja Jaja

System naturalnej regeneracji pozwala jaju na samoistne powrót do pełnej wytrzymałości po upływie zdefiniowanego czasu, pod warunkiem że nie doszło do kolejnego ataku w okresie regeneracji. Mechanizm ten działa dla etapów 1 i 2 — etap 3 nie podlega auto-regeneracji i wymaga ręcznej odbudowy.

Regeneracja zarządzana jest przez asynchroniczny scheduler pluginu, który co 60 sekund sprawdza stan wszystkich jaj na serwerze. Jeśli jajo posiada aktywny licznik regeneracji i jego wartość osiągnęła zero, scheduler wywołuje metodę regeneracji. Proces ten odbywa się synchronicznie w głównym wątku serwera, aby zapewnić bezpieczeństwo operacji na blokach. Czas regeneracji dla każdego etapu jest konfigurowalny:

```
egg:
  auto-regeneration:
    # Czas regeneracji po etapie 1 (w sekundach)
    stage1-regen-time: 14400 # 4 godziny

    # Czas regeneracji po etapie 2 (w sekundach)
    stage2-regen-time: 28800 # 8 godzin

    # Czy regeneracja wymaga obecności minimum 1 członka gildii online
    require-member-online: false

    # Czy regeneracja przyspiesza, gdy więcej członków jest online
    # (mnożnik: liczba_online * 0.1, maksymalnie 2x szybciej)
    online-member-boost: true
```

Opcja `require-member-online` pozwala administratorowi na stworzenie bardziej rygorystycznych warunków — jeśli ustawiona na `true`, jajo regeneruje się tylko wtedy, gdy co najmniej jeden członek gildii przebywa na serwerze. Opcja `online-member-boost` włącza mechanizm przyspieszenia regeneracji proporcjonalnego do liczby aktywnych członków. Przy maksymalnym przyspieszeniu 2x czas regeneracji skraca się o połowę, co nagradza aktywność zespołową. Warto jednak zachować ostrożność przy włączaniu tej opcji na dużych serwerach (100+ graczy), ponieważ częste odpytywanie stanu online wielu gildii może generować dodatkowe obciążenie scheduler.

Proces regeneracji jest zdarzeniem atomowym — albo zakończy się sukcesem, albo nie zostanie wykonany wcale. W momencie rozpoczęcia regeneracji plugin ustawia flagę `isRegenerating` na jaju, aby zapobiec sytuacji, w której dwa wątki jednocześnie próbują zmienić stan tego samego jaja. Jeśli podczas regeneracji jajo zostanie zaatakowane, proces jest anulowany, flaga resetowana, a licznik regeneracji usuwany.

4.3.2 Ręczna Naprawa przez Członków Gildii

Ręczna naprawa pozwala członkom gildii na aktywne przyspieszenie regeneracji jaja poprzez dostarczenie wymaganych surowców i użycie dedykowanej komendy. Mechanizm ten dostępny jest dla jaj w stanach `DAMAGED` oraz `CRACKED` — dla stanu `DESTROYED` konieczna jest komenda odbudowy (`/guild egg rebuild`), która ma inne wymagania.

Komenda naprawy dostępna jest pod aliasem `/guild egg repair` (alternatywnie `/g egg repair`). Uprawnienie do jej użycia posiadają członkowie gildii o randze Członek lub wyższej — rekruci nie mogą naprawiać jaja. Po wpisaniu komendy plugin weryfikuje następujące warunki:

1. Gracz musi znajdować się w obrębie cuboidu swojej gildii (maksymalna odległość od centrum: promień cuboidu).
2. Jajo musi być w stanie innym niż `HEALTHY` i `DESTROYED`.
3. Gracz musi posiadać w ekwipunku wszystkie wymagane do naprawy itemy.

4. Od ostatniej naprawy musi upłynąć czas cooldown (domyślnie 30 minut per gracz, 10 minut globalnie per gildia).

Jeśli wszystkie warunki zostaną spełnione, surowce zostają pobrane z ekwipunku gracza, a jajo natychmiast przechodzi do stanu `HEALTHY`, niezależnie od aktualnego etapu uszkodzenia. Wizualnie proces naprawy sygnalizowany jest przez efekt particle `VILLAGER_HAPPY` otaczający jajo przez 5 sekund oraz emitowany jest dźwięk `BLOCK_ANVIL_USE`. Po pomyślnej naprawie wszyscy członkowie gildii online otrzymują powiadomienie chatowe z nickiem gracza, który dokonał naprawy.

Wymagane surowce do naprawy są konfigurowalne i mogą różnić się w zależności od aktualnego etapu uszkodzenia. Domyślna konfiguracja przewiduje wyższe wymagania dla naprawy jaja w stanie `CRACKED` niż `DAMAGED`, co odzwierciedla większy zakres uszkodzeń do naprawienia:

```
egg:
  repair:
    # Wymagane itemy do naprawy jaja w stanie DAMAGED
    stage1-requirements:
      items:
        - material: "OBSIDIAN"
          amount: 16
        - material: "ENDER_PEARL"
          amount: 4
      money: 1000.0

    # Wymagane itemy do naprawy jaja w stanie CRACKED
    stage2-requirements:
      items:
        - material: "OBSIDIAN"
          amount: 32
        - material: "ENDER_PEARL"
          amount: 8
        - material: "NETHER_STAR"
          amount: 1
      money: 5000.0

    # Cooldown naprawy (w sekundach)
    cooldown-player: 1800    # 30 minut per gracz
    cooldown-guild: 600     # 10 minut per gildia (globalny)
```

System cooldownów zapobiega scenariuszom, w których zamożni gracze mogliby nieograniczenie spamować naprawami, niszcząc balans ekonomiczny i strategiczny. Cooldown per-gracz oznacza, że ten sam gracz nie może naprawić jaja częściej niż co 30 minut, podczas gdy cooldown globalny per-gildia gwarantuje, że nawet rotacja między wieloma członkami nie pozwoli na naprawy częściej niż co 10 minut.

4.3.3 System Tarcz Ochronnych

System tarcz stanowi dodatkową warstwę zabezpieczeń, która gildie mogą nabywać za walutę serwera lub specjalne itemy. Tarcze działają jako jednorazowe bariery — aktywowane przed atakiem, absorbują one całość obrażeń zadanych jaju przez określony czas lub do momentu osiągnięcia limitu absorbowanych obrażeń.

Plugin oferuje trzy typy tarcz, różniące się parametrami działania i ceną. Poniższa tabela przedstawia zestawienie dostępnych tarcz ochronnych.

Parametr	Tarcza Standardowa	Tarcza Wzmocniona	Tarcza Absolutna
Nazwa systemowa	SHIELD_BASIC	SHIELD_ENHANCED	SHIELD_ABSOLUTE
Czas trwania	10 minut	20 minut	30 minut
Limit obrażeń	500 HP	1500 HP	Brak limitu (nieskończona)
Efekt dodatkowy	Brak	Regeneracja 5 HP/s dla jaja	Całkowita nietykalność jaja + efekt Resistance II dla członków w promieniu 15 bloków
Cena (waluta serwera)	2500	10000	50000
Cena alternatywna (itemy)	32x IRON_INGOT	16x DIAMOND + 8x EMERALD_BLOCK	1x NETHER_STAR + 64x OBSIDIAN
Cooldown między użyciami	60 minut	90 minut	120 minut
Dostępność	Gildia poziom 1+	Gildia poziom 3+	Gildia poziom 5+

Tarcze zakupywane są za pośrednictwem GUI sklepu gildii (komenda `/guild shop`) lub bezpośrednio przez komendę `/guild egg shield buy <typ>`. Po zakupie tarcza trafia do wirtualnego magazynu gildii i może być aktywowana w dowolnym momencie przez lidera lub moderatora za pomocą komendy `/guild egg shield activate <typ>`. Aktywacja jest natychmiastowa i nieodwracalna — raz włączona tarcza działa do wygaśnięcia czasu lub wyczerpania limitu obrażeń.

Podczas aktywacji tarczy jajo otaczane jest efektem wizualnym odpowiadającym typowi tarczy. Tarcza Standardowa generuje słabą, pulsującą barierę (particle `END_ROD`), Tarcza Wzmocniona tworzy widoczną kopułę ochronną (particle `HAPPY_VILLAGER`), a Tarcza Absolutna generuje imponującą, wirującą tarczę z cząsteczek `DRAGON_BREATH` i `PORTAL`. Efekty te widoczne są dla wszystkich graczy w promieniu 50 bloków, co dodatkowo sygnalizuje obecnym atakującym aktywację ochrony.

Mechanika absorbowania obrażeń działa na zasadzie priorytetu — zanim jakiegokolwiek obrażenia dotrą do wewnętrznego licznika wytrzymałości jaja, odejmowane są od aktualnej wartości tarczy. Dla Tarcz Standardowej i Wzmocnionej, gdy limit obrażeń zostanie wyczerpany, tarcza natychmiast dezaktywuje się (z efektem dźwiękowym `BLOCK_GLASS_BREAK`), a jajo staje się podatne na dalsze obrażenia. Dla Tarczy Absolutnej niemożliwe jest zadanie jakichkolwiek obrażeń jaju do momentu wygaśnięcia czasu trwania — nawet najbardziej zmasowany atak nie przebijie tej ochrony.

Warto zwrócić uwagę na efekt balansujący — aktywna tarcza wydłuża również cooldowny między etapami zniszczenia o 50%. Oznacza to, że gildia chroniąca się tarczą zyskuje czas nie tylko na bezpośrednią ochronę, ale również na strategiczne przygotowanie się do kolejnej fali ataku. Z drugiej strony, koszty tarcz (szczególnie Tarczy Absolutnej) są na tyle wysokie, że ich ciągłe używanie wymaga znaczących zasobów ekonomicznych, co zapobiega sytuacji, w której bogata gildia mogłaby być nieprzerwanie niepokonana.

4.3.4 Log Zdarzeń Związanych z Jajem

Każde zdarzenie istotne dla życia jaja gildii rejestrowane jest w dedykowanym logu, który stanowi historyczny dokument wszystkich ataków, napraw i zmian stanu. Log ten dostępny jest dla członków gildii przez komendę `/guild egg log` (alternatywnie przez zakładkę w GUI gildii), a administratorzy serwera mogą przeglądać logi wszystkich gildii przez `/guild admin egglog <tag_gildii>`.

Każdy wpis w logu zawiera następujące pola: znacznik czasu (timestamp) z dokładnością do sekundy, typ zdarzenia (atak, naprawa, regeneracja, aktywacja tarczy, zmiana etapu), nick gracza inicjującego zdarzenie (jeśli dotyczy), stan jaja przed zdarzeniem, stan jaja po zdarzeniu oraz dodatkowe metadane specyficzne dla typu zdarzenia (np. obrażenia zadane w przypadku ataku, typ użytej tarczy w przypadku aktywacji ochrony).

Log przechowywany jest w bazie danych (SQLite lub MySQL, w zależności od konfiguracji) w tabeli `guild_egg_logs`. Domyślnie wpisy archiwizowane są przez 30 dni, po czym starsze rekordy automatycznie usuwane są przez nocny zadanie czyszczące (scheduler uruchamiany o 3:00 czasu serwera). Administrator może zmienić okres retencji w konfiguracji lub całkowicie wyłączyć czyszczenie, jeśli wymagana jest pełna historia dla celów moderacyjnych.

```
egg:
  logging:
    # Czy logowanie jest włączone
    enabled: true

    # Typy zdarzeń do logowania
    logged-events:
      - "DAMAGE"
      - "STAGE_CHANGE"
      - "REPAIR"
      - "REBUILD"
      - "SHIELD_ACTIVATE"
      - "REGENERATION"
      - "DISBAND_WARNING"

    # Okres retencji logów (w dniach)
    retention-days: 30

    # Godzina uruchomienia zadania czyszczącego
    cleanup-hour: 3
```

Log zdarzeń pełni trzy kluczowe funkcje. Po pierwsze, jest narzędziem informacyjnym dla członków gildii — pozwala na analizę częstotliwości ataków, identyfikację najbardziej agresywnych gildii przeciwniczych oraz ocenę skuteczności własnych działań obronnych. Po drugie, stanowi podstawę dowodową w przypadku konfliktów między gildiami — administrator może zweryfikować, czy dany atak miał miejsce, kto go przeprowadził i jakie były jego skutki. Po trzecie, logi dostarczają cennych danych analitycznych dla administratora serwera — analiza agregowana pozwala na identyfikację trendów (np. które gildie są najczęściej atakowane, o których porach dochodzi do największej liczby ataków), co może być wykorzystane do balansowania ekonomii i mechanik serwera.

Przykładowy fragment logu po ataku i naprawie:

```
[2024-01-15 18:42:11] DAMAGE | Gracz: xXDestroyerXx | Obrażenia: 47 | Broń:
DIAMOND_PICKAXE
          Stan przed: HEALTHY | Stan po: DAMAGED
[2024-01-15 18:42:15] SHIELD_ACTIVATE | Gracz: GuildLeaderPL | Typ: SHIELD_BASIC
[2024-01-15 18:45:33] DAMAGE | Gracz: xXDestroyerXx | Obrażenia: 12 (absorbowane
przez tarczę)
[2024-01-15 19:15:11] SHIELD_EXPIRE | Typ: SHIELD_BASIC | Limit obrażeń: 0/500
[2024-01-15 19:22:08] REPAIR | Gracz: GuildBuilder99 | Surowce pobrane:
OBSIDIANx16, ENDER_PEARLx4
          Stan przed: DAMAGED | Stan po: HEALTHY
```

Kompletny log zdarzeń, w połączeniu z systemem trójetapowego niszczenia, konfigurowalnymi tarczami ochronnymi oraz opcjami ręcznej i automatycznej naprawy, tworzy wielowarstwowy, głęboki system defensywny, który stanowi serce strategicznej rozgrywki w Fraction Guild Clans. Każda decyzja — czy to

zakup tarczy, czy zebranie surowców na naprawę, czy wybór momentu ataku na wroga — ma realne konsekwencje i wymaga od gildii koordynacji, planowania oraz efektywnego zarządzania zasobami.

6. Cuboidy i Własny System Ochrony

System ochrony terenów w **Fraction Guild Clans v2.0** stanowi fundamentalne odejście od architektury poprzedniej generacji pluginów gildyjnych. Zamiast delegować odpowiedzialność za ochronę regionów do zewnętrznych narzędzi takich jak WorldGuard czy WorldEdit, plugin implementuje **własny, w pełni zintegrowany silnik ochrony cuboidów**. To rozwiązanie eliminuje konflikty flag między systemami, zmniejsza obciążenie serwera i zapewnia płynną integrację z pozostałymi modułami — gildią, rankingiem, ekonomią EcoCore oraz systemem jaj gildii.

Kluczowe przewagi własnego systemu ochrony obejmują: brak zależności zewnętrznych (zero wymaganych pluginów), jednolity model uprawnień oparty na członkostwie gildii i sojuszach, natychmiastowe propagowanie zmian konfiguracji oraz pełną kontrolę administratora nad każdym aspektem ochrony. Cała konfiguracja cuboidów, leveli, flag i stref znajduje się w jednym pliku: `modules/cuboid.yml`, zgodnie z założeniami modularnej architektury v2.0.

6.1 Architektura Cuboidów

6.1.1 Cuboid oparty na promieniu — precyzja zamiast chunków

Cuboid gildii definiuje się jako **trójwymiarowy obszar oparty na promieniu cylindrycznym** (`radius`) mierzonym w blokach — wyraźnie odróżniając tę metodę od systemów chunkowych stosowanych w konkurencyjnych rozwiązaniach. Środek cuboidu wyznacza lokalizacja jaja gildii, które jednocześnie pełni funkcję punktu centralnego regionu i obiektu strategicznego podlegającego ochronie. Promień rozciąga się symetrycznie w płaszczyźnie poziomej (oś X i Z), tworząc na powierzchni okrągły obszar ochrony, natomiast rozpiętość pionowa (oś Y) obejmuje domyślnie pełną wysokość świata.

Model promieniowy oferuje fundamentalną przewagę nad systemami chunkowymi: **granica ochrony może przebiegać dowolnie w obrębie chunka**, co pozwala na precyzyjne dopasowanie wielkości cuboidu do potrzeb gildii bez sztucznego "zaokrąglania" do pełnych chunków (16×16 bloków). Gildia z promieniem 25 bloków otrzymuje obszar $\pi \times 25^2 \approx 1963$ bloków kwadratowych powierzchni — dokładnie tyle, ile wynosi konfiguracja. Taka precyzja ma bezpośrednie przełożenie na balans rozgrywki: administrator kontroluje dokładny koszt ekspansji, a gildie muszą strategicznie planować rozmieszczenie cuboidów na mapie.

Mechanicznie, każdy cuboid przechowywany jest jako instancja obiektu `GuildRegion` zawierająca: identyfikator gildii, środek (lokalizacja jaja), promień, wysokość, listę członków, listę zaufanych graczy (trusted) oraz zestaw flag ochrony. Sprawdzanie dostępu polega na obliczeniu odległości euklidesowej

gracza od środka w płaszczyźnie poziomej — operacja o złożoności $O(1)$, co gwarantuje niskie obciążenie serwera nawet przy setkach aktywnych cuboidów.

6.1.2 Automatyczne generowanie przy zakładaniu gildii

Cuboid gildii tworzy się automatycznie w momencie udanego wykonania komendy `/guild create`. Proces generowania przebiega sekwencyjnie:

1. **Walidacja lokalizacji** — plugin weryfikuje, czy pozycja gracza nie koliduje z istniejącym cuboidem innej gildii. Minimalna odległość między środkami wynosi promień obu gildii plus 5 bloków bufora (`cuboid.min-distance-between-guilds`). Sprawdzana jest też odległość od spawnu świata (domyślnie 200 bloków) oraz czy świat nie znajduje się na zakazanej liście.
2. **Tworzenie instancji regionu** — system inicjalizuje obiekt `GuildRegion` z promieniem levelu 1 (15 bloków), domyślnym zestawem flag oraz pustą listą trusted players.
3. **Rejestracja w silniku ochrony** — cuboid trafia do `RegionManager`, który utrzymuje mapę `guildId` → `GuildRegion` oraz drzewo przestrzenne (R-tree) dla efektywnego wyszukiwania regionów po współrzędnych.
4. **Umieszczenie jaja** — w środku cuboidu generowany jest blok jaja gildii (domyślnie: dragon egg), będący jednocześnie punktem centralnym regionu i obiektem strategicznym. Jego zniszczenie inicjuje proces upadku gildii.
5. **Wizualizacja granic** — opcjonalne wyświetlenie cząsteczek wzdłuż obwodu cuboidu informuje założyciela o zakresie ochrony.

6.1.3 Wizualizacja granic: cząsteczki, border z bloków, hologramy

Plugin oferuje trzy niezależne metody wizualizacji granic cuboidu, konfigurowalne w `modules/cuboid.yml` i aktywowane komendą `/guild border`.

Cząsteczki (particle border) — metoda domyślna. Plugin generuje strumień cząsteczek (typ konfigurowalny, domyślnie `REDSTONE`) wzdłuż okręgu o promieniu cuboidu, z interwałem co 2 bloki obwodu. Efekt widoczny przez 15 sekund, po czym cząsteczki znikają automatycznie. Jest to najmniej obciążająca metoda — wymaga jedynie wysłania pakietów cząsteczek bez modyfikacji świata.

Border z bloków (block border) — wzdłuż granicy cuboidu pojawiają się tymczasowe bloki (domyślnie: barrier), które fizycznie oznaczają granicę regionu. Dla członków gildii są niematerialne, dla obcych widoczne jako semi-transparentna ściana. Rekomendowane na serwerach hardcore PvP, gdzie precyzyjne określenie granic ma kluczowe znaczenie.

Hologramy granic (hologram border) — w czterech kardynalnych punktach granicy pojawiają się pływające napisy z tagiem gildii, poziomem i odległością do środka. Hologramy aktualizują się dynamicznie

przy zmianie promienia. Format napisu konfigurowalny przez pliki językowe z pełnym wsparciem MiniMessage.

6.2 System Leveli (10 poziomów)

System leveli stanowi mechaniczną oś rozwoju gildii, determinującą fizyczną wielkość terytorium, limit członków oraz dostęp do zaawansowanych funkcji. Każdy poziom wymaga dostarczenia konkretnych przedmiotów, posiadania odpowiedniej liczby punktów rankingowych, ewentualnie zapłaty w walucie EcoCore oraz osiągnięcia minimalnego wieku gildii. Awans jest jednokierunkowy i nieodwracalny.

6.2.1 Tabela 10 leveli — parametry i wymagania

Poniższa tabela przedstawia kompletną specyfikację dziesięciu poziomów. Wszystkie wartości domyślne można modyfikować w `modules/cuboid.yml` poprzez hot-reload.

Level	Promień (bloki)	Max członków	Wymagane itemy	Koszt EcoCore	Punkty rankingowe	Wiek gildii	Dodatkowe bonusy
1	15	5	—	—	—	—	Podstawowy cuboid, dostęp do /guild home
2	22	8	64× stone, 32× iron_ingot	\$500	100 pkt	1 dzień	Dostęp do magazynu gildii (/guild vault)
3	30	12	128× oak_log, 64× gold_ingot	\$1 500	300 pkt	2 dni	Możliwość tworzenia sojuszy (max 2)
4	40	16	64× diamond, 32× emerald	\$3 000	600 pkt	3 dni	Odblokowanie customowego GUI gildii
5	50	22	1× netherite_ingot, 128× obsidian	\$5 000	1 000 pkt	5 dni	Dostęp do systemu dyplomatycznego (wojny)
6	60	28	3× netherite_ingot, 256× quartz_block	\$8 000	1 500 pkt	7 dni	Możliwość ustawienia 2 respawn pointów

7	72	35	5× netherite_ingot, 64× ender_pearl	\$12 000	2 200 pkt	10 dni	Sojusze: max 4; efekt Regeneration I w spawn-zone
8	85	42	10× netherite_ingot, 128× amethyst_shard	\$18 000	3 000 pkt	14 dni	Ochrona TNT zmniejszona o 25% (szybsze przebicie)
9	95	48	16× netherite_ingot, 1× nether_star	\$25 000	4 000 pkt	21 dni	Dostęp do elytra-spawn w cuboidzie
10	100	50	32× netherite_ingot, 3× nether_star	\$50 000	6 000 pkt	30 dni	Maksymalny poziom; efekt Speed I w spawn-zone

Analiza tabeli leveli ujawnia kilka założeń balansu. Krzywa wzrostu promienia jest najbardziej stroma na niskich poziomach (level 1 → 2: +47%, level 9 → 10: +5%), co zapewnia nowym gildiom szybką ekspansję początkową przy jednoczesnym wymogu długoterminowego zaangażowania dla maksymalnego terytorium. Wymagania ekonomiczne rosną wykładniczo z \$500 do \$50 000, tworząc naturalny sink walutowy i zachęcając do aktywnej gry ekonomicznej. Wymagania przedmiotowe ewoluują od surowców podstawowych (stone, iron) przez średnie (diamond, emerald) do end-game materials (netherite, nether star), co mapuje standardową progresję gracza w Minecraft. Bonusy mechaniczne na wyższych levelach — efekty beaconowe, dodatkowe respawny, elytra-spawn — zapewniają poza terytorialnymi korzyściami także wymierne przewagi rozgrywkowe, motywując do dążenia do poziomu 10.

Każdy parametr w tabeli jest w pełni konfigurowalny. Administrator może dostosować krzywą wzrostu promienia, zmienić wymagane przedmioty (używając nazw Material z Bukkit API), dostosować koszty do ekonomii serwera oraz dodać własne bonusy. Struktura pliku `modules/cuboid.yml` przechowuje definicje jako listę map YAML, co pozwala na łatwe dodawanie poziomów powyżej 10.

6.2.2 Proces awansu: `/guild levelup` z potwierdzeniem

Awans inicjowany jest komendą `/guild levelup`, dostępną dla lidera lub graczy z uprawnieniem `guild.levelup`. Komenda uruchamia wieloetapowy pipeline weryfikacji:

1. **Sprawdzenie poziomu** — jeśli gildia osiągnęła poziom maksymalny, system zwraca błąd (`level.max-level-reached`).
2. **Weryfikacja wieku** — porównanie timestampu utworzenia gildii z bieżącym czasem; komunikat zawiera dokładny czas pozostały do spełnienia warunku.
3. **Weryfikacja punktów rankingowych** — sprawdzenie sumy punktów członków i celów gildii.
4. **Weryfikacja środków EcoCore** — jeśli moduł ekonomii jest aktywny, system sprawdza stan konta gildii lub lidera (konfigurowalne: `levelup.payment-source`).
5. **Weryfikacja przedmiotów** — skan ekwipunku lidera pod kątem wymaganych materiałów; każdy stack musi być w jednym slocie (brak sumowania stacków częściowych).

6. **Potwierdzenie** — podsumowanie kosztów z prośbą o potwierdzenie `/guild levelup confirm` w ciągu 30 sekund, co zapobiega przypadkowemu zużyciu materiałów.
7. **Egzekucja** — pobranie zasobów, zwiększenie poziomu, aktualizacja promienia w `RegionManager`, broadcast do członków oraz efektowny efekt cząsteczek w centrum cuboidu.

Awans propaguje się natychmiastowo — nowy promień aktywuje się bez restartu serwera, a gracze w rozszerzonym terytorium automatycznie otrzymują powiadomienie wejścia do cuboidu.

```
# Fragment konfiguracji modules/cuboid.yml – definicja leveli
levels:
  '1':
    radius: 15
    max-members: 5
    requirements:
      items: []
      ecocore: 0
      points: 0
      age-hours: 0
  '2':
    radius: 22
    max-members: 8
    requirements:
      items:
        - "STONE:64"
        - "IRON_INGOT:32"
      ecocore: 500
      points: 100
      age-hours: 24
```

6.2.3 Tymczasowe rozszerzenia: boosty weekendowe, eventowe i ze sklepu

Oprócz trwałego awansu leveli, system wspiera **tymczasowe modyfikatory promienia** w postaci mnożników bazowego promienia, ze sobą współistniejących z różnymi priorytetami.

Boosty weekendowe aktywowane są automatycznie przez harmonogram czasowy. Domyślnie w każdy weekend (sobota–niedziela) wszystkie gildie otrzymują mnożnik `×1.3` promienia. Gildia na levelu 5 z bazowym promieniem 50 bloków posiada w weekend efektywny promień 65 bloków. Boost nie wymaga akcji od graczy — jest transparentnie aplikowany przez system czasowy.

Boosty eventowe to ręcznie aktywowane mnożniki dla administratorów. Komenda `/guild admin boost activate <mnożnik> <czas> [tag_gildii]` przyznaje dowolny mnożnik (1.1–3.0) na określony czas, opcjonalnie dla konkretnej gildii (brak argumentu = globalny boost). Zastosowania: eventy świąteczne, nagrody turniejowe, kompensaty za przestoje serwera.

Boosty ze sklepu to opcjonalne mnożniki zakupione za realną walutę. Po weryfikacji transakcji system przyznaje boost o określonym mnożniku i czasie trwania. Typowe pakiety: "Mini Boost" (×1.2 na 24h),

"Mega Boost" ($\times 1.5$ na 72h), "Premium Boost" ($\times 2.0$ na tydzień). Boosty płatne kumulują się addytywnie z weekendowymi, ale nie z innymi płatnymi — aktywowany jest zawsze najwyższy mnożnik płatny.

```
# Fragment konfiguracji modules/cuboid.yml – system boostów
boosts:
  weekend:
    enabled: true
    multiplier: 1.3
    active-days: [SATURDAY, SUNDAY]
  event:
    max-multiplier: 3.0
  shop:
    stacking: additive
    queue-enabled: true
```

6.3 System Flag i Ochrony (Własny — bez WorldGuard)

System flag stanowi serce własnego silnika ochrony. Plugin implementuje **wewnętrzny evaluator uprawnień**, który przechwytuje zdarzenia Bukkit/Spigot i decyduje o zablokowaniu lub dopuszczeniu akcji na podstawie skonfigurowanych flag i stref. Mechanizm opiera się na listenerach zarejestrowanych priorytetem `HIGHEST`, co gwarantuje, że decyzja pluginu nadpisuje decyzje innych pluginów.

Pipeline decyzyjny dla każdego zdarzenia: (1) określenie współrzędnych i wyszukanie aktywnego cuboidu w `RegionManager`, (2) rozpoznanie strefy wewnątrz cuboidu, (3) ewaluacja flagi i porównanie tożsamości gracza, (4) podjęcie decyzji o anulowaniu lub dopuszczeniu akcji.

6.3.1 Siedem flag ochrony — konfigurowalne per-gildia

Plugin definiuje siedem flag ochrony, każda z pięcioma możliwymi wartościami. Flagi konfiguruje lider komendą `/guild flag <nazwa> <wartość>` lub przez GUI (`/guild settings`).

Flaga	Zakres ochrony	Wartości	Domyślna	Przechwytywane zdarzenia
build	Stawianie bloków	ALLOW / DENY / MEMBERS / ALLIES / LEADER	MEMBERS	<code>BlockPlaceEvent</code> , <code>BlockMultiPlaceEvent</code>
destroy	Niszczanie bloków	ALLOW / DENY / MEMBERS / ALLIES / LEADER	MEMBERS	<code>BlockBreakEvent</code> , <code>HangingBreakEvent</code>

pvp	Walka graczy	ALLOW / DENY / MEMBERS / ALLIES / LEADER	DENY	EntityDamageByEntityEvent (gracz ↔ gracz)
use	Otwieranie kontenerów	ALLOW / DENY / MEMBERS / ALLIES / LEADER	MEMBERS	PlayerInteractEvent (chest, furnace, hopper, barrel, shulker)
bucket	Użycie wiader	ALLOW / DENY / MEMBERS / ALLIES / LEADER	LEADER	PlayerBucketEmptyEvent , PlayerBucketFillEvent
entry	Wejście na teren	ALLOW / DENY / MEMBERS / ALLIES / LEADER	ALLOW	PlayerMoveEvent (teleport / push out)
interact	Przyciski, dźwignie, drzwi	ALLOW / DENY / MEMBERS / ALLIES / LEADER	MEMBERS	PlayerInteractEvent (buttons, levers, doors, trapdoors, plates)

Semantyka wartości flag: **ALLOW** — każdy gracz, w tym obcy, może wykonać akcję (przydatne dla publicznych sklepów lub stref PvP). **DENY** — nikt nie może wykonać akcji, z wyjątkiem administratorów z `fgc.admin.bypass` (stosowane do całkowitego zamknięcia strefy). **MEMBERS** — tylko członkowie gildii z odpowiednią rangą (domyślnie wszyscy). **ALLIES** — członkowie oraz gracze z gildii sojuszniczych (sojusz musi być wzajemny). **LEADER** — wyłącznie lider i zastępcy z permission `guild.flag.leaderoverride`.

Flaga `pvp` z domyślną wartością `DENY` oznacza, że cuboid gildii jest domyślnie **strefą bezpieczną**. Zmiana tej flagi na `ALLOW` wymaga świadomej decyzji lidera i zalecana jest wyłącznie w dedykowanej strefie PvP (patrz podsekcja 6.3.2). Flaga `bucket` ograniczona do `LEADER` zabezpiecza przed griefingiem wodą i lawą — tylko lider może manipulować cieczami w obrębie cuboidu. Flaga `entry` ustawiona na `DENY` tworzy *zamknięty cuboid* — obcy gracze otrzymują komunikat `cuboid.entry-denied` i są fizycznie blokowani na granicy przez anti-clip mechanizm.

Zmiana dowolnej flagi logowana jest do pliku `logs/guild_flags.log` z timestampem i wartościami przed/po. Lider może przywrócić domyślne wartości komendą `/guild flag reset` (wymaga potwierdzenia).

6.3.2 Trzy strefy wewnątrz cuboidu: spawn, budowlana, PvP

Cuboid każdej gildii dzieli się na trzy koncentryczne strefy o różnym poziomie ochrony, determinujące strategię budowania bazy i model obrony.

Strefa	Zakres (od środku)	Flagi aktywne	Charakterystyka
Spawn (Bezpieczna)	0 – 8 bloków	build: LEADER, destroy: LEADER, pvp: DENY, use: MEMBERS, bucket: LEADER, entry: ALLOW, interact: MEMBERS	Strefa absolutnego bezpieczeństwa. Tylko lider modyfikuje bloki. Efekty beaconowe z leveli aktywne wyłącznie tu. Jajo gildii znajduje się w centrum. Gracze otrzymują efekty Regeneration I i Resistance I (jeśli odblokowane). Mechanicznie spawn to ostatnia linia obrony — nawet po przebicciu zewnętrznych warstw, strefa spawn pozostaje nie do zdobycia bez zniszczenia jaja.
Budowlana	8 – radiu s bloków	build: [skonfigurowana], destroy: [skonfigurowana], pvp: DENY, use/interact/bucket/entry: [skonfigurowane]	Główna strefa konstrukcyjna. Członkowie mogą swobodnie budować i niszczyć (domyślnie MEMBERS). PvP wyłączone — bezpieczna praca przy surowcach. Sojusznicy mogą uzyskać dostęp przez flagę ALLIES. Rozmiar zależy bezpośrednio od levelu gildii.
PvP (Zewnętrzna)	radiu s – radius +5 bloków	build: DENY, destroy: DENY, pvp: ALLOW, use/bucket/interact: DENY, entry: ALLOW	Opcjonalny pierścień walki aktywowany przez <code>/guild pvpzone toggle</code> . Sygnalizowany komunikatem ostrzegawczym i zmianą action bar. Walka dozwolona dla każdego, ale budowa i niszczenie zablokowane — zapobiega griefingowi podczas PvP.

Podział na trzy strefy tworzy hierarchiczną strukturę obrony. Strefa PvP to pole walki, gdzie obrońcy konfrontują nacierających bez ryzyka zniszczenia infrastruktury. Strefa budowlana to magazyny, farmy i warsztaty — chronione przed PvP, dostępne dla członków do modyfikacji. Strefa spawn to bunkier bezpieczeństwa z jajem gildii, dostępnym wyłącznie dla lidera — ostateczny punkt oporu.

Strefy definiuje się w `modules/cuboid.yml`:


```

zones:
  safe-zone:
    radius: 8
    force-leader-build: true
    force-deny-pvp: true
    beacon-effects:
      enabled: true
      effects:
        - "REGENERATION:0"
        - "RESISTANCE:0"
  build-zone:
    inherits-flags: true
  pvp-zone:
    enabled: false
    width: 5
    force-deny-build: true
    force-allow-pvp: true
    enter-warning: "&Wchodzisz na teren PvP! Walka dozwolona!"

```

Strefa PvP istnieje *poza* formalnym promieniem cuboidu jako pierścień otaczający region. Walka w niej nie podlega standardowym flagom cuboidu, lecz dedykowanym regułom strefy. Lider może w każdej chwili dezaktywować strefę PvP — pierścień znika, a teren wraca do stanu domyślnego.

6.3.3 Ochrona przed TNT i wybuchami — integracja z jajem gildii

Własny system ochrony implementuje **dedykowany moduł anty-eksplozyjny**, niezależny od jakichkolwiek mechanizmów WorldGuard. Ochrona działa na dwóch poziomach: globalnym (konfiguracja serwera) oraz per-gildia (stan jaja).

Na poziomie globalnym parametr `explosion-protection.enabled` w `modules/cuboid.yml` kontroluje przechwytywanie zdarzeń `EntityExplodeEvent` i `BlockExplodeEvent`. Gdy ochrona jest aktywna, każda eksplozja podlega ewaluacji:

1. **Lokalizacja eksplozji** — jeśli epicentrum mieści się w obrębie cuboidu, aktywują się reguły ochrony gildii.
2. **Źródło wybuchu** — wybuchy wywołane przez członków gildii we własnym cuboidzie są **zawsze blokowane**.
3. **Stan jaja gildii** — decydujący element integracji z systemem jaj. Jajo posiada 3 "życia" (stany), a każdy wybuch TNT uszkadza je o 1 punkt:
4. **Jajo pełne (3/3)** — eksplozje wewnątrz cuboidu są w **100% blokowane**; TNT znika bez efektu.
5. **Jajo uszkodzone (2/3)** — eksplozje **niszczą 50% bloków** w promieniu, z wyjątkiem obsydianu i crying obsydianu (odporne). Jajo emituje particle `SMOKE`.

6. Jajo krytyczne (1/3) — eksplozje **niszczą 100% standardowych bloków** w promieniu; obsydian i crying obsidian pozostają nienaruszone. Jajo emituje particle `FLAME`.

7. Jajo zniszczone (0/3) — cuboid dezaktywowany, wszystkie flagi przestają obowiązywać, a gildia otrzymuje countdown do automatycznego rozwiązania (domyślnie 24h).

Obsydian i crying obsidian zachowują nietykalność niezależnie od stanu jaja. Bazy zbudowane z tych materiałów zachowują strukturę nawet przy zniszczonym jaju — to celowy zabieg balansujący nagradzający inwestycję w solidną budowlę.

Lider może naprawić jajo o jedno życie komendą `/guild egg repair`, wymagającą 64× obsydian, 16× diamond i \$5000 z EcoCore. Cooldown naprawy wynosi 6 godzin.

6.3.4 Trusted Players — zaufani gracze bez członkostwa

System **Trusted Players** rozwiązuje problem współpracy między gildiami bez konieczności formalnego sojuszu ani przyjmowania obcych do składu. Trusted player to gracz z indywidualnym uprawnieniem do wykonywania określonych akcji w cudzym cuboidzie bez bycia członkiem gildii.

Zarządzanie odbywa się przez komendy:

- `/guild trust add <nick>` — dodaje gracza (wymaga `guild.trust.manage`)
- `/guild trust remove <nick>` — usuwa gracza
- `/guild trust list` — wyświetla listę z datą dodania
- `/guild trust clear` — usuwa wszystkich (wymaga potwierdzenia)

Limit trusted wynosi domyślnie 10 per-gildia (`trust.max-per-guild`). Trusted gracze nie wliczają się do limitu członków. Ich uprawnienia definiuje osobny zestaw flag: `trust.build`, `trust.destroy`, `trust.use`, `trust.interact`, `trust.bucket` — każda przyjmuje `true` lub `false`. Domyślnie: `build=true`, `destroy=false`, `use=true`, `interact=true`, `bucket=false` — co pozwala na pomoc przy budowie i używanie mechanizmów, ale chroni przed niszczeniem struktur i wylewaniem cieczy.

Ewaluator uprawnień sprawdza tożsamość gracza w kolejności priorytetowej: administrator z `fgc.admin.bypass` → lider → członek → sojusznik → **trusted player** → obcy. Trusted gracz otrzymuje powitalny komunikat na wejściu (klucz `cuboid.trusted-welcome`), a jego status automatycznie wygasa po 30 dniach (`trust.auto-expire-days`) lub ręcznie przez lidera.

System znajduje zastosowanie przy: wynajmie budowniczych, współpracy z merchantami, tymczasowym dostępie dla sojuszników bez formalizowania sojuszu, czy pomocy technicznej moderatorów. Trusted players nie mogą używać komend gildii, nie otrzymują beacon efektów i nie są widoczni jako członkowie w tagu — ich status jest czysto funkcjonalny, bez przywilejów społecznych członkostwa.

6. Wieśniacy Handlowi

System Wieśniaków Handlowych stanowi integralny moduł pluginu Fraction Guild Clans, który umożliwia gildiom tworzenie wewnętrznej infrastruktury ekonomicznej. Wieśniacy (ang. *villagers*) funkcjonują jako spersonalizowane jednostki niezależające (Non-Player Character, NPC), zlokalizowane wyłącznie w obrębie cuboidu gildii, oferujące unikalne transakcje wymienne oraz zadania dzienne dla członków gildii. Mechanika ta wzbogaca rozgrywkę o element progresji ekonomicznej i buduje ekosystem wewnątrzgildyjny, w którym członkowie mogą wymieniać nadmiarowe surowce na przedmioty specjalistyczne, ekskluzywne enchantmenty lub walutę serwera.

6.1 Tworzenie Wieśniaków

6.1.1 Komenda `/guild villager create`

Proces tworzenia wieśniaka handlowego rozpoczyna się od wywołania komendy `/guild villager create <typ> <nazwa>`. Komenda ta dostępna jest wyłącznie dla graczy posiadających rangę **Lider** lub **Moderator** wewnątrz gildii, co zapewnia kontrolę nad rozwojem infrastruktury handlowej. Przy wywołaniu komendy plugin przeprowadza sekwencję walidacyjną: najpierw weryfikuje, czy gracz znajduje się fizycznie w obrębie cuboidu własnej gildii, następnie sprawdza, czy nie został przekroczony limit wieśniaków przypisanych do danego poziomu gildii, a na końcu waliduje poprawność typu wieśniaka (musi należeć do zdefiniowanej puli obsługiwanych typów, zob. sekcja 6.1.3).

W przypadku pomyślnej walidacji plugin generuje encję NPC w lokalizacji gracza wykonującego komendę. Wieśniak pojawia się jako zwykły wieśniak Minecraft z niestandardową profesją i nazwą wyświetlaną nad głową, formatowaną zgodnie z szablonem zdefiniowanym w pliku `config.yml`. Domyślnie nazwa przyjmuje formę `&e<nazwa>` (kolor żółty), a pod nią wyświetlany jest typ profesji w kolorze szarym. Lokalizacja spawnu wieśniaka zapisywana jest w bazie danych pluginu wraz z identyfikatorem UUID NPC, identyfikatorem gildii, typem profesji oraz stanem początkowym relacji. Wieśniak nie podlega standardowej mechanice zaludniania (ang. *village population*) z Minecraft — jest to encja zarządzana wyłącznie przez plugin Fraction Guild Clans, co oznacza, że nie reaguje on na dzwony, nie zmienia profesji w zależności od workstation ani nie jest podatny na ataki mobów (posiada flagę `invulnerable` ustawioną na `true`).

W celu zapewnienia estetycznego i funkcjonalnego rozmieszczenia NPC, zaleca się utworzenie dedykowanego pomieszczenia lub strefy handlowej wewnątrz gildii przed wywołaniem komendy. Gracz powinien stanąć w dokładnym miejscu, w którym wieśniak ma zostać zrespiony, ponieważ plugin nie oferuje narzędzia do precyzyjnego pozycjonowania — koordynaty NPC odpowiadają bezpośrednio pozycji gracza w momencie wykonania komendy, zaokrąglone do pełnych bloków.

6.1.2 Limit Wieśniaków Zależny od Poziomu Gildii

Liczba wieśniaków, które gildia może utrzymywać jednocześnie, powiązana jest ściśle z systemem poziomów cuboidu opisanym w rozdziale 5. Mechanizm ten zapobiega sytuacji, w której nowo powstałe gildie pierwszego poziomu dysponowałyby pełną infrastrukturą handlową, jednocześnie tworząc motywację do inwestowania w rozwój gildii. Konfiguracja limitów znajduje się w sekcji `villagers.limits` pliku `config.yml` i pozwala administratorowi serwera na dowolne skalowanie progów.

Domyślna konfiguracja przypisuje po jednym wieśniaku na poziomy 1–3, dwa na poziomy 4–6, trzy na poziomy 7–8 oraz maksymalnie cztery na poziomy 9–10. Administrator może jednak dowolnie modyfikować te wartości, definiując niestandardowy mnożnik lub precyzyjne przypisania per poziom. Przy próbie utworzenia wieśniaka po przekroczeniu limitu plugin zwraca komunikat błędu z szablonu `villager.limit_reached`, informując gracza o bieżącym limicie oraz poziomie wymaganym do jego zwiększenia. Komunikat ten obsługuje placeholderzy `%current%`, `%limit%` oraz `%required_level%`.

Limit obliczany jest dynamicznie przy każdej próbie utworzenia wieśniaka. Oznacza to, że jeśli gildia utraci poziom (np. w wyniku mechaniki rankingu), a liczba aktywnych wieśniaków przewyższa nowy limit, żaden z istniejących NPC nie zostanie usunięty automatycznie. System wprowadza jednak blokadę: gildia nie może utworzyć nowego wieśniaka, dopóki liczba aktywnych NPC nie spadnie poniżej obowiązującego limitu. Mechanizm ten zapewnia stabilność ekonomiczną gildii podczas fluktuacji rankingowych, jednocześnie zapobiegając wykorzystywaniu tymczasowych awansów do masowego tworzenia wieśniaków.

6.1.3 Typy Wieśniaków

Plugin definiuje sześć podstawowych typów wieśniaków handlowych, z których każdy specjalizuje się w innej kategorii ofert. Typ określa pulę dostępnych transakcji, estetykę NPC oraz kategorie zadań dziennych generowanych przez danego wieśniaka. Poniższa tabela przedstawia kompleksowe zestawienie typów wraz z ich charakterystyką.

Typ Wieśniaka	Profesja Wyświetlana	Kategoria Ofert	Przykładowe Transakcje	Specjalizacja Zadań
Handlarz	&6Handlarz	Ogólna wymiana surowców	Wymiana żelaza na złoto, sprzedaż surowców za walutę serwera, wymiana diamentów na obsydian	Dostarczanie stosów surowców, sprzedaż określonej liczby itemów
Kowal	&8Kowal	Przedmioty bojowe i narzędzia	Zakup żelaznego miecza z losowym enchantem, naprawa narzędzi za surowce, wymiana rud na gotowe przedmioty	Zdobycie określonej liczby zabójstw, dostarczenie rud metali, naprawa narzędzi

Alchemik	&dA lche mik	Mikstury i ingredencje	Zakup mikstury leczenia II, wymiana nether wart na proszek blaze'a, brew customowych mikstur	Zebranie ingrediencji alchemicznych, warzenie mikstur, dostarczenie specyficznych itemów
Bibliotekarz	&3B ibli otek arz	Enchanted Booki i wiedza	Zakup losowego Enchanted Booka, wymiana lapis lazuli na doświadczenie, zakup map eksploracyjnych	Zdobycie punktów doświadczenia, dostarczenie książek i papieru, przeczytanie (użycie) określonych itemów
Rybak	&9R yba k	Zasoby wodne i żywność	Zakup surowych dorszy, wymiana ryb na perły endermana, sprzedaż rzadkich ryb za walutę	Wyłowienie określonej liczby ryb, dostarczenie specyficznych gatunków ryb, hodowla akwaryczna
Łowca	&4Ł owc a	Drop z mobów i skóry	Zakup strzał, wymiana kości na sznurek, sprzedaż skór mobów, zakup specjalistycznych łuków	Zabicie określonej liczby mobów, dostarczenie dropu ze specyficznych stworzeń, zdobycie rzadkich trofeów

Analiza powyższej tabeli wykazuje, że system typów wieśniaków zaprojektowany został jako zamknięty ekosystem gospodarczy, w którym każda profesja odpowiada za inną gałąź wewnątrzgildyjnej ekonomii. Handlarz pełni funkcję wymiany ogólnej, konwertując nadmiarowe surowce na bardziej wartościowe materiały lub walutę serwera, co czyni go najbardziej uniwersalnym typem, zalecanym jako pierwszy wieśniak dla nowych gildii. Kowal i Alchemik adresują potrzeby bojowe — pierwszy dostarcza wyposażenie fizyczne, drugi zaś wsparcie konsumpcyjne w postaci mikstur. Bibliotekarz wprowadza element progresji magicznej poprzez dostęp do enchantowanych ksiąg, które trudniej pozyskać standardowymi metodami. Rybak i Łowca, z kolei, nagradzają graczy aktywnie uczestniczących w eksploracji świata, oferując konwersję dropu z mobów i rybołówstwa na przedmioty o wyższej wartości użytkowej. Administrator serwera może modyfikować pulę ofert każdego typu w pliku `villagers.yml`, dodawać nowe transakcje, usuwać istniejące lub zmieniać wymagane składniki wymiany, co pozwala na precyzyjne dopasowanie ekonomii do profilu rozgrywki na danym serwerze.

6.2 System Handlu

6.2.1 Konfiguracja Ofert Handlowych

Każda oferta handlowa (ang. *trade offer*) składa się z czterech elementów: przedmiotu wejściowego pierwszego (slot lewy), opcjonalnego przedmiotu wejściowego drugiego (slot prawy, zazwyczaj używany jako dodatkowa waluta lub surowiec wspomagający), przedmiotu wyjściowego (slot wynikowy) oraz limitu

transakcji definiującego maksymalną liczbę możliwych wymian przed resetem. Konfiguracja ofert odbywa się w pliku `villagers.yml`, gdzie dla każdego typu wieśniaka definiowana jest niezależna lista transakcji.

Pojedyncza oferta w pliku konfiguracyjnym przyjmuje następującą strukturę:

```
offers:
  - input1:
      material: IRON_INGOT
      amount: 16
      name: "&fSztabka Żelaza"
    input2:
      material: GOLD_INGOT
      amount: 4
    output:
      material: DIAMOND
      amount: 1
      enchantments:
        - DURABILITY:1
    max_trades: 8
    required_relation: FRIENDLY
```

Parametr `max_trades` definiuje, ile razy gracz może skorzystać z danej oferty przed jej wyczerpaniem. System resetu ofert (ang. *trade cycle reset*) odbywa się automatycznie w cyklach czasowych zdefiniowanych w `config.yml` pod kluczem `villagers.trade_reset_interval`. Domyślnie reset następuje co 24 godziny (wartość podawana w minutach: `1440`), co oznacza, że wszystkie limity transakcji dla wszystkich wieśniaków zostają przywrócone do wartości maksymalnych o wskazanej godzinie serwerowej. Administrator może skonfigurować również tryb resetu per-wieśniak lub globalny, a także wymusić natychmiastowy reset komendą `/guild admin villager reset <gildia>`.

Plugin obsługuje wszystkie typy przedmiotów dostępne w API Bukkit/Spigot, w tym przedmioty z niestandardowymi nazwami, lore, enchantami oraz flagami NBT (poprzez integrację z NBT API). Pozwala to na tworzenie zaawansowanych transakcji, takich jak wymiana surowców na enchantowane narzędzia z unikalnymi atrybutami lub customowe itemy z innych pluginów. Parametr `required_relation` pozwala na warunkowanie dostępności oferty od poziomu relacji gracza z danym wieśniakiem, co tworzy system progresji ofert, w którym lepsze transakcje odblokowywane są dopiero po zbudowaniu odpowiedniej relacji.

6.2.2 Poziomy Relacji z Wieśniakiem

System relacji z wieśniakiem (ang. *villager reputation system*) oparty jest na pięciu stopniach zażyłości, które gracz buduje poprzez regularne transakcje handlowe oraz wykonywanie zadań dziennych. Każdy poziom relacji wpływa na dwie kluczowe zmienne: mnożnik ceny ofert handlowych oraz dostępność ekskluzywnych transakcji zarezerwowanych dla wyższych szczebli relacji.

Progresja relacji mierzona jest w punktach reputacji, zdobywanych przy każdej udanej transakcji (standardowo 1–3 punkty za wymianę, w zależności od wartości oferty) oraz za wykonanie zadania

dziennego (standardowo 10–25 punktów). Punktacja jest specyficzna per gracz i per wieśniak, co oznacza, że relacja budowana jest indywidualnie z każdym NPC. Pięć poziomów relacji obejmuje: **Neutralny** (0–49 punktów, mnożnik cen `1.0`, dostępne podstawowe oferty), **Znajomy** (50–149 punktów, mnożnik `0.9`, odblokowanie ofert średniej jakości), **Przyjacielski** (150–299 punktów, mnożnik `0.8`, dostęp do rzadkich transakcji), **Zaufany** (300–499 punktów, mnożnik `0.7`, oferty ekskluzywne z unikalnymi przedmiotami) oraz **Mistrzowski** (500+ punktów, mnożnik `0.6`, najlepsze oferty z najwyższymi bonusami). Punkty relacji nie ulegają degradacji w czasie — system nie wprowadza mechaniki spadku reputacji, co zapewnia, że gracze nie tracą postępu przy krótszych przerwach w grze.

Mnożnik cen stosowany jest do przedmiotów wejściowych — im wyższa relacja, tym mniejsza liczba surowców wymagana do przeprowadzenia transakcji. Na przykład oferta wymagająca 16 sztabek żelaza na poziomie Neutralnym wymagać będzie zaledwie 10 sztabek na poziomie Mistrzowskim (zaokrąglenie w dół do pełnej liczby). Progi punktowe oraz wartości mnożników konfigurowalne są w sekcji `villagers.relations` pliku `config.yml`, co pozwala administratorowi na precyzyjne balansowanie tempa progresji ekonomicznej.

6.2.3 Ochrona przed Scamem

System handlu z wieśniakami wyposażony został w zabezpieczenia chroniące zarówno przed nadużyciami ze strony graczy, jak i przed nieautoryzowanym dostępem do infrastruktury handlowej gildii. Podstawowym mechanizmem ochronnym jest **wymóg członkostwa w gildii** — interakcja z wieśniakiem (prawy przycisk myszy) zwraca dostęp do GUI handlu wyłącznie graczom należącym do gildii, na której terenie NPC został zrespony. Gracze spoza gildii nie mogą otworzyć okna handlu ani w żaden sposób korzystać z ofert, a przy próbie interakcji otrzymują komunikat z szablonu `villager.not_member`.

Drugim zabezpieczeniem jest **cooldown transakcji globalnej**, który ogranicza częstotliwość wymian na poziomie całej gildii. Parametr `villagers.global_trade_cooldown` (domyślnie `0`, wyłączony) pozwala administratorowi na wymuszenie przerwy między kolejnymi transakcjami wykonywanymi przez dowolnego członka gildii. Mechanizm ten uniemożliwia sytuacje, w których grupa graczy masowo wykorzystuje jedną korzystną ofertę do błyskawicznej konwersji surowców, potencjalnie destabilizując ekonomię serwera. Cooldown mierzony jest w tickach serwera (20 ticków = 1 sekunda).

Trzecim elementem ochronnym jest **log transakcji**, który rejestruje każdą wymianę wykonaną z wieśniakiem, zapisując: nick gracza, UUID gracza, identyfikator transakcji, przedmioty wymienione, timestamp oraz aktualny poziom relacji. Log dostępny jest dla Lidera i Moderatorów gildii poprzez komendę `/guild villager log <nazwa_wieśniaka>`, co pozwala na audyt aktywności handlowej i wykrywanie nieprawidłowości. Logi przechowywane są przez domyślny okres 30 dni, po czym automatycznie archiwizowane lub usuwane w zależności od konfiguracji `villagers.log_retention_days`.

6.3 Zarządzanie

6.3.1 Przenoszenie, Usuwanie i Modyfikacja Wieśniaków

Administracja istniejącymi wieśniakami odbywa się za pomocą zestawu dedykowanych komend dostępnych dla Lidera i Moderatorów gildii. Komenda `/guild villager move <nazwa>` pozwala na przeniesienie NPC do aktualnej pozycji gracza — funkcja przydatna przy przebudowie strefy handlowej lub relokacji całej infrastruktury. Przy przenoszeniu plugin aktualizuje współrzędne w bazie danych i teleportuje encję NPC do nowej lokalizacji z efektem cząsteczek wizualnych. Komenda `/guild villager remove <nazwa>` trwale usuwa wieśniaka, zwracając gildii jeden slot limitu. Przed usunięciem plugin wymusza potwierdzenie operacji w formie GUI z przyciskami TAK/NIE, aby zapobiec przypadkowemu skasowaniu NPC wraz z całym postępem relacji. Komenda `/guild villager rename <stara_nazwa> <nowa_nazwa>` pozwala na zmianę wyświetlanej nazwy bez wpływu na poziom relacji i historię transakcji.

Dodatkowo, komenda `/guild villager info <nazwa>` wyświetla szczegółowe dane o wybranym NPC: typ profesji, poziom relacji wszystkich członków gildii (w formie rankingu), liczbę wykonanych transakcji, status limitów ofert oraz historię ostatnich dziesięciu transakcji. Uprawnienia do poszczególnych komend zarządzania przypisywane są poprzez system rang gildii (rozdział 3.2.1) — domyślnie pełne uprawnienia posiada Lider, Moderatorzy mogą przenosić i wyświetlać informacje, ale nie mogą usuwać wieśniaków bez zgody Lidera. Wszystkie operacji modyfikacyjne logowane są w centralnym rejestrze gildii z timestampem i identyfikatorem gracza wykonującego akcję.

6.3.2 System Zadań Dziennych od Wieśniaków

System zadań dziennych (ang. *daily quests*) rozszerza funkcjonalność handlu o mechanikę aktywnościową, w której każdy wieśniak oferuje członkom gildii jedno zadanie dziennie, losowane z puli specyficznej dla typu profesji. Zadania resetowane są o stałej porze serwerowej (domyślnie 04:00), co synchronizuje cykl aktywności wszystkich członków gildii. Pojedyncze zadanie składa się z celu do wykonania, nagrody za wykonanie oraz bonusowej nagrody za wykonanie wszystkich zadań danego dnia (tzw. *daily streak*).

Poniższa tabela przedstawia strukturę systemu zadań dziennych wraz z kategoriami celów, typowymi nagrodami oraz wpływem na relację z wieśniakiem.

Element Systemu	Opis Mechaniki	Konfiguracja
Cele Zadań	Zadania losowane z puli 10–15 celów per typ wieśniaka. Cele obejmują: dostarczenie stosu konkretnych surowców, wykonanie określonej liczby akcji (zabójstw, połowów, wytworzeń), zebranie rzadkiego dropu, odwiedzenie konkretnego regionu świata.	Plik <code>quests/<typ>.yml</code> , lista celów z wagami losowania

Nagroda Podstawowa	Nagroda przyznawana za pojedyncze wykonane zadanie. Może obejmować: walutę serwera (poprzez Vault), punkty doświadczenia, przedmioty materialne, punkty rankingu gildii, punkty relacji z danym wieśniakiem (10–25 pkt).	Sekcja <code>rewards.base</code> w pliku <code>quests/<typ>.yaml</code> , wsparcie dla <code>Material</code> , <code>money</code> , <code>exp</code> , <code>relation_points</code>
Bonus Streak	Dodatkowa nagroda przyznawana, gdy gracz wykona zadania od wszystkich aktywnych wieśniaków w gildii w ciągu jednego dnia. Bonus skalowany jest liniowo: $\text{base_reward} \times \text{streak_multiplier}$ (domyślnie <code>1.5</code>). Przerwa w streaku resetuje licznik.	<code>quests.streak_bonus_multiplier</code> oraz <code>quests.streak_required_all</code> w <code>config.yml</code>
Progresja Relacji	Każde wykonane zadanie przyznaje punkty relacji z konkretnym wieśniakiem (wartość zależna od trudności zadania: 10 pkt za łatwe, 15 pkt za średnie, 25 pkt za trudne). Punkty te sumują się z punktami z transakcji handlowych.	<code>quests.relation_reward_easy</code> , <code>quests.relation_reward_medium</code> , <code>quests.relation_reward_hard</code>
Współdzielenie Postępu	Postęp w zadaniach zbierany jest indywidualnie per gracz, jednak niektóre cele (np. "Dostarcz 5 stosów żelaza") mogą być skonfigurowane jako współdzielone — postęp sumuje się ze wszystkich członków gildii. Parametr <code>shared: true/false</code> per cel.	<code>quests.allow_shared_progress</code> (globalne włączenie) oraz flaga <code>shared</code> per zadanie
Limit Czasowy	Każde zadanie musi zostać wykonane przed kolejnym resetem dziennym. Niewykonane zadania przepadają bez kary, ale nie przyznają nagród. Gracz może śledzić postęp w GUI zadań (<code>/guild quests</code>).	<code>quests.daily_reset_hour</code> (domyślnie <code>4</code> , format 24h serwera)
GUI Zadań	Dedykowane okno graficzne otwierane komendą <code>/guild quests</code> , wyświetlające aktywne zadania od wszystkich wieśniaków, pasek postępu (w formacie X/Y), pozostały czas do resetu oraz podgląd nagród. GUI odświeżane jest co 20 ticków.	Konfiguracja slotów i wyglądu w <code>gui/quests_gui.yml</code>

Tabela powyższa demonstruje, że system zadań dziennych zaprojektowany został jako mechanika wspierająca aktywność członków gildii i współpracę wewnątrz grupy. Kluczową cechą jest dualny system nagród — indywidualny za pojedyncze zadanie oraz zbiorowy bonus streak — który motywuje graczy do wykonywania wszystkich dostępnych zadań, a nie tylko tych najłatwiejszych. Progresja relacji powiązana z wykonywaniem zadań tworzy pozytywną pętlę zwrotną: więcej zadań oznacza wyższą relację, wyższa relacja oznacza lepsze oferty handlowe, a lepsze oferty zachęcają do dalszego handlu i wykonywania zadań.

Mechanika współdzielenia postępu (opcjonalna) pozwala na kooperacyjne realizowanie zadań gromadniczych, co wzmacnia aspekt socjalny rozgrywki — na przykład zadanie wymagające dostarczenia 320 sztabek żelaza (5 stosów) może zostać rozłożone na kilku członków gildii, z których każdy dostarcza część wymaganej ilości. GUI zadań zapewnia przejrzysty interfejs do śledzenia aktywności, eliminując konieczność zapamiętywania celów i ułatwiając koordynację w grupie. Administrator może dowolnie modyfikować pulę zadań, balansować nagrody oraz dostosowywać poziom trudności poprzez edycję plików YAML, co pozwala na precyzyjne skalowanie mechaniki do profilu serwera — od trybów casual z łatwymi zadaniami i hojnymi nagrodami po tryby hardcore, w których zadania stanowią istotne wyzwanie ekonomiczne i logistyczne.

7. System Dołączania do Gildii

System dołączania do gildii stanowi bramę wejściową do wspólnoty graczy i składa się z dwóch niezależnych ścieżek: klasycznego zaproszenia wysyłanego przez lidera lub moderatora oraz opcjonalnego modelu pay-to-join, w którym gildia definiuje własną cenę członkostwa. Każda z tych ścieżek podlega konfiguracji w pliku `config.yml` i wspiera się wspólnym zestawem walidacji wymagań kandydata, zapewniając spójność rekrutacji niezależnie od wybranego modelu.

7.1 Standardowe Zaproszenie

Standardowe zaproszenie stanowi domyślną metodę rekrutacji w Fraction Guild Clans. Każda gildia posiadająca wolne sloty w limicie członków może samodzielnie wybrać nowych członków poprzez system zaproszeń z kontrolowanym czasem ważności. Model ten daje liderowi pełną kontrolę nad składem gildii, eliminując ryzyko automatycznego przyjęcia niepożądanych graczy.

7.1.1 Komenda `/guild invite` — wysyłanie zaproszenia, limit czasowy, akceptacja/odrzućenie

Komenda `/guild invite <nick>` umożliwia członkom gildii posiadającym odpowiednie uprawnienia (domyślnie ranga **Moderator** lub wyższa) wysłanie zaproszenia do wybranego gracza znajdującego się online. Po wykonaniu komendy system weryfikuje: rangę zapraszającego, limit członków gildii, przynależność kandydata do innej gildii oraz ewentualne aktywne bany. Po pozytywnej walidacji zaproszenie zostaje zapisane w bazie danych z sygnaturą czasową, a kandydat otrzymuje powiadomienie chatowe wraz z klikalnymi komendami akceptacji i odrzucenia.

Czas ważności zaproszenia to parametr konfigurowalny w sekcji `join-system` pliku `config.yml` — domyślnie **10 minut**. Po upływie tego czasu zaproszenie wygasa automatycznie, a rekord zostaje usunięty z bazy danych. Kandydat akceptuje zaproszenie komendą `/guild join <tag-gildii>` lub odrzuca przez `/guild invite decline <tag-gildii>`. Lider może anulować wszystkie wysłane zaproszenia

komendą `/guild invite cancel`. System dopuszcza jednocześnie maksymalnie **5 aktywnych zaproszeń** per gildia, co zapobiega spamowaniu.

7.1.2 Wymagania dla kandydata: minimalny czas gry, brak aktywnych banów, limit gildii na konto

Każdy gracz ubiegający się o członkostwo musi spełnić warunki zdefiniowane w konfiguracji pluginu. Po pierwsze, system weryfikuje **minimalny czas gry** (ang. *playtime*) — domyślnie 60 minut — który zapobiega tworzeniu kont jednorazowych. Czas gry jest odczytywany z wbudowanego trackera lub zewnętrznego systemu przez PlaceholderAPI.

Po drugie, kandydat nie może posiadać aktywnych banów ani blokad *guild-ban* nałożonych przez administratorów lub liderów innych gildii. *Guild-ban* to status uniemożliwiający dołączenie do dowolnej gildii przez konfigurowalny okres (domyślnie 7 dni). Po trzecie, system egzekwuje limit **jednej gildii na konto** — gracz należący już do jakiegokolwiek gildii nie może zaakceptować nowego zaproszenia bez opuszczenia obecnej wspólnoty komendą `/guild leave`. Opuśczenie gildii wiąże się z cooldownem 24 godzin przed dołączeniem do nowej, co zapobiega szybkiemu przeskakiwaniu między gildiami w celu szpiegowania.

7.2 Dołączanie za Itemy (Pay-to-Join)

System pay-to-join stanowi opcjonalną alternatywę dla standardowych zaproszeń, umożliwiając gildiom ustalenie ceny członkostwa w surowcach lub walucie serwera. Mechanika ta otwiera gildiom drogę do budowania ekonomicznego modelu organizacji, w którym wpływy z członkostwa mogą być przeznaczane na rozwój cuboidu, naprawę jaja lub inwestycje w wieśniaków handlowych.

7.2.1 Opcjonalny system włączany w config.yml — gildie mogą ustawić cenę dołączenia

Aktywacja systemu odbywa się przez ustawienie `pay-to-join.enabled: true` w sekcji `join-system` pliku `config.yml`. Lider gildii zyskuje wtedy dostęp do komendy `/guild setprice <kwota>`, która definiuje cenę dołączenia. Cena jest publicznie widoczna dla wszystkich graczy w GUI oraz w komendzie `/guild list`. System pay-to-join nie wyklucza standardowych zaproszeń — lider może akceptować oba modele jednocześnie lub dezaktywować płatne dołączanie ustawiając cenę na zero. Administrator definiuje globalne ramy cenowe przez `pay-to-join.min-price` i `pay-to-join.max-price`, zapobiegając nadużyciom.

7.2.2 Konfiguracja ceny: itemy lub waluta serwera, jednorazowa lub subskrypcja (co tydzień)

Plugin wspiera dwa modele płatności pay-to-join. W modelu **jednorazowym** gracz uiszcza opłatę w momencie dołączania i nie ponosi kolejnych kosztów. Płatność przyjmuje formę itemów (lista w formacie Material z ilością w sekcji `pay-to-join.price.items`) lub waluty serwera (przekazanej przez Vault). W modelu **subskrypcji tygodniowej** członek ponosi cykliczną opłatę co 7 dni, kontrolowaną przez parametr `pay-to-join.subscription.enabled`. Przy braku opłaty system obniża rangę gracza do **Rekruta** z

ograniczonymi uprawnieniami, a po 48 godzinach automatycznie usuwa go z gildii. Gracz otrzymuje powiadomienia 24, 12 i 1 godzinę przed terminem płatności, a cały proces jest rejestrowany w logu gildii.

7.2.3 Proces dołączania — przeglądanie dostępnych gildii z ceną, wybór, płatność, automatyczne nadanie rangi

Proces rozpoczyna się od komendy `/guild join list`, która wyświetla gildie z włączonym pay-to-join wraz z nazwą, tagiem, ceną i typem płatności. Po wyborze gildii gracz wykonuje `/guild join <tag>`, co inicjuje weryfikację środków w ekwipunku (itemy) lub na koncie bankowym (waluta). System wyświetla ekran potwierdzenia z pełną specyfikacją transakcji, a po akceptacji pobiera opłatę i automatycznie nadaje rangę **Rekrut**.

Nadanie rangi rekruta zamiast pełnoprawnego członka to celowa decyzja projektowa — pozwala liderowi ocenić nowego gracza przed awansem, zabezpieczając gildię przed natychmiastowym dostępem do cennych zasobów. Awans na członka odbywa się ręcznie komendą `/guild promote <nick>` po okresie próbnym.

7.2.4 System zwrotów i ochrona gracza — okres próbny, zwrot przy wyrzuceniu bez przyczyny

System pay-to-join wymaga mechanizmów ochronnych dla obu stron transakcji. Centralnym elementem jest **okres próbny** (ang. *trial period*) domyślnie trwający **48 godzin**, konfigurowany przez `pay-to-join.trial-period-hours`. W tym czasie nowy członek ma ograniczone uprawnienia — może poruszać się po cuboidzie, ale nie ma dostępu do skrzyń o wartości powyżej progu ani do funkcji administracyjnych. Jednocześnie lider nie może wyrzucić nowego członka bez podania powodu (minimum 10 znaków, logowany w bazie), co chroni przed przyjęciem opłaty i natychmiastowym usunięciem.

Po okresie próbnym obowiązują standardowe zasady zwrotów. Zwrot kosztów jest możliwy tylko w sytuacjach rozpoznawanych automatycznie przez system na podstawie logów aktywności.

Warunek zwrotu	Zwrot jednorazowy	Zwrot subskrypcji	Opis mechanizmu
Wyrzucenie bez powodu w okresie próbnym	100% kosztu	100% ostatniej płatności	Automatyczny zwrot; lider otrzymuje ostrzeżenie w logach admina
Wyrzucenie z uzasadnionym powodem (regulamin)	0%	0%	Brak zwrotu; system weryfikuje powód pod kątem słów kluczowych
Rozwiązanie gildii przez lidera	100% kosztu	Proporcjonalny do pozostałych dni	Pełny zwrot w modelu jednorazowym; subskrypcja liczona do dnia

Samodzielne gildii	opuszczenie	0%	0%	Decyzja gracza; brak roszczeń
Usunięcie nieaktywność (30 dni)	przez	50% kosztu	50% płatności	ostatniej Połowa zwrotu jako gest dobrej woli; konfigurowalne
Błąd serwera / rollback danych		100%	100%	Pełny zwrot; realizowany przez /guild admin refund <nick>

Tabela przedstawia kompletny zbiór scenariuszy obsługiwanych przez system zwrotów pluginu Fraction Guild Clans. Mechanizm równoważy interesy ekonomiczne gildii z ochroną praw nowych członków, eliminując ryzyko oszustw przy rekrutacji płatnej. Zwroty realizowane są automatycznie — dla płatności itemami gracz otrzymuje zwrot do ekwipunku (lub do skrzynki depozytowej przy pełnym ekwipunku), dla waluty środki wracają przez API Vault. Parametr `pay-to-join.refund-chest-location` pozwala zdefiniować globalną skrzynkę depozytową na mapie. Zwroty subskrypcji są proporcjonalne do niewykorzystanego czasu z dokładnością do pełnego dnia. Administrator może nadpisać decyzję systemu komendą `/guild admin refund`, co przydaje się w spornych sytuacjach wymagających indywidualnej oceny. Cały proces zwrotu jest logowany w bazie danych z identyfikatorem transakcji, sygnaturą czasową i powodem, co zapewnia pełną audytowalność operacji ekonomicznych w obrębie pluginu.

8. GUI — Interfejs Graficzny

Interfejs graficzny (ang. *Graphical User Interface*, GUI) stanowi centralny punkt interakcji gracza z systemem gildii w pluginie Fraction Guild Clans. Zamiast wymagać od użytkownika zapamiętywania dziesiątek komend tekstowych, plugin oferuje w pełni funkcjonalne menu oparte na mechanice skrzyni w Minecraft (ang. *chest inventory*). Każdy element GUI jest klikalnym przedmiotem (ang. *icon*) reprezentującym określoną funkcję lub sekcję zarządzania gildią. Architektura interfejsu opiera się na modularnej strukturze, w której główne menu pełni rolę nawigacyjnego hubu, a poszczególne podmenu realizują wyspecjalizowane zadania — od zarządzania członkami, przez konfigurację cuboidu, aż po handel z wieśniakiem NPC. Komenda wywołująca główne menu to `/guild menu` (alias `/g menu`), dostępna dla każdego członka gildii posiadającego uprawnienie `guild.gui.menu`.

8.1 Główne Menu Gildii (/guild menu)

8.1.1 Struktura GUI — 6 rzędów, układ slotów, dekoracyjne ramki, główne sekcje

Główne menu gildii renderowane jest jako podwójna skrzynia (ang. *double chest*), co daje 54 sloty w 6 rzędach po 9 slotów każdy. Rozkład slotów zaprojektowano według wzorca "ramka-sekcja-ramka": zewnętrzne sloty tworzą obramówkę wizualną, a sloty wewnętrzne zawierają elementy interaktywne. Ramka

dekoracyjna zajmuje zawsze sloty zewnętrzne: rząd 0 (sloty 0–8), rząd 5 (sloty 45–53) oraz skrajne sloty boczne (9, 18, 27, 36 i 17, 26, 35, 44). Wypełnienie tych slotów stanowią kolorowe szyby (ang. *stained glass panes*) lub niestandardowe głowice (ang. *custom player heads*), których typ definiuje administrator w pliku `gui/main-menu.yml`. Sekcja centralna obejmuje sloty 10–16, 19–25 oraz 28–34.

Poniższa tabela przedstawia szczegółowy rozkład slotów w głównym menu. Sloty oznaczone jako "dekoracja" nie posiadają akcji po kliknięciu (event jest anulowany z dźwiękiem `BLOCK_NOTE_BLOCK_BASS`).

Slot(y)	Rząd	Typ	Zawartość	Akcja po kliknięciu
0–8	1	Dekoracja	Kolorowe szkło / głowice wzór	Brak (blokada)
9, 17	2	Dekoracja	Szkło w kolorze akcentu gildii	Brak (blokada)
10–12	2	Informacyjny	Główka lidera + nazwa i tag gildii	Tooltip z datą założenia
13	2	Informacyjny	Ikona poziomu (numer poziomu cuboidu)	Prog następnego poziomu
14–16	2	Informacyjny	Ikona rankingu (pozycja + punkty)	Przełącznik: punkty / zabójstwa / poziom
18, 26	3	Dekoracja	Szkło w kolorze akcentu gildii	Brak (blokada)
19–21	3	Nawigacyjny	Główki 3 ostatnio aktywnych członków	Otwiera GUI zarządzania członkami
22	3	Nawigacyjny	Książka — ikona "Ustawienia"	Otwiera GUI ustawień gildii
23–25	3	Nawigacyjny	Blok trawy — ikona "Cuboid"	Otwiera GUI informacji o cuboidzie
27, 35	4	Dekoracja	Szkło w kolorze akcentu gildii	Brak (blokada)
28	4	Nawigacyjny	Ikona wieśniaka — "Wieśniacy"	Otwiera GUI zarządzania wieśniakami
29	4	Nawigacyjny	Flaga — ikona "Sojusze"	Otwiera GUI sojuszy
30	4	Nawigacyjny	Miecz — ikona "Wojny"	Otwiera GUI wojen z deklaracjami
31–33	4	Funkcyjny	Rozwiąż gildię / Opublikuj info / Help	Akcja z potwierdzeniem (dla rozwiązania)
34	4	Nawigacyjny	Strzałka — "Ranking serwera"	Otwiera GUI rankingu gildii

36–44	5	Dekoracja	Szkło + stopka z tekstem	Brak (blokada)
45–53	6	Dekoracja	Dolna ramka z wersją pluginu	Brak (blokada)

Analiza struktury slotowej ujawnia celowe zastosowanie hierarchii w rozmieszczeniu elementów. Sekcja informacyjna (sloty 10–16) zajmuje górny centralny obszar — gracz po otwarciu GUI widzi najpierw nazwę gildii, jej poziom oraz pozycję rankingową. Sekcja zarządcza (sloty 19–34) podzielona jest na cztery podsekcje: członkowie (19–21), ustawienia i cuboid (22–25), wieśniacy oraz relacje międzygildyjne (28–30) i funkcje pomocnicze (31–34). Taki układ minimalizuje liczbę kliknięć potrzebnych do wykonania akcji — średnia ścieżka kliknięcia (ang. *click path*) od otwarcia menu do wykonania operacji wynosi 1,4 dla akcji podstawowych i 2,0 dla operacji zaawansowanych. Dekoracyjna ramka nie tylko pełni funkcję estetyczną, ale także zapobiega przypadkowemu przesunięciu przedmiotu z ekwipunku gracza do slotu GUI, ponieważ wszystkie zewnętrzne sloty mają anulowany event `InventoryClickEvent`.

8.1.2 Sekcja informacyjna: nazwa, tag, poziom, liczba członków, ranking, punkty

Sekcja informacyjna zajmuje sloty 10–16 w rzędzie 2 i stanowi pulpit wyświetlający najważniejsze dane o gildii. Slot 10 zawiera głowicę gracza (ang. *player head*) lidera uzyskiwaną przez API Mojang na podstawie UUID. Na lore tej głowki wyświetlane są: pełna nazwa gildii, tag w formacie `[TAG]` oraz data założenia.

Slot 13 prezentuje aktualny poziom cuboidu — ikona odpowiada materiałowi z pliku `gui/icons.yml` (domyślnie EMERALD_BLOCK dla poziomów 1–3, DIAMOND_BLOCK dla 4–6, OBSIDIAN dla 7–9, BEACON dla poziomu 10). Lore zawiera: numer poziomu, promień cuboidu w blokach, liczbę członków `Aktywni: X / Y` oraz wymagania do awansu z procentowym wskaźnikiem ukończenia.

Sloty 14–16 agregują dane rankingowe. Ikona (domyślnie złoty miecz) wyświetla w lore: pozycję w globalnym rankingu, punkty rankingowe, liczbę zabójstw i śmierci członków oraz wskaźnik K/D gildii. Kliknięcie prawym przyciskiem przełącza tryb sortowania między punktami, zabójstwami a poziomem cuboidu — wartości aktualizują się bez zamykania GUI.

8.1.3 Sekcja zarządzania: członkowie, ustawienia, cuboid, wieśniacy, sojusze

Sloty 19–21 wyświetlają głowy trzech ostatnio aktywnych członków sortowanych według timestampu ostatniego wejścia. Każda głowka zawiera nick, rangę oraz czas ostatniej aktywności. Kliknięcie otwiera pełne GUI listy członków z paginacją.

Slot 22 (ikona książki) prowadzi do GUI ustawień, gdzie lider i moderatorzy modyfikują opis, powitanie (ang. *motd*), kolory, flagi dostępu oraz powiadomienia. Sloty 23–25 (blok trawy) otwierają GUI cuboidu z mapą 2D terenu, aktualnym promieniem i przyciskami rozszerzenia. Slot 28 (głowka wieśniaka) otwiera GUI zarządzania NPC handlowymi. Slot 29 (flaga) prowadzi do GUI sojuszy z listą aktywnych sojuszy i oczekującymi zaproszeniami. Slot 30 (miecz) otwiera GUI deklaracji wojen.

8.1.4 Personalizacja GUI — konfiguracja tytułu, typu tła (glass pane, custom heads), kolorów

Wygląd menu konfiguruje administrator w pliku `gui/main-menu.yml`. Parametr `title` obsługuje kody kolorów Minecraft (&) oraz placeholder PlaceholderAPI, np. `%guild_name%`. Domyślnie: `&8[&c%guild_tag%&8] &7Menu Gildii`.

Parametr `frame.type` przyjmuje wartości: `STAINED_GLASS_PANE` (kolorowe szyby — domyślnie), `CUSTOM_HEAD` (główce z bazy Mojang) lub `EMPTY` (brak dekoracji). Dla szyb parametr `frame.color` definiuje kolor z palety barwników (WHITE przez BLACK). Dla główek parametr `frame.head-texture` przyjmuje URL tekstury w Base64. Kolor akcentu definiuje sekcja `accent-color` i może być nadpisywany per-gildia przez lidera (uprawnienie `guild.gui.customize`).

```
# gui/main-menu.yml – przykładowa konfiguracja
menu:
  title: "&8[&c%guild_tag%&8] &7Menu Gildii"
  rows: 6
  frame:
    type: STAINED_GLASS_PANE
    color: GRAY
  accent-color: "&c"
  refresh-interval: 20
```

8.2 Podmenu

System GUI realizuje architekturę wielopoziomową, w której główne menu jest węzłem nadrzędnym, a każde podmenu stanowi osobny ekran z dedykowaną funkcjonalnością. Każde podmenu posiada własny plik konfiguracyjny w folderze `gui/submenus/`. Nawigacja między poziomami odbywa się przez ikonę nawigacyjną (wejście) lub ikonę powrotu (slot 49 każdego podmenu), która zamyka bieżące okno i przywraca poprzedni ekran ze zachowaniem stanu (ang. *stateful navigation*).

Poniższa tabela zestawia wszystkie podmenu z ich kluczowymi parametrami: rozmiarem, przeznaczeniem, liczbą akcji interaktywnych oraz wymaganym uprawnieniem.

Podmenu	Rozmiar (rzędy)	Główne przeznaczenie	Liczba akcji	Wymagane uprawnienie
GUI Członków	6	Przegląd listy członków z głowicami, nadawanie rang, wyrzucanie, wyciszanie	8	<code>guild.gui.members</code>
GUI Ustawień	5	Edycja opisu, kolorów, flag, powiadomień; przełączniki toggle	12	<code>guild.gui.settings</code>

GUI Cuboidu	5	Mapa 2D terenu, status ochrony, awans poziomu, flagi strefowe	6	<code>guild.gui.cuboid</code>
GUI Wieśniaków	6	Lista NPC handlowych, zarządzanie ofertami, status relacji	5	<code>guild.gui.villager</code>
GUI Sojuszy	5	Aktywne sojusze, zaproszenia, historia konfliktów	4	<code>guild.gui.alliance</code>
GUI Wojen	5	Deklaracje wojen, status konfliktów, historia bitew	3	<code>guild.gui.war</code>
GUI Rankingu	6	TOP gildii, filtrowanie, podgląd statystyk	4	<code>guild.gui.ranking</code>
GUI Handlu	4	Oferty handlowe wieśniaka, wykonywanie transakcji	2	<code>guild.gui.trade</code>

Analiza tabeli ujawnia kluczowe różnice architektoniczne między podmenu. Najbardziej rozbudowanym jest GUI Członków (6 rzędów, 8 akcji), co wynika z konieczności obsługi paginacji — sloty 45–53 zarezerwowane są na kontrolki stronicowania, a sloty 0–44 prezentują główki w układzie 5×9. GUI Ustawień i Cuboidu przyjęły format 5-rzędowy, ponieważ ich zawartość jest statyczna lub wymaga jedynie pojedynczych przełączników. Interesującą decyzją projektową jest rozdzielenie handlu na GUI Wieśniaków (zarządzanie NPC) i GUI Handlu (transakcje), co separuje uprawnienia administracyjne od zwykłych członków. System uprawnień dziedziczy strukturę `guild.gui.<nazwa>`, a akcje wewnątrz podmenu dodatkowo weryfikują rangę w hierarchii gildii — np. kick wymaga rangi Moderator, a edycja opisu może być dostępna dla każdego członka w zależności od flagi `settings.edit-description-public`.

8.2.1 GUI zarządzania członkami — lista z głowicami graczy, akcje (promote, kick, mute), stronicowanie

GUI członków (6 rzędów) prezentuje listę wszystkich członków w postaci główek posortowanych według rangi (Lider → Moderator → Członek → Rekrut), a w obrębie rangi alfabetycznie. Każda główka wyświetla: nick z kolorem rangi, datę ostatniego wejścia, status online/offline (modyfikacja enchantu) oraz statystyki (zabójstwa, śmierci, zadania).

Kliknięcie lewym przyciskiem na główce otwiera panel akcji kontekstowych — miniaturowe GUI 1-rzędowe z przyciskami: PROMOTE (awans, dla Lidera/Moderatora przy celu niższej rangi), DEMOTE (degradacja, tylko Lider), KICK (wyrzucenie z potwierdzeniem), MUTE/UNMUTE (wyciszenie czatu na konfigurowalny czas), STATS (rozbudowane statystyki) oraz MESSAGE (wiadomość prywatna). Panel filtruje niedostępne opcje — Moderator nie widzi DEMOTE dla innego Moderadora, a Członek nie widzi opcji administracyjnych.

Stronicowanie w slotach 45–53: "Poprzednia strona" (48, nieaktywna na pierwszej stronie), "Następna strona" (50, nieaktywna na ostatniej) oraz "Powrót" (49). Liczba członków na stronę to 36, co przy limicie 50+ członków na poziomie 10 wymaga maksymalnie 2 stron.

8.2.2 GUI ustawień gildii — przełączniki (toggle), edycja opisu, zmiana kolorów, flagi

GUI ustawień (5 rzędów) organizuje opcje w cztery kategorie. Rząd 1 to nagłówek z nazwą gildii i kolorem akcentu. Rzędy 2–3 zawierają przełączniki (ang. *toggles*): włączony to blok szmaragdowy z enchantem, wyłączony — blok redstone'u bez enchantu. Dostępne toggles: `public-guild`, `invite-only`, `friendly-fire`, `mob-spawning`, `pvp-in-cuboid`, `ally-chat`.

Rząd 4 zawiera elementy edycji tekstowej: książka z piórem (slot 30, edycja opisu przez anvil z walidacją do 64 znaków), barwnik (slot 31, paleta 16 kolorów), flaga (slot 32, sub-GUI z flagami dostępu: budowanie przez sojuszników, użycie skrzyń, wejście na teren). Rząd 5 zawiera ZAPISZ (slot 40, zielona wełna) i POWRÓT (slot 49). Zmiany buforowane są w pamięci do momentu kliknięcia ZAPISZ — zamknięcie GUI bez zapisu przywraca poprzednie wartości.

8.2.3 GUI handlu z wieśniakiem — wyświetlanie ofert, status relacji, dostępne zadania

GUI handlu (4 rzędy) otwiera się po kliknięciu prawym przyciskiem na wieśniaka NPC w obrębie cuboidu. Górny rząd to pasek informacyjny: główka wieśniaka z imieniem i typem (Handlarz/Kowal/Alchemik/Bibliotekarz/Rybak/Łowca), pasek relacji zbudowany z bloków wełny (zielona/żółta/czerwona) reprezentujących poziom Neutralny → Zaufany → Przyjacielski → Mistrzowski, oraz ikona zadania dziennego z licznikiem postępu.

Rzędy 2–3 (sloty 9–26) prezentują oferty handlowe. Każda oferta zajmuje 2 sloty: lewy z wymaganymi itemami, prawy z nagrodą. Kliknięcie wykonuje transakcję jeśli gracz posiada itemy — w przeciwnym razie słychać `ENTITY_VILLAGER_NO`. W lore wyświetlany jest licznik `Transakcje: X / Y` jeśli skonfigurowano limit. Poziom relacji wpływa na ceny — każdy poziom powyżej Neutralnego redukuje wymagane itemy o 5%. Dolny rząd zawiera przycisk zadań (slot 30) i zamknięcia (slot 35).

8.2.4 GUI rankingu — lista top gildii, filtrowanie (punkty, zabójstwa, poziom), stronicowanie

GUI rankingu (6 rzędów) jest jedynym podmenu dostępnym bez członkostwa w gildii (uprawnienie `guild.guiranking`, domyślnie wszyscy gracze). Górny rząd zawiera kontrolki: slot 2 (szmaragd) sortuje po punktach, slot 4 (miecz) po zabójstwach, slot 6 (szlak) po poziomie cuboidu. Aktywny filtr ma enchant połysku. Slot 8 (zegar) przełącza zakres czasowy: cały czas / sezon / tydzień / dzień.

Rzędy 2–5 wyświetlają 20 najlepszych gildii. Każda pozycja to 1 slot z: numerem pozycji, główką lidera, tagiem `[TAG]`, nazwą gildii (16 znaków) oraz lore z wartością sortowania, liczbą członków i datą aktywności. Pozycje 1–3 wyróżnione materiałami: złoto, żelazo, miedź (lub kamień dla 1.8). Kontrolki

stronicowania w rzędzie 6 pozwalają przeglądać pozycje poza TOP 20. Kliknięcie na pozycję otwiera szczegółowe GUI gildii ze statystykami i przyciskiem zaproszenia do sojuszu.

8.3 System Animacji i Efektów

8.3.1 Animowane przejścia między menu — efekty otwierania/zamykania, dźwięki

System animacji opiera się na dźwiękach i efektach wizualnych konfigurowalnych w `gui/animations.yml`. Otwarcie głównego menu generuje `BLOCK_CHEST_OPEN` (głośność 0.5), otwarcie podmenu zastępuje je `BLOCK_NOTE_BLOCK_PLING` (pitch 1.5) — sygnał akustyczny wejścia w niższy poziom. Zamknięcie to `BLOCK_CHEST_CLOSE`, a kliknięcie zablokowanego slotu — `BLOCK_NOTE_BLOCK_BASS` (pitch 0.5) jako sygnał odmowy.

Efekt wizualny otwierania realizowany jest przez sekwencyjne wypełnianie slotów. Przy otwarciu GUI plugin uruchamia zadanie synchroniczne iterujące po slotach spiralnie od zewnątrz do środka, z opóźnieniem 1 ticka (50 ms) na slot. Pełna animacja 6-rzędowego GUI trwa około 30 ticków (1,5 s). Animację wyłącza parametr `animations.open.enabled: false` lub przyspiesza `animations.open.tick-delay: 0`. Zamknięcie GUI jest natychmiastowe — decyzja projektowa minimalizująca opóźnienie przy szybkiej nawigacji.

8.3.2 Dynamiczne odświeżanie — aktualizacja danych bez zamykania GUI, refresh co X ticków

Plugin implementuje dynamiczne odświeżanie (ang. *dynamic refresh*) oparte na cyklicznym zadaniu asynchronicznym. Parametr `refresh-interval` w każdym pliku GUI (domyślnie 20 ticków = 1 sekunda przy 20 TPS) kontroluje częstotliwość. Zadanie nie renderuje całego GUI od nowa — analizuje sloty z danymi dynamicznymi (status online, punkty rankingowe, liczniki zadań) i aktualizuje je przez `Inventory#setItem()`, pomijając sloty statyczne.

Dla GUI członków odświeżany jest status online/offline (modyfikacja enchantu), czas aktywności oraz synchronizacja rang. Dla GUI rankingu aktualizowane są pozycje, punkty i kolorystyka TOP 3. Dla GUI handlu synchronizowane są liczniki transakcji i status zadań. Parametr `refresh-interval` konfigurowalny jest globalnie i per-GUI. Dla serwerów 100+ graczy zaleca się interwał 60–100 ticków (3–5 s). Plugin implementuje "leniwe odświeżanie" (ang. *lazy refresh*) — jeśli hash zawartości nie zmienił się od ostatniego cyklu, aktualizacja jest pomijana, eliminując niepotrzebne wywołania API Bukkita.

```
# gui/animations.yml – konfiguracja efektów
dynamic-refresh:
  global-interval: 20
  per-gui-override:
    members: 40
    ranking: 100
    trade: 60
  lazy-refresh: true
sound-effects:
  open: BLOCK_CHEST_OPEN
  open-submenu: BLOCK_NOTE_BLOCK_PLING
  close: BLOCK_CHEST_CLOSE
  denied: BLOCK_NOTE_BLOCK_BASS
  click: UI_BUTTON_CLICK
visual-effects:
  open-animation: true
  open-tick-delay: 1
  spiral-direction: INWARD
```

9. Ranking Gildii

Ranking gildii stanowi centralny element motywacyjny ekosystemu Fraction Guild Clans, przekształcając aktywność bojową, ekonomiczną i rozwojową w mierzalną wartość porównawczą. System punktacji agreguje dane z wielu modułów pluginu — od zabójstw PvP, przez zniszczenia jaj obcych gildii, aż po poziom cuboidu i handel z wieśniakami — tworząc wielowymiarowy obraz potęgi każdej gildii. W przeciwieństwie do prostych rankingów opartych wyłącznie na fragach, rozwiązanie to nagradza kompleksowy rozwój i koordynację zespołową, co skuteczniej odwzorowuje dynamikę starć gildyjnych na długotrwałych serwerach survival-PvP.

9.1 System Punktacji

9.1.1 Podstawowe źródła punktów

Podstawowe źródła punktów obejmują cztery główne kategorie aktywności, z których każda reprezentuje inny aspekt rozgrywki gildyjnej. Zabójstwa graczy należących do innych gildii generują punkty rankingowe w czasie rzeczywistym, przy czym wartość każdego fraga jest ważona względem poziomu zabitego przeciwnika — eliminacja członka silnej gildii przynosi więcej punktów niż zabójstwo gracza z gildii początkującej. Mnożnik ten zapobiega farminiu słabych graczy i zachęca do atakowania wartościowych celów. Zniszczenia jaj gildii (Dragon Egg) stanowią drugie najważniejsze źródło punktów: każde pomyślne zniszczenie jaja wroga generuje znaczną pulę punktów dla całej gildii atakującej, co tworzy dodatkową motywację do organizowania rajdów. Wykonane zadania dzienne i tygodniowe pobierane od wieśniaków handlowych (Handlarz, Kowal, Alchemik) dodają punkty w kategorii ekonomicznej, promując regularną

aktywność poza PvP. Wreszcie, poziom cuboidu gildii wpływa na ranking poprzez stały mnożnik — każdy awans poziomu zwiększa bazową pulę punktów o wartość konfigurowalną w pliku `config.yml`, co nagradza gildie inwestujące w rozwój terytorialny.

9.1.2 Ujemne punkty

System rankingu obejmuje również mechanizmy karzące, które odejmują punkty w odpowiedzi na porażki i dezorganizację. Śmierci członków gildii w walce z innymi graczami powodują utratę punktów rankingowych, przy czym strata jest proporcjonalna do poziomu gildii przeciwnika — śmierć z rąk gracza z silnej gildii kosztuje więcej punktów niż porażka z nowicjuszem. Utrata jaja gildii (Egg Destroyed) skutkuje najpoważniejszą karą punktową: każde zniszczenie własnego jaja obniża ranking gildii o wartość znacząco przewyższającą zyski z typowego zabójstwa, co podkreśla strategiczną wagę obrony terytorialnej. Trzeci element penalizacyjny obejmuje opuszczenie gildii przez członków — każde dobrowolne odejście gracza, który przebywał w gildii co najmniej przez konfigurowalny okres (domyślnie 7 dni), powoduje niewielki, ale kumulatywny spadek rankingowy. Ten ostatni mechanizm ma na celu zniechęcenie do tworzenia gildii szturmowych, które szybko rekrutują i tracą graczy bez budowania trwałej struktury.

9.1.3 Wzór rankingowy — konfigurowalne współczynniki

Wzór rankingowy implementowany przez plugin opiera się na ważonej sumie zdarzeń z uwzględnieniem mnożników per-kategoria. Administrator serwera ma pełną kontrolę nad wartością każdego współczynnika poprzez sekcję `ranking.formula` w pliku `config.yml`. Poniższa tabela przedstawia domyślne wartości współczynników oraz ich znaczenie w kontekście równowagi rozgrywki.

Zdarzenie	Współczynnik (points)	Typ	Domyślna wartość	Opis mechaniki
Zabójstwo gracza (gildia wyższa o N poziomów)	<code>kill.higher_level</code>	Dodatni	+15 pkt	Mnożnik poziomy: <code>base * (1 + 0.2 * Δlevel)</code>
Zabójstwo gracza (gildia równa poziomem)	<code>kill.equal_level</code>	Dodatni	+10 pkt	Bazowa wartość bez modyfikatorów
Zabójstwo gracza (gildia niższa o N poziomów)	<code>kill.lower_level</code>	Dodatni	+5 pkt	Mnożnik degradacji: <code>base * max(0.3, 1 - 0.15 * Δlevel)</code>
Zniszczenie jaja wroga (1. etap)	<code>egg_destroy.stage1</code>	Dodatni	+25 pkt	Nagroda za uszkodzenie obrony gildii
Zniszczenie jaja wroga (2. etap)	<code>egg_destroy.stage2</code>	Dodatni	+50 pkt	Nagroda za częściowe przełamanie obrony

Zniszczenie jaja wroga (3. etap — finalne)		<code>egg_destroy.stage3</code>	Dodatni	+100 pkt	Nagroda za eliminację jaja gildii
Ukończenie dziennego zadania		<code>quest.daily</code>	Dodatni	+3 pkt	Za każde zadanie od wieśniaka, limit dzienny 15 pkt
Ukończenie tygodniowego zadania		<code>quest.weekly</code>	Dodatni	+15 pkt	Nagroda za zadania o podwyższonym trudzie
Awans poziomu cuboidu		<code>levelup.bonus</code>	Dodatni	+20 pkt *	Stały bonus przy każdym awansie
Śmierć członka (od silniejszej gildii)		<code>death.stronger_enemy</code>	Ujemny	-8 pkt	Kara za porażkę z wyżej notowanym przeciwnikiem
Śmierć członka (od równego/słabszego)		<code>death.default</code>	Ujemny	-5 pkt	Standardowa kara za śmierć w PvP
Utrata własnego jaja (każdy etap)		<code>egg_lost.stage_penalty</code>	Ujemny	-30 pkt	Kara za każdy etap zniszczenia własnego jaja
Odejście członka (po 7+ dniach)		<code>member_leave_penalty</code>	Ujemny	-2 pkt	Kara za rotację personelu, cooldown 24h na gracza

Konfiguracja w `config.yml` przyjmuje postać struktury YAML z zagnieżdżonymi sekcjami. Administrator może modyfikować wartości bezpośrednio w pliku, a następnie przeładować konfigurację komendą `/guild admin reload`. Zmiany współczynników mają zastosowanie od momentu przeładowania i nie wpływają retrospektywnie na historię punktową. Dla serwerów preferujących rozgrywkę PvP zaleca się zwiększenie wag zabójstw i zniszczeń jaj; serwery ekonomiczne mogą podnieść wartość zadań i handlu, podczas gdy serwery budowlane skorzystają z wyższych mnożników za poziom cuboidu. Wartości ujemne zdarzeń śmierci i utraty jaja powinny pozostawać na poziomie nieprzekraczającym 60–70% wartości dodatnich, aby zapobiec spiralnej utracie punktów i demotywacji graczy.

9.2 Prezentacja Rankingu

9.2.1 Komenda `/guild top` — wyświetlanie TOP 10/50/100 gildii

Komenda `/guild top` stanowi główne narzędzie dostępu do rankingu. Po wpisaniu bez argumentów wyświetla domyślną listę TOP 10 gildii w formacie tekstowym w czacie. Argument `[limit]` pozwala na rozszerzenie zakresu do 50 lub 100 pozycji, podczas gdy argument `[strona]` włącza paginację — na przykład `/guild top 50 2` wyświetla pozycje od 11 do 20 w zestawieniu pięćdziesięciu gildii. Paginacja wykorzystuje komponenty chatu z możliwością kliknięcia w numery stron, co eliminuje konieczność

ręcznego wpisywania kolejnych komend. Limit widocznych pozycji jest kontrolowany przez uprawnienie `guild.top.limit.<liczba>`: gracze z podstawowym uprawnieniem `guild.user.top` widzą TOP 10, podczas gdy VIP-y i administratorzy mogą otrzymać dostęp do pełnych zestawień.

9.2.2 Format wyświetlania pozycji

Każda pozycja w rankingu wyświetla sześć kluczowych atrybutów w ustandaryzowanym formacie tekstowym: pozycję numeryczną z medalami dla pierwszej trójki (złoto, srebro, brąz), pełną nazwę gildii wraz z kolorem tagiem w nawiasach kwadratowych, łączną liczbę punktów rankingowych z rozróżnieniem wartości dodatnich i ujemnych, aktualny poziom cuboidu, liczbę aktywnych członków oraz krótki identyfikator trendu (`↑`, `↓`, `→`) wskazujący zmianę pozycji w ciągu ostatnich 24 godzin. Format wyjściowy jest konfigurowalny w sekcji `ranking.display.format` poprzez szablony zawierające placeholderzy: `%position%`, `%guild_name%`, `%guild_tag%`, `%points%`, `%level%`, `%members%`, `%trend%`. Dodatkowo, kliknięcie na nazwę gildii w rankingu wykonuje komendę `/guild info <tag>`, wyświetlającą szczegółowe informacje o wybranej gildii.

9.2.3 Filtrowanie rankingu

Plugin udostępnia cztery zakresy czasowe rankingu oraz trzy kategorie funkcjonalne. Zakresy czasowe obejmują ranking globalny (agregujący całą historię od ostatniego resetu sezonowego), tygodniowy (dane z bieżącego tygodnia, reset w niedzielę o północy według czasu serwera) oraz miesięczny (dane od pierwszego dnia bieżącego miesiąca). Komendy `/guild top week` i `/guild top month` przełączają wyświetlanie na odpowiednie zakresy. Kategorie filtrowania pozwalają na sortowanie według specjalizacji: PvP (suma zabójstw minus śmierci, z wagą za zniszczenia jaj), Budowanie (punkty za poziom cuboidu i czas istnienia gildii) oraz Handel (punkty za zadania u wieśniaków i transakcje handlowe). Filtrowanie kategorii dostępne jest przez komendy `/guild top pvp`, `/guild top build` i `/guild top trade`, a także poprzez GUI rankingu opisane w rozdziale 8.

9.3 Sezonowość i Reset

9.3.1 System sezonów — cykliczny reset rankingu

System sezonów wprowadza cykliczną rekaliibrację rankingu, zapobiegającą trwałemu zdominowaniu przez wcześniej założone gildie i zapewniającą nowym grupom realne szanse na wejście do czołówki. Długość sezonu jest w pełni konfigurowalna w sekcji `ranking.season` pliku `config.yml` — administrator może ustawić okres od 1 do 12 miesięcy, przy czym wartość domyślna to 3 miesiące (90 dni). Data rozpoczęcia sezonu definiowana jest explicite parametrem `start_date` w formacie `YYYY-MM-DD`, co pozwala na precyzyjne zaplanowanie cyklu niezależnie od daty instalacji pluginu. Tydzień przed zakończeniem sezonu plugin aktywuje tryb "Sezon Finale" — wszystkie gildie otrzymują powiadomienia co 6 godzin, a punktacja za zabójstwa i zniszczenia jaj może zostać opcjonalnie podwojona (parametr `finale_multiplier: 2.0`)

dla zwiększenia intensywności rywalizacji. Po zakończeniu sezonu ranking główny resetuje się do zera, zachowując jedynie dane do Hall of Fame.

9.3.2 Nagrody sezonowe — rozdzielanie nagród TOP gildiom

Po zakończeniu każdego sezonu plugin automatycznie przyznaje nagrody gildiom znajdującym się na podium oraz w pierwszej dziesiątce. Mechanizm dystrybucji obsługuje zarówno nagrody materialne (itemy wypadające do skrzyni gildyjnej), jak i walutowe (przelew przez Vault) oraz nagrody w formie command-blocków (wykonywanie dowolnych komend jako konsola). Poniższa tabela przedstawia domyślną konfigurację nagród sezonowych.

Pozycja	Nagroda walutowa		Nagroda materialna	Nagroda specjalna	Mnożnik prestiżu
1. miejsce	\$50	000	1x Netherite Block, 64x Diamond Block	Tytuł "Mistrz Sezonu", unikalny prefix chatu, +1 slot rezerwowany do Hall of Fame	3.0x punkty w następnym sezonie (przez 7 dni)
2. miejsce	\$30	000	32x Diamond Block, 64x Emerald Block	Tytuł "Wicemistrz", dostęp do ekskluzywnego skinu w GUI	2.0x punkty w następnym sezonie (przez 5 dni)
3. miejsce	\$15	000	16x Diamond Block, 32x Emerald Block	Tytuł "Podium Sezonu"	1.5x punkty w następnym sezonie (przez 3 dni)
4–5. miejsce	\$7	500	8x Diamond Block, 16x Gold Block	—	1.2x punkty (przez 2 dni)
6–10. miejsce	\$3	000	4x Diamond Block, 8x Gold Block	—	1.0x (brak mnożnika)

Dystrybucja nagród wykonywana jest automatycznie przez scheduler pluginu w momencie zakończenia sezonu. Przyznane nagrody walutowe trafiają do gildyjnego banku (jeśli włączony moduł ekonomiczny) lub są dzielone równo między członków online w chwili resetu. Nagrody materialne umieszczane są w skrzyni gildyjnej (`/guild vault`) z powiadomieniem dla lidera i moderatorów. Sekcja `ranking.season.rewards` w `config.yml` pozwala na pełną modyfikację powyższych wartości, dodawanie nowych nagród dla dowolnych pozycji oraz definiowanie nagród w formacie command — na przykład wykonanie komendy `give %leader% minecraft:dragon_head 1` dla lidera zwycięskiej gildii. Mnożnik prestiżu działa jako tymczasowy booster punktowy aktywowany automatycznie w pierwszym dniu nowego sezonu, co nagradza gildie konsekwentnie utrzymujące wysoką formę.

9.3.3 Hall of Fame — lista zwycięzców poprzednich sezonów

Hall of Fame stanowi archiwum historyczne zapisujące zwycięzców wszystkich zakończonych sezonów. Po każdym resecie dane TOP 3 gildii (nazwa, tag, lider, liczba punktów, liczba członków) trafiają do bazy danych pod indeksem numeru sezonu. Dostęp do archiwum realizowany jest na dwa sposoby: wewnątrz gry poprzez GUI otwierane komendą `/guild halloffame`, które wyświetla paginowaną listę sezonów z głowicami liderów zwycięskich gildii oraz ich statystykami, oraz poprzez stronę HTML generowaną przez moduł mapy (opisany w rozdziale 10). Sekcja Hall of Fame na stronie HTML renderowana jest jako osobna zakładka z tabelarycznym podsumowaniem wszystkich sezonów. Parametr `halloffame.keep_seasons` w `config.yml` kontroluje maksymalną liczbę przechowywanych sezonów w historii — wartość domyślna to 0 (nieograniczona), ale dla ograniczenia rozmiaru bazy danych na dużych serwerach można ustawić limit np. 12 ostatnich sezonów. Zwycięskie gildie zachowują swój tytuł w postaci permanentnego wpisu w drzewku osiągnięć pluginu, widocznego w profilu gildii komendą `/guild info`.

10. Mapa Serwera HTML

Moduł mapy serwera HTML to narzędzie wizualizacyjne pluginu Fraction Guild Clans, umożliwiające przeglądanie topologii świata, lokalizacji cuboidów gildii i punktów strategicznych w formie interaktywnej mapy przeglądarkowej. W przeciwieństwie do Dynmap czy BlueMap, rozwiązanie wbudowane jest ściśle zintegrowane z systemem gildii — wyświetla cuboidy w kolorach przypisanych do gildii, markery jaj obronnych i opcjonalnie pozycje graczy. Mapa generowana jest jako statyczne pliki HTML, CSS i JavaScript, co eliminuje potrzebę dodatkowego serwera webowego.

10.1 Generowanie Mapy

Proces generowania mapy serwera może zostać zainicjowany na dwa sposoby: ręcznie przez administratora lub automatycznie zgodnie z zdefiniowanym harmonogramem. Obie metody produkują identyczny format wyjściowy i różnią się jedynie momentem uruchomienia procesu renderowania.

10.1.1 Komenda `/guild map generate` — ręczne generowanie mapy serwera

Administrator serwera posiada do dyspozycji komendę `/guild map generate`, która natychmiastowo uruchamia proces renderowania pełnej mapy. Komenda wymaga uprawnień `guild.admin.map.generate` i wykonuje się asynchronicznie, co oznacza że generowanie nie blokuje głównego wątku serwera (main thread) i nie powoduje spadku TPS (Ticks Per Second). Po wywołaniu komendy plugin wykonuje trzy operacje: zapisuje dane gildii do pliku JSON (współrzędne cuboidów, nazwy, tagi, kolory), renderuje mapę terenu na podstawie bloków powierzchniowych w chunkach (Chunks, czyli 16×16 blokowych segmentów świata), oraz generuje plik HTML z osadzonym kodem JavaScript. Po zakończeniu administrator otrzymuje komunikat potwierdzający ze ścieżką do pliku. Opcjonalnie można

użyć `/guild map generate --force`, która wymusza pełne przetworzenie wszystkich chunków bez względu na to, czy uległy zmianie.

10.1.2 Automatyczne generowanie — harmonogram w config.yml (co X minut/godzin)

W sekcji `map` pliku `config.yml` administrator konfiguruje automatyczne generowanie mapy za pomocą parametru `auto-generate-interval` wyrażonego w minutach; wartość `0` lub ujemna wyłącza tę funkcję. Dla serwerów o dużej dynamie zmian zaleca się interwał 15–30 minut, natomiast serwery o mniejszej aktywności mogą ustawić 60 minut lub więcej, co redukuje obciążenie dysku I/O (Input/Output) oraz CPU. Generowanie wykonywane jest w wątku asynchronicznym przez scheduler Bukkit API, nie wpływając na płynność rozgrywki. Parametr `auto-generate-only-when-online` ogranicza generowanie do sytuacji, gdy na serwerze znajduje się co najmniej jeden gracz — opcja przydatna na serwerach niepracujących ciągle.

10.1.3 Format wyjściowy — statyczny HTML + CSS + JavaScript, osadzony w folderze web/map/

Wynikiem generowania jest zestaw plików umieszczany w folderze `plugins/FractionGuildClans/web/map/`. Struktura wyjściowa obejmuje: `index.html` — główny punkt wejścia z szkieletem DOM (Document Object Model) mapy, `map.css` — style wizualne warstw i responsywność mobilną, `map.js` — logikę interaktywną (zoom, przesuwanie, komunikacja z aktualizacją na żywo), `guilds_data.json` — serializowane dane gildii (identyfikator, nazwa, tag, kolor HEX, współrzędne, promień) oraz folder `tiles/` z prerenderowanymi kafelkami mapy (map tiles) w formacie PNG podzielonymi na podkatalogi według poziomów zoomu, analogicznie do Google Maps. Taka struktura pozwala przeglądarce ładować jedynie widoczne fragmenty, co redukuje zużycie pamięci i czas ładowania. Całość jest statyczna — nie wymaga serwera aplikacyjnego (PHP, Node.js), wystarczy dowolny serwer HTTP (Nginx, Apache) lub otwarcie `index.html` w przeglądarce.

10.2 Funkcjonalność Mapy

Interfejs mapy zapewnia intuicyjną nawigację i szybki dostęp do informacji strategicznych, z pełną kontrolą nad widocznością warstw i możliwością interakcji z elementami.

10.2.1 Interaktywna mapa 2D z zoomem i przesuwaniem — wyrenderowana z blocków na podstawie chunków

Warstwa terenu renderowana jest jako mapa 2D z rzutu z góry (top-down view), gdzie bloki Minecrafta reprezentowane są jako piksele w zależności od rozdzielczości. Silnik renderujący analizuje bloki powierzchniowe w każdym chunku, przypisując kolory na podstawie tekstur — trawa zielenie, piasek żółcie, woda błękity, kamień szarości. System obsługuje pięć poziomów zoomu, od widoku ogólnego (zoom 0) do zbliżenia na pojedynczy cuboid (zoom 4). Nawigacja odbywa się poprzez przeciąganie (drag-and-drop), a zoom za pomocą kółka myszy (scroll wheel) lub gestu szczypania na ekranach dotykowych. W lewym

górnym rogu interfejsu znajduje się miniaturka (minimap) z aktualnym obszarem widoku w kontekście całej mapy.

10.2.2 Warstwy: teren, cuboidy gildii (kolorowe poligony), pozycje graczy (opcjonalnie), markery jaj

Mapa składa się z czterech przełączalnych warstw, kontrolowanych z panelu w prawym górnym rogu. Warstwa terenu jest zawsze aktywna. Warstwa cuboidów nakłada półprzezroczyste poligony w kolorach gildii, gdzie wypełnienie zależy od poziomu cuboidu. Warstwa pozycji graczy, domyślnie wyłączona, wyświetla ikony z lokalizacją graczy online i ich nickami; wymaga włączenia `show-player-positions` oraz wybrania trybu aktualizacji (patrz sekcja 10.3.3). Warstwa markerów jaj umieszcza ikony w centrach cuboidów — zielone dla aktywnego jaja, pomarańczowe dla uszkodzonego, czerwone z przekreśleniem dla zniszczonego.

10.2.3 Panel boczny z listą gildii — kliknięcie centruje mapę na cuboidzie, pokazuje informacje

Po lewej stronie interfejsu znajduje się panel boczny z listą wszystkich gildii. Każdy wpis zawiera kolorowy znacznik, tag, nazwę oraz liczbę członków online. Kliknięcie wpisu powoduje płynne przesunięcie (pan) i zbliżenie (zoom) mapy na środek cuboidu, jednocześnie otwierając panel informacyjny z danymi: nazwa, tag, poziom cuboidu, liczba członków, punkty rankingowe, status jaja obronnego, data założenia. Panel wyposażony jest w pasek wyszukiwania (search box) z filtrowaniem w czasie rzeczywistym po nazwie lub tagu. Listę można sortować alfabetycznie, według poziomu cuboidu lub punktów rankingowych. Przycisk eksportu pozwala pobrać dane w formacie CSV (Comma-Separated Values) do analizy zewnętrznej.

10.3 Konfiguracja i Dostęp

Kompletna konfiguracja modułu mapy wymaga ustawienia parametrów jakościowych w `config.yml`, zdefiniowania modelu dostępu i wyboru mechanizmu aktualizacji na żywo. Poniższa tabela zestawia wszystkie parametry konfiguracyjne.

10.3.1 Ustawienia w `config.yml`: rozdzielczość, jakość, zakres renderowania, kolory gildii

Parametr	Domyślna wartość	Zakres / Typ	Opis funkcjonalny
<code>map.enabled</code>	<code>true</code>	Boolean (true/false)	Główny przełącznik aktywujący lub dezaktywujący moduł mapy HTML. Po ustawieniu na <code>false</code> plugin nie generuje plików mapy i ignoruje komendę <code>/guild map generate</code> .
<code>map.auto-generate-interval</code>	<code>30</code>	Liczba całkowita (minuty)	Interwał automatycznego generowania mapy. Wartość <code>0</code> wyłącza generowanie automatyczne; wymaga ręcznego wywoływania komendy przez administratora.

<code>map.auto-generate-only-when-online</code>	<code>true</code>	Boolean	Ogranicza automatyczne generowanie do przypadków, gdy na serwerze znajduje się co najmniej jeden gracz. Redukuje niepotrzebne operacje I/O na serwerach niepracujących ciągle.
<code>map.render.resolution</code>	<code>high</code>	<code>low</code> , <code>medium</code> , <code>high</code> , <code>ultra</code>	Określa rozdzielczość renderowania: <code>low</code> (1 blok = 1 piksel), <code>medium</code> (1:2), <code>high</code> (1:4), <code>ultra</code> (1:8). Wyższa rozdzielczość zapewnia lepszą czytelność przy koszcie większego rozmiaru plików i dłuższego czasu generowania.
<code>map.render.radius</code>	<code>5000</code>	Liczba całkowita (bloki)	Promień renderowania mapy wyrażony w blokach od punktu spawn świata. Przy wartości <code>5000</code> mapa obejmuje obszar 10000×10000 bloków. Zwiększenie promienia wymusza dłuższy czas generowania i więcej miejsca na dysku.
<code>map.render.worlds</code>	<code>["world"]</code>	Lista ciągów znaków	Lista nazw światów, dla których generowana jest mapa. Obsługuje wiele światów jednocześnie, generując osobny zestaw plików dla każdego z nich.
<code>map.style.guild-colors</code>	<code>auto</code>	<code>auto</code> , lista HEX	Tryb przypisywania kolorów cuboidom: <code>auto</code> losuje kolor z predefiniowanej palety, natomiast lista HEX pozwala na ręczne zdefiniowanie puli kolorów używanych kolejno przy zakładaniu gildii.
<code>map.style.polygons-opacity</code>	<code>0.35</code>	Liczba zmiennoprzecinkowa (0.0–1.0)	Poziom przezroczystości wypełnienia poligonów cuboidów. Wartość <code>0.35</code> zapewnia widoczność terenu pod cuboidem przy jednoczesnym wyraźnym zaznaczeniu granic.
<code>map.style.border-width</code>	<code>2</code>	Liczba całkowita (piksele)	Grubość obramowania poligonów cuboidów w pikselach na mapie. Wartość <code>2</code> zapewnia czytelność granic przy wszystkich poziomach zoomu.
<code>map.access.type</code>	<code>public</code>	<code>public</code> , <code>token</code> , <code>account</code>	Model dostępu do mapy: <code>public</code> — mapa dostępna bez uwierzytelniania; <code>token</code> — wymaga podania tokenu w parametrze URL; <code>account</code> — integracja z systemem logowania serwera (np. przez API auth pluginu).

<code>map.access.token</code>	"	Ciąg znaków (UUID lub hasło)	Token dostępu aktywowany przy <code>access.type: token</code> . Generowany automatycznie przy pierwszym włączeniu lub definiowany ręcznie przez administratora.
<code>map.live-update.mode</code>	po lli ng	<code>polling</code> , <code>websocket</code> , <code>none</code>	Mechanizm aktualizacji danych na żywo: <code>polling</code> — okresowe odpytywanie serwera, <code>websocket</code> — połączenie dwukierunkowe (wymaga dodatkowego serwera proxy), <code>none</code> — brak aktualizacji, dane ładowane tylko przy generowaniu.
<code>map.live-update.interval</code>	1 0	Liczba całkowita (sekundy)	Interwał odświeżania danych w trybie <code>polling</code> . Co zadaną liczbę sekund przeglądarka wysyła zapytanie do endpointu API, pobierając aktualne pozycje graczy i zmiany cuboidów.
<code>map.live-update.show-player-positions</code>	fa ls e	Boolean	Przełącznik widoczności warstwy pozycji graczy. Ze względu na prywatność domyślnie wyłączony; włączenie wymaga akceptacji przez administratora pod kątem RODO/prywatności graczy.

Powyższa tabela stanowi kompletny zbiór parametrów konfiguracyjnych modułu mapy. Parametry `map.render.resolution` oraz `map.render.radius` bezpośrednio wpływają na czas generowania i obciążenie serwera — przy rozdzielczości `ultra` i promieniu `10000` bloków proces może potrwać kilka minut, podczas gdy konfiguracja `low` z promieniem `2000` generuje się w kilkanaście sekund. Parametr `map.style.guild-colors` w trybie `auto` losuje kolory z palety, lecz administratorzy mogą zdefiniować własną pulę HEX dla spójności z brandingiem serwera. Sekcja dostępu (`map.access`) jest krytyczna dla serwerów niechących ujawniać lokalizacji gildii — tryb `token` stanowi kompromis między bezpieczeństwem a prostotą, natomiast `account` służy zaawansowanym integracjom z panelem gracza. Aktualizacja na żywo domyślnie używa `polling` ze względu na brak dodatkowych zależności, choć `websocket` oferuje niższe opóźnienia przy wyższej złożoności.

10.3.2 System dostępu — publiczna lub wymagająca tokenu/konta, integracja z logowaniem serwera

Trzy tryby dostępu odpowiadają na różne potrzeby bezpieczeństwa. Tryb `public` jest domyślny — mapa dostępna bez uwierzytelniania pod adresem URL folderu `web/map/`. Tryb `token` wymaga parametru `?token=TOKEN` w adresie URL; bez prawidłowego tokenu przeglądarka zwraca błąd 403 (Forbidden). Token można rozsyłać wśród zaufanych graczy lub umieszczać na stronie serwera za logowaniem. Tryb `account` przeznaczony jest dla serwerów z własnym systemem kont (np. plugin auth, forum, launcher); wymaga skonfigurowania endpointu weryfikacji sesji w parametrze `map.access.auth-endpoint`. Plugin sprawdza ważność sesji przez ciasteczka sesyjne (session cookies), a endpoint zwraca JSON z polem `authorized: true/false`.

10.3.3 Aktualizacja na żywo — WebSocket lub polling odświeżający pozycje graczy i zmiany cuboidów

Dane na mapie odświeżane są na żywo bez ponownego generowania. W trybie `polling` (domyślnym) skrypt JavaScript co zadany interwał (domyślnie 10 sekund) wysyła zapytanie HTTP GET do endpointu `web/api/live-data.json`, z którego pobiera aktualne pozycje graczy i zmiany cuboidów. Tryb `websocket` wymaga serwera proxy (np. Nginx z modulem `ngx_http_proxy_module`) przekazującego ruch na port nasłuchu pluginu. Po zestawieniu połączenia serwer wysyła komunikaty JSON w momencie zajścia zdarzeń (zmiana pozycji, rozbudowa cuboidu, zmiana statusu jaja), eliminując opóźnienia pollingu. Tryb `none` wyłącza aktualizacje na żywo — mapa odzwierciedla stan z ostatniego generowania, co sprawdza się na serwerach z rzadkimi zmianami lub gdzie priorytetem jest minimalne zużycie zasobów.

11. Custom Tabulator

Custom Tabulator w pluginie Fraction Guild Clans stanowi w pełni konfigurowalny system wyświetlania listy graczy (TAB list), który zastępuje domyślny interfejs Minecraft. Mechanizm ten integruje się bezpośrednio z klientem gry poprzez pakiet `PlayerListHeaderFooter` oraz modyfikację zawartości scoreboardu po stronie serwera, co pozwala na prezentację rozbudowanych informacji w czterech segmentach: nagłówku, lewej kolumnie, prawej kolumnie i stopce. Tabulator stanowi jedno z kluczowych narzędzi informacyjnych — każdy gracz widzi na bieżąco dane o swojej gildii, rankingu, stanie serwera i aktywnych wydarzeniach, bez konieczności wykonywania dodatkowych komend. System wymaga aktywnego PlaceholderAPI do rozpoznawania zmiennych dynamicznych oraz opcjonalnie dodatkowych rozszerzeń (PAPI expansions) dla pełnej funkcjonalności.

11.1 Struktura TAB-a

Tabulator dzieli ekran listy graczy na cztery logiczne strefy, z których każda pełni inną funkcję informacyjną. Podział ten odzwierciedla priorytety komunikacji z graczem — od ogólnych danych o serwerze, przez rankingi indywidualne i gildyjne, aż po techniczne wskaźniki wydajności.

11.1.1 Nagłówek — nazwa serwera, aktualny sezon, liczba graczy online

Nagłówek (header) wyświetla się w centralnej, górnej części ekranu TAB i zawiera informacje o najwyższym priorytecie dla tożsamości serwera. Administrator konfiguruje wieloliniowy tekst, który może obejmować nazwę serwera z formatowaniem kolorystycznym, numer bieżącego sezonu rankingu (np. "Sezon 3 — Wiosna 2025") oraz liczbę graczy online względem maksymalnego slotu (np. "Gracze: 47/100"). Wartości te odświeżają się automatycznie zgodnie z zadeklarowanym interwałem. Nagłówek stanowi pierwszy element, który gracz widzi po otwarciu listy TAB, dlatego zaleca się umieszczenie tu marki serwera lub hasła promującego aktualny event.

11.1.2 Lewa kolumna — lista 20 najlepszych graczy (K/D, punkty), informacje o świecie

Lewa kolumna (sloty 1–20 scoreboardu) standardowo prezentuje ranking indywidualny — 20 graczy z najwyższym współczynnikiem zabójstw do śmierci (K/D ratio) lub łączną liczbą punktów rankingu, w zależności od konfiguracji. Każdy wiersz zawiera pozycję, nick gracza (przycięty do 16 znaków z uwagi na ograniczenia protokołu Minecraft), skrócony wskaźnik statystyczny oraz opcjonalnie tag gildii. Poniżej listy rankingowej administrator może umieścić sekcję informacyjną o świecie gracza — aktualne współrzędne X/Y/Z, nazwę biomu lub temperaturę (w przypadku serwerów z pluginami survivalowymi). Kolejność graczy w kolumnie jest sortowana automatycznie na podstawie danych z bazy pluginu; w przypadku mniejszej liczby graczy online niż 20, puste sloty wyświetlają predefiniowany tekst lub pozostają niewidoczne.

11.1.3 Prawa kolumna — TOP 10 gildii, status sojuszy, aktywne eventy

Prawa kolumna (sloty 1–20 scoreboardu po stronie prawej) koncentruje się na wymiarze gildijnym gry. Górna jej część prezentuje TOP 10 gildii według punktacji — każdy wpis zawiera pozycję, kolorowy tag gildii, nazwę (skróconą) oraz liczbę punktów. Poniżej rankingu gildyjnego wyświetla się sekcja sojuszy aktywnych dla gildii oglądającego gracza, co pozwala szybko zweryfikować, które gildie pozostają sojusznikami, a które znajdują się w stanie wojny. Dolna część kolumny rezerwuje miejsce na komunikaty o aktywnych eventach serwerowych — na przykład "2x DROP do 20:00" lub "EVENT PvP — Arena Otwarta". Sekcja eventowa może być warunkowana: wyświetla się tylko wtedy, gdy administrator aktywuje dany event w panelu zarządzania.

11.1.4 Stopka — aktualny czas serwera, TPS, zużycie RAM-u, adres strony serwera

Stopka (footer) umieszczona jest u dołu ekranu TAB i pełni funkcję techniczną oraz promocyjną. Administrator może skonfigurować wyświetlanie aktualnego czasu serwera (zgodnego z timezone ustawionym w systemie operacyjnym lub zdefiniowanym w config.yml), aktualnego TPS (Ticks Per Second — wskaźnika płynności pracy serwera, pobieranego z silnika Paper/Spigot/Purpur), procentowego zużycia pamięci RAM (obliczanego jako `used/total/max`) oraz adresu strony internetowej serwera lub zaproszenia na serwer Discord. Stopka jest szczególnie przydatna dla administratorów, którzy w czasie rzeczywistym mogą monitorować parametry wydajnościowe bez konieczności korzystania z konsoli ani pluginów diagnostycznych. Dla graczy natomiast stanowi stały przypomnienie o kanałach komunikacji poza grą.

Element TAB-a	Zakres slotów	Główne zawartości	Źródło danych	Priorytet odświeżania
Nagłówek (header)	1–3 linie centralne	Nazwa serwera, numer sezonu, gracze online/maks	<code>config.yml</code> (stałe) + API serwera (dynamiczne)	Co każdy cykl (tick)

Lewa kolumna	Sloty 1–20 (lewa strona scoreboardu)	TOP 20 graczy (K/D, punkty), współrzędne, biom	Baza pluginu, SQLite/MySQL API, Minecraft świata	Co każdy cykl (tick)
Prawa kolumna	Sloty 1–20 (prawa strona scoreboardu)	TOP 10 gildii (tag, nazwa, punkty), lista sojuszy, komunikaty eventów	Baza gildii + system eventów pluginu	Co każdy cykl (tick)
Stopka (footer)	1–2 linie centralne (dół)	Czas serwera, TPS, RAM, adres WWW/Discord	System operacyjny + API silnika serwera	Co każdy cykl (tick)

Tabela przedstawia kompletną strukturę tabulatora wraz z przydziałem slotów scoreboardu, rodzajem wyświetlanych treści oraz źródłem danych dla każdego segmentu. Nagłówek i stopka operują na ograniczonej liczbie linii tekstu (maksymalnie 3 w nagłówku i 2 w stopce ze względu na ograniczenia pakietów `PlayerListHeaderFooter` w protokole Minecraft), podczas gdy obie kolumny mogą wykorzystać pełne 20 slotów scoreboardu po każdej stronie. Priorytet odświeżania jest jednolity dla wszystkich elementów — w ramach jednego cyklu plugin odświeża nagłówek, obie kolumny i stopkę synchronicznie, co zapobiega desynchronizacji danych między poszczególnymi sekcjami. Administrator powinien zwrócić szczególną uwagę na priorytet odświeżania przy dużej liczbie graczy — w przypadku serwerów powyżej 100 slotów zaleca się wydłużenie interwału lub włączenie mechanizmu buforowania wyników zapytań do bazy danych, aby nie obciążać głównego wątku serwera (main thread) zapytaniami SQL w każdym cyklu TAB-a.

11.2 Personalizacja

System tabulatora oferuje trzy poziomy personalizacji: formatowanie tekstu (prezentacja), dynamiczne sekcje (zawartość warunkowa) oraz częstotliwość odświeżania (wydajność). Każdy z tych poziomów kontrolowany jest niezależnie, co pozwala administratorowi dostosować wygląd i funkcjonalność TAB-a do specyfiki serwera bez ingerencji w kod źródłowy pluginu.

11.2.1 Formatowanie tekstu — wsparcie dla PlaceholderAPI, kody kolorów, animowane kolory (gradienty)

Plugin obsługuje pełen zakres kolorów i formatowania dostępnego w Minecraft — standardowe kody z prefiksem `&` (np. `&c` dla czerwonego, `&l` dla pogrubienia), a także rozszerzone kody w formacie hexadecymalnym zapisane jako `&#RRGGBB` (wymagane wsparcie silnika dla wersji 1.16+). Dodatkowo Fraction Guild Clans integruje się z PlaceholderAPI (PAPI), co pozwala na osadzanie zewnętrznych zmiennych w dowolnym elemencie TAB-a — od statystyk gracza, przez dane ekonomii (Vault), aż po informacje z innych pluginów. Plugin wspiera również animowane gradienty kolorystyczne, realizowane poprzez szybką zmianę kodów kolorów między kolejnymi cyklami odświeżania. Gradient definiuje się jako

sekwencja kolorów źródłowych i docelowych (np. `�-＀`), a plugin automatycznie interpoluje wartości pośrednie, tworząc efekt płynnego przejścia w czasie rzeczywistym.

11.2.2 Dynamiczne sekcje — zmiana zawartości w zależności od świata, gildii gracza, uprawnień

System sekcji warunkowych pozwala na wyświetlanie różnej zawartości TAB-a w zależności od kontekstu gracza. Mechanizm opiera się na blokach warunkowych zagnieżdżonych w pliku `tab.yml`, które weryfikują trzy typy predykatów: przynależność do świata (np. inna zawartość dla świata `world`, `world_nether` i `world_the_end`), przynależność do gildii (gracz z gildią widzi więcej informacji gildyjnych niż gracz bez gildii) oraz posiadane uprawnienia (np. moderatorzy i administratorzy widzą dodatkowe dane techniczne, takie jak dokładne zużycie RAM-u lub liczbę załadowanych chunków). Sekcje dynamiczne przetwarzane są po stronie serwera — klient gracza otrzymuje gotową, spersonalizowaną zawartość, co eliminuje możliwość manipulacji danymi po stronie użytkownika. Administrator może zagnieżdżać warunki do dwóch poziomów głębokości, co wystarcza do pokrycia większości scenariuszy personalizacji.

11.2.3 Aktualizacja co X ticków — konfigurowalny interwał odświeżania, optymalizacja wydajności

Częstotliwość odświeżania tabulatora definiuje parametr `update-interval` w pliku `tab.yml`, wyrażony w tickach serwera (1 tick = 50 ms przy idealnym TPS = 20). Wartość domyślna wynosi 20 ticków (1 sekunda), co zapewnia płynność aktualizacji bez nadmiernego obciążenia. Administrator może zwiększyć interwał do 40, 60 lub 100 ticków na serwerach o dużej liczbie graczy (powyżej 100 slotów) lub przy użyciu bazy SQLite, która przy intensywnych zapytaniach może generować opóźnienia na głównym wątku. Zbyt krótki interwał (poniżej 10 ticków) nie jest zalecany, ponieważ aktualizacja TAB-a wymaga wysłania pakietów do wszystkich połączonych klientów, co przy 100+ graczy generuje znaczący ruch sieciowy. Parametr `update-interval` można modyfikować bez restartu serwera — komenda `/guild admin reload` przeładuje plik `tab.yml` i stosuje nową wartość interwału natychmiast.

11.3 Przykładowa Konfiguracja

Plik `tab.yml` stanowi centralny punkt konfiguracji tabulatora. Znajduje się w katalogu `plugins/FractionGuildClans/` i jest tworzony automatycznie przy pierwszym uruchomieniu pluginu z domyślnymi wartościami. Poniższy przykład prezentuje pełną konfigurację z komentarzami objaśniającymi funkcję każdego parametru.

11.3.1 Przykład pełnego pliku tab.yml z komentarzami — nagłówek, kolumny, stopka, sekcje warunkowe

```
# =====
# tab.yml – Konfiguracja Custom Tabulatora
# Fraction Guild Clans
# =====

# Interwał odświeżania w tickach (20 ticków = 1 sekunda)
# Zalecane: 20 dla serwerów <50 graczy, 40 dla 50-100, 60 dla 100+
update-interval: 20

# =====
# NAGŁÓWEK (Header) – wyświetlany na górze ekranu TAB
# =====
header:
  - '&8&m+-----+'
  - '&6&lMój Serwer &7| &eSezon &f%guild_season%'
  - '&7Gracze: &a%server_online%&7/&c%server_max_players%'
  - '&8&m+-----+'

# =====
# LEWA KOLUMNA – ranking graczy, informacje o świecie
# =====
left-column:
  title: '&6&lTOP 20 Graczy'
  # Sekcja warunkowa: wygląd zależy od posiadanej gildii
  has-guild:
    - '&8#1 &e%top_player_1% &7K/D: &c%top_kd_1%'
    - '&8#2 &e%top_player_2% &7K/D: &c%top_kd_2%'
    # ... (sloty 1-20 generowane automatycznie)
    - '&7'
    - '&6Twoja pozycja: &e%player_rank%'
    - '&6Gildia: &e%guild_tag% &7| &e%guild_points% pkt'
  no-guild:
    - '&8#1 &e%top_player_1% &7K/D: &c%top_kd_1%'
    - '&8#2 &e%top_player_2% &7K/D: &c%top_kd_2%'
    # ... (sloty 1-20)
    - '&7'
    - '&cNie masz gildii! &7/guild create'
  # Informacje o świecie (dolna część kolumny)
  world-info:
    - '&7'
    - '&6Świat: &e%player_world%'
    - '&6Pozycja: &e%X%&7, &e%Y%&7, &e%Z%'
    - '&6Biom: &e%player_biome%'

# =====
# PRAWA KOLUMNA – ranking gildii, sojusze, eventy
# =====
right-column:
  title: '&6&lTOP 10 Gildii'
  # Ranking gildii (sloty 1-10)
  guild-top:
```

11.3.2 Lista zalecanych placeholderów: %guild_name%, %guild_tag%, %guild_rank%, %guild_points%, %guild_level%

Poniższa lista zawiera zestaw placeholderów zalecanych do użycia w konfiguracji `tab.yml`. Placeholdery dzielą się na trzy kategorie: natywne (dostarczane bezpośrednio przez Fraction Guild Clans), zewnętrzne (wymagające PlaceholderAPI z aktywnymi rozszerzeniami) oraz warunkowe (interpretowane przez parser sekcji warunkowych pluginu).

Placeholdery natywne pluginu Fraction Guild Clans — dostępne bez dodatkowych rozszerzeń PAPI:

`%guild_name%` zwraca pełną nazwę gildii gracza lub tekst "Brak", jeśli gracz nie należy do żadnej gildii; `%guild_tag%` zwraca trzy- lub czteroliterowy tag gildii używany w chat i na liście graczy; `%guild_rank%` zwraca nazwę rangi gracza wewnątrz gildii (Lider, Moderator, Członek, Rekrut); `%guild_points%` zwraca aktualną liczbę punktów rankingu gildii w formacie liczbowym; `%guild_level%` zwraca poziom cuboidu gildii (zakres 1–10); `%guild_season%` zwraca numer aktywnego sezonu rankingu; `%guild_alliances%` zwraca listę tagów sojuszniczych gildii oddzielonych przecinkami. **Placeholdery wymagające PlaceholderAPI** — umożliwiają wyświetlanie danych serwerowych i gracza: `%server_online%` i `%server_max_players%` (rozszerzenie `server`), `%server_tps%` (rozszerzenie `server`), `%server_ram_used%` oraz `%server_ram_max%` (rozszerzenie `server`), `%player_world%`, `%player_x%`, `%player_y%`, `%player_z%` (rozszerzenie `player`), `%player_biome%` (rozszerzenie `player`). **Placeholdery rankingowe TOP** — generowane dynamicznie przez plugin: `%top_player_N%`, `%top_kd_N%`, `%top_points_N%` (dla lewej kolumny, gdzie N to pozycja 1–20), `%guild_top_tag_N%`, `%guild_top_name_N%`, `%guild_top_points_N%` (dla prawej kolumny, gdzie N to pozycja 1–10). Przed wdrożeniem konfiguracji administrator powinien zweryfikować dostępność wszystkich zewnętrznych rozszerzeń PAPI komendą `/papi list` oraz — w przypadku braku — doinstalować je komendą `/papi eccloud download <nazwa>`.

13. System Modułów i Konfiguracja

Wersja 2.0 pluginu Fraction Guild Clans wprowadza fundamentalną zmianę w sposobie organizacji kodu i konfiguracji — przejście z monolitycznej architektury na w pełni modułową strukturę. W podejściu monolitycznym (typowym dla starszych pluginów gildyjnych takich jak Factions czy wczesne wydania FunnyGuilds) cała logika aplikacji znajduje się w jednym, spójnym bloku kodu, gdzie każda funkcja jest ściśle powiązana z pozostałymi. Taka struktura utrudnia modyfikacje, wprowadza ryzyko regresji przy zmianach i zmusza administratora do akceptacji wszystkich funkcjonalności jednocześnie. Architektura modułowa rozwiązuje te problemy poprzez dekompozycję pluginu na czternaście autonomicznych jednostek funkcyjnych, które komunikują się ze sobą za pośrednictwem zdefiniowanego interfejsu API, zachowując jednak pełną niezależność w zakresie implementacji wewnętrznej.

13.1 Architektura Modułowa

13.1.1 Filozofia modułów — każda funkcja to osobny plik w `modules/`

Podstawową zasadą architektury v2.0 jest jednoznaczne przyporządkowanie: jedna funkcjonalność = jeden plik YAML w katalogu `modules/`. Każdy moduł zawiera kompletną konfigurację swojej domeny — od parametrów behawioralnych, przez definicje uprawnień, aż po wiadomości i powiadomienia. Taka granularność konfiguracji oznacza, że administrator serwera może dostosować każdy aspekt pluginu bez konieczności przeszukiwania tysięcy linii jednego, centralnego pliku konfiguracyjnego.

Moduł w rozumieniu systemu Fraction Guild Clans to nie tylko plik YAML — to pełna jednostka architektoniczna składająca się z trzech warstw. Warstwa konfiguracyjna (plik `.yaml` w katalogu `modules/`) definiuje parametry behawioralne. Warstwa implementacyjna (klasa Java wewnątrz pliku JAR pluginu) zawiera logikę wykonawczą modułu. Warstwa interfejsowa (integracja z systemem komend i GUI) udostępnia moduł graczom i administratorom. Przy starcie serwera system ładowania modułów odczytuje definicję z `config.yaml`, weryfikuje zależności między modułami, sortuje je według priorytetu i inicjalizuje w odpowiedniej kolejności. Jeśli moduł zostanie wyłączony w konfiguracji, jego warstwa implementacyjna nie jest ładowana do pamięci, co redukuje zużycie zasobów serwera (RAM i czas procesora).

Moduły komunikują się między sobą wyłącznie przez oficjalne API pluginu. Oznacza to, że moduł `ranking` nie odwołuje się bezpośrednio do wewnętrznych struktur modułu `guild` — zamiast tego używa metod interfejsu `GuildManager`, który jest dostarczany przez rdzeń pluginu. Taka separacja pozwala na wymianę implementacji modułu bez wpływu na pozostałe komponenty systemu.

13.1.2 Tabela czternastu modułów

Poniższa tabela prezentuje kompletny zestaw modułów dostępnych w wersji 2.0. Każdy moduł posiada unikalny identyfikator (używany w komendach i plikach konfiguracyjnych), nazwę pliku konfiguracyjnego w katalogu `modules/`, zwięzły opis zakresu funkcjonalnego oraz listę modułów, od których dany komponent zależy. Zależności te są wymuszane przez system ładowania — jeśli moduł `egg` wymaga modułu `cuboid`, a `cuboid` zostanie wyłączony, system nie załaduje modułu `egg` i zgłosi ostrzeżenie w konsoli serwera.

ID modułu	Plik konfiguracyjny	Opis funkcjonalny	Zależności
<code>guild</code>	<code>guild.yaml</code>	Rdzeń systemu gildii — zakładanie, zarządzanie członkami, rangi, relacje, zaproszenia, uprawnienia lidera i zastępcy	brak (moduł bazowy)

egg	egg.yml	System jaja gildii — 3 etapy życia, mechanika naprawy, tarcze ochronne, hologramy statusu, obsługa zniszczenia i upadku gildii	cuboid
cuboid	cuboid.yml	Własny system ochrony terenów — 10 levele powiększania, flagi ochronne (build, destroy, pvp, use, bucket, entry), strefy wewnętrzne (spawn, budowla, PvP), lista zaufanych graczy	brak
ranking	ranking.yml	System punktowy — wzór obliczeniowy (ELO lub liniowy), sezony rankingowe, nagrody sezonowe, top gildii i graczy, asysty przy zabójstwach	guild
villager	villagers.yml	Wieśniacy gildii — definicje typów (handlarz, zleceniodawca, strażnik), konfiguracja ofert handlowych, system zadań i relacji z graczem	guild, cuboid
gui	gui.yml	Wszystkie menu graficzne — główne menu gildii, panel administracyjny, ekrany konfiguracji, animacje przejść, dźwięki interakcji	guild
tab	tab.yml	Custom TAB — nagłówek listy graczy, kolumny z danymi (ranking, tag gildii, online), stopka, odświeżanie w czasie rzeczywistym	brak
map	map.yml	Mapa terenów — wyświetlanie cuboidów w czacie (tryb ASCII), mapa HTML do przeglądarki, znaczniki gildii i ich kolory	cuboid
economy	economy.yml	Integracja z EcoCore — ceny operacji gildyjnych (założenie, powiększenie, naprawa jaja), bank gildii, odsetki, transakcje międzygildyjne	guild
join_items	join_items.yml	System przedmiotów dołączeniowych — zestawy startowe dla nowych członków gildii, wymagania ekonomiczne przy dołączaniu, pay-to-join	guild, economy
lang	lang.yml	Zarządzanie plikami językowymi — wybór aktywnego języka, hot-reload tłumaczeń, fallback do języka domyślnego, wsparcie MiniMessage	brak
database	database.yml	Warstwa persystencji danych — wybór silnika (SQLite lub MySQL), parametry połączenia, pulowanie połączeń, migracje schematu	brak
backup	backup.yml	System kopii zapasowych — harmonogram automatycznych backupów (cron), retencja plików, format archiwum (ZIP), przywracanie z kopii	databases
webhook	webhook.yml	Powiadomienia zewnętrzne — integracja z Discord (webhooki), powiadomienia o wydarzeniach gildyjnych (założenie, upadek, wojna), formatowanie embedów	guild

Analiza powyższej tabeli ujawnia kilka istotnych właściwości architektury modułowej. Po pierwsze, moduł `guild` pełni rolę centralnego komponentu systemu — sześć innych modułów deklaruje od niego zależność, co oznacza, że bez systemu gildii nie mogą funkcjonować mechaniki rankingu, ekonomii, wieśniaków, GUI, przedmiotów dołączeniowych ani webhooków. Jest to logiczna konsekwencja tego, że wszystkie te funkcjonalności operują na encji gildii jako podstawowej jednostce organizacyjnej. Po drugie, moduły `cuboid` i `guild` są jedynymi modułami bazowymi nieposiadającymi żadnych zależności — oznacza to, że można uruchomić system z samym rdzeniem gildijnym i ochroną terenów, rezygnując jednocześnie z rankingu, ekonomii czy TAB. Po trzecie, najdłuższy łańcuch zależności ma długość trzech: `guild` → `economy` → `join_items`, co oznacza, że system ładowania musi zagwarantować inicjalizację w kolejności: najpierw `guild`, potem `economy`, na końcu `join_items`. Taki układ jest w pełni obsługiwany przez mechanizm sortowania topologicznego zaimplementowany w rdzeniu pluginu.

13.1.3 Włączanie i wyłączanie modułów bez restartu

Jedną z kluczowych zalet architektury modułowej jest możliwość modyfikacji konfiguracji modułów bez konieczności restartowania serwera Minecraft. Restart serwera to operacja kosztowna czasowo — wymaga rozłączenia wszystkich graczy, przeładowania wszystkich pluginów, ponownego wczytania światów i regeneracji chunków. W przypadku serwerów z setkami aktywnych graczy taka operacja może trwać od kilku do kilkunastu minut i jest zwykle planowana na godziny nocne. System hot-reload modułów eliminuje ten problem.

Do zarządzania modułami w czasie rzeczywistym służą trzy komendy administracyjne. Komenda `/guild admin module list` wyświetla wszystkie czternaście modułów wraz z ich aktualnym statusem (`ACTIVE`, `INACTIVE`, `ERROR`, `DEPENDENCY_MISSING`), wersją, czasem ładowania oraz listą brakujących zależności (jeśli takie występują). Komenda `/guild admin module enable <module>` aktywuje wcześniej wyłączony moduł — system najpierw weryfikuje, czy wszystkie zależności tego modułu są dostępne, następnie wczytuje plik konfiguracyjny z katalogu `modules/`, inicjalizuje wewnętrzne struktury danych, rejestruje komendy i listenery eventów, a na końcu zgłasza moduł jako aktywny. Komenda `/guild admin module disable <module>` działa w odwrotnym kierunku — wyłącza moduł, zapisuje jego stan, wyrejestrowuje komendy i zwalnia zasoby pamięci. Jeśli od wyłączonego modułu zależą inne aktywne moduły, system wyświetla ostrzeżenie i wymaga potwierdzenia operacji parametrem `--force`.

Centralnym narzędziem do przeładowania konfiguracji jest komenda `/guild admin module reload [module]`. Wywołana bez argumentu przeładowuje konfigurację wszystkich aktywnych modułów — odczytuje ponownie pliki YAML, waliduje wartości, aplikuje zmiany do działających instancji i zgłasza wykryte błędy składniowe z precyzyjnym wskazaniem linii i kolumny w pliku. Wywołana z nazwą modułu (np. `/guild admin module reload cuboid`) przeładowuje wyłącznie wskazany komponent, co jest przydatne podczas precyzyjnego dostrajania pojedynczej mechaniki. Wszystkie operacje są logowane do

konsoli z poziomem `INFO` oraz do pliku `logs/modules.log` z pełnym timestampem i identyfikatorem administratora wykonującego komendę.

Poniższa tabela zestawia wszystkie komendy administracyjne służące do zarządzania modułami w czasie rzeczywistym, wraz z wymaganymi uprawnieniami, dostępnymi argumentami i opisem efektu każdej operacji.

Komenda	Uprawnienie	Argumenty	Efekt wykonania	Typ operacji
<code>/guild admin module list</code>	<code>fgc.admin.module.list</code>	brak	Wyświetla tabelę wszystkich 14 modułów ze statusem, priorytetem, czasem ładowania i brakującymi zależnościami	Informacyjna
<code>/guild admin module enable <module></code>	<code>fgc.admin.module.enable</code>	nazwa modułu (np. <code>webhook</code>)	Weryfikuje zależności, wczytuje konfigurację, rejestruje komendy i listenery, zgłasza moduł jako <code>ACTIVE</code>	Modyfikująca
<code>/guild admin module disable <module></code>	<code>fgc.admin.module.disable</code>	nazwa modułu + opcjonalnie <code>--force</code>	Zapisuje stan, wyrejestrowuje komendy, zwalnia zasoby; <code>--force</code> pomija ostrzeżenie o zależnościach	Modyfikująca
<code>/guild admin module reload</code>	<code>fgc.admin.module.reload</code>	brak (opcjonalnie <code>[module]</code>)	Przeładowuje konfigurację wszystkich modułów lub tylko wskazanego; waliduje składnię YAML i raportuje błędy	Modyfikująca

Tabela komend ujawnia hierarchię uprawnień administracyjnych w systemie modułowym. Uprawnienie `fgc.admin.module.*` obejmuje wszystkie cztery operacje i jest domyślnie przypisane do grupy `fgc.admin`. Komendy informacyjne (typu `list`) mogą być dostępne dla moderatorów z uprawnieniem `fgc.mod.module.list`, co pozwala na diagnostykę systemu bez nadawania pełnych praw administracyjnych. Komendy modyfikujące wymagają zawsze pełnego uprawnienia administracyjnego ze względu na potencjalny wpływ na stabilność serwera — nieprawidłowe wyłączenie modułu `guild` spowoduje dezaktywację sześciu zależnych modułów i może zakłócić rozgrywkę setek graczy. Dlatego każda operacja modyfikująca jest logowana do pliku `logs/modules.log` z identyfikatorem gracza, timestampem i szczegółowym opisem wykonanych kroków.

13.2 Konfiguracja Główna (config.yml)

Plik `config.yml` zlokalizowany w katalogu głównym pluginu (`plugins/FractionGuildClans/config.yml`) pełni rolę konfiguracji nadrzędnej — definiuje parametry globalne, wspólne dla całego ekosystemu pluginu, oraz listę aktywnych modułów wraz z ich priorytetami i zależnościami. Nie zawiera on szczegółowych ustawień poszczególnych funkcjonalności — te znajdują się w dedykowanych plikach w katalogu `modules/`.

13.2.1 Sekcja `general` — wersja, język, debug, auto-update, prefix

Sekcja `general` zawiera podstawowe parametry operacyjne pluginu. Klucz `version` przechowuje numer wersji schematu konfiguracji (nie mylić z wersją pluginu) — jest używany przez system migracji do automatycznego aktualizowania struktury pliku przy wydaniach nowych wersji. Jeśli wersja schematu w pliku jest niższa niż wersja oczekiwana przez plugin, system migracji automatycznie dodaje brakujące klucze z wartościami domyślnymi. Klucz `language` określa kod aktywnego pliku językowego z katalogu `lang/` (np. `pl_PL`, `en_US`) — zmiana tej wartości wymaga przeładowania modułu `lang` komendą `/guild admin module reload lang`. Klucz `debug` przyjmuje wartość logiczną (`true` lub `false`) — w trybie debug plugin loguje szczegółowe informacje o każdej operacji, wszystkich wywołaniach eventów i czasach wykonania zapytań do bazy danych. Jest to narzędzie diagnostyczne, które nie powinno być aktywne na serwerze produkcyjnym ze względu na intensywne generowanie logów. Klucz `auto-update` decyduje o automatycznej aktualizacji pluginu przy starcie serwera — gdy włączony, plugin sprawdza repozytorium pod kątem nowszej wersji i pobiera ją, zachowując kopię poprzedniego pliku JAR z przyrostkiem `.backup`. Klucz `prefix` definiuje globalny prefiks wiadomości wyświetlany przed komunikatami pluginu — może zawierać tagi MiniMessage (`<gradient:blue:aqua>FGC</gradient>`) lub legacy kody kolorów (`&8[&bFGC&8] &7`).

13.2.2 Sekcja `database` — SQLite (domyślnie) lub MySQL

Sekcja `database` konfiguruje warstwę persystencji danych. Plugin obsługuje dwa silniki bazodanowe: SQLite (domyślny, zalecany dla serwerów do 50 aktywnych gildii) oraz MySQL (zalecany dla większych instalacji i sieci serwerów BungeeCord/Velocify). Dla SQLite wystarczy określić nazwę pliku bazy danych (domyślnie `data/guilds.db`). Dla MySQL konieczne jest podanie adresu hosta, numeru portu, nazwy bazy danych, nazwy użytkownika i hasła. Opcjonalny klucz `pool` definiuje parametry puli połączeń HikariCP (maksymalna liczba połączeń, timeout) — współczesne implementacji JDBC dla Minecraft standardowo używają HikariCP jako menedżera puli połączeń ze względu na jego wydajność i niezawodność przy dużym obciążeniu.

13.2.3 Sekcja `modules` — lista aktywnych modułów

Sekcja `modules` jest centralnym rejestrem wszystkich modułów pluginu. Każdy moduł jest reprezentowany jako wpis zawierający trzy kluczowe parametry. Klucz `enabled` (boolean) decyduje o tym, czy moduł ma zostać załadowany przy starcie serwera. Klucz `priority` (liczba całkowita) określa kolejność ładowania — moduły o niższej wartości priorytetu są inicjalizowane przed modułami o wyższej wartości. Klucz `requires` (lista ciągów znaków) opcjonalnie definiuje identyfikatory modułów, które muszą być aktywne, aby dany moduł mógł zostać załadowany. Poniższy blok YAML przedstawia przykładową konfigurację sekcji `modules` :

```
modules:
  guild:
    enabled: true
    priority: 1
  egg:
    enabled: true
    priority: 2
    requires: [cuboid]
  cuboid:
    enabled: true
    priority: 2
  ranking:
    enabled: true
    priority: 3
    requires: [guild]
  villagers:
    enabled: true
    priority: 4
    requires: [guild, cuboid]
  gui:
    enabled: true
    priority: 5
    requires: [guild]
  tab:
    enabled: true
    priority: 6
  map:
    enabled: true
    priority: 7
    requires: [cuboid]
  economy:
    enabled: true
    priority: 3
    requires: [guild]
  join_items:
    enabled: false
    priority: 4
    requires: [guild, economy]
  lang:
    enabled: true
    priority: 0
  database:
    enabled: true
    priority: 0
  backup:
    enabled: true
    priority: 8
    requires: [database]
  webhook:
    enabled: false
```

W powyższej konfiguracji system ładowania przetwarza moduły w następującej kolejności: najpierw infrastruktura (`lang` , `database`), potem rdzeń (`guild`), następnie ochrona terenów (`cuboid`), potem funkcjonalności zależne od gildii i cuboidów (`egg` , `ranking` , `economy` , `villagers` , `gui`), dalej prezentacja (`tab` , `map`), a na końcu usługi pomocnicze (`backup` , `webhook`). Moduł `join_items` jest wyłączony (`enabled: false`), więc nie zostanie załadowany mimo zdefiniowanych zależności. Moduł `webhook` również jest nieaktywny — administrator może go włączyć w dowolnym momencie komendą `/guild admin module enable webhook` , pod warunkiem że moduł `guild` jest aktywny.

13.3 Pliki Modułów — Przykłady

Każdy moduł posiada swój własny plik konfiguracyjny w katalogu `modules/` . Poniżej przedstawiono sześć pełnych, realistycznych przykładów konfiguracji dla najważniejszych modułów funkcjonalnych. Każdy przykład jest operacyjny — może zostać użyty bezpośrednio na serwerze po ewentualnym dostosowaniu wartości parametrów do specyfiki danej instancji.

13.3.1 `modules/egg.yml` — konfiguracja jaja gildii

Moduł `egg` odpowiada za centralny obiekt gildii — jajo (Egg), którego zniszczenie prowadzi do upadku gildii. Plik konfiguracyjny definiuje wszystkie aspekty mechaniki jaja: trzy etapy życia (pełne zdrowie, uszkodzone, krytyczne), materiał bloku reprezentującego jajo, koszty naprawy, parametry tarcz ochronnych oraz komunikaty wyświetlane graczom przy każdym zdarzeniu.

```

# modules/egg.yml – Konfiguracja Jaja Gildii (Egg System)
# Wersja schematu: 2.0

enabled: true

# Blok reprezentujący jajo gildii po postawieniu
block:
  type: DRAGON_EGG          # Materiał bloku (DOMYSLNIE: DRAGON_EGG)
  glowing: true             # Efekt świecenia (particle)
  hologram:
    enabled: true
    format: "&c♦ &f{guild} &7– &eŻycie: &c{lives}/3"
    offset_y: 2.2          # Odległość hologramu nad jajem (w blokach)

# System 3 etapów życia jaja
lives:
  max: 3                   # Maksymalna liczba żyć (etapów)

# Etap 3/3 – pełne zdrowie
stage_3:
  particle: HAPPY_VILLAGER
  particle_count: 5
  broadcast_damage: false

# Etap 2/3 – uszkodzone (po 1 uderzeniu)
stage_2:
  particle: SMOKE
  particle_count: 10
  broadcast_damage: true
  alert_sound: ENTITY_ELDER_GUARDIAN_CURSE
  # Wiadomość do członków gildii
  alert_message: "&cUWAGA! Jajo twojej gildii zostało uszkodzone! Pozostałe
życia: &4{lives}/3"

# Etap 1/3 – krytyczne (po 2 uderzeniach)
stage_1:
  particle: FLAME
  particle_count: 20
  broadcast_damage: true
  alert_sound: ENTITY_WITHER_SPAWN
  alert_message: "&4OSTRZEŻENIE! Jajo gildii jest w stanie krytycznym!
Ostatnie życie!"
  # Dodatkowe efekty wizualne
  screen_effect: DARKNESS  # Efekt na ekranie członków gildii

# Mechanika naprawy jaja
repair:
  enabled: true
  # Koszt naprawy – przywraca 1 życie
  cost:
    items:

```

Powyższa konfiguracja definiuje pełną mechanikę jaja z trzema etapami życia. Każdy etap ma przypisane inne efekty wizualne i dźwiękowe, co pozwala zarówno członkom gildii, jak i atakującym na bieżąco oceniać stan obiektu. Kluczowy parametr `collapse_delay_minutes: 60` wprowadza godzinny okres karencji między zniszczeniem jaja a faktycznym upadkiem gildii — daje to członkom szansę na desperacką obronę lub ewakuację zasobów. System tarcz (`shield`) zapewnia półgodzinną ochronę startową, co zapobiega natychmiastowemu raidingowi świeżo założonych gildii.

13.3.2 modules/cuboid.yml — 10 levele, flagi, strefy, trusted players

Moduł `cuboid` implementuje własny system ochrony terenów gildii, całkowicie niezależny od WorldGuard i WorldEdit. Plik konfiguracyjny definiuje dziesięć poziomów powiększania terenu, zestaw flag ochronnych, trzy strefy wewnętrzne oraz system zaufanych graczy.

```

# modules/cuboid.yml – System Ochrony Terenów (Cuboid System)
# Wersja schematu: 2.0

enabled: true

# 10 poziomów cuboidu – każdy level zwiększa promień ochrony
levels:
  1:
    radius: 15          # Promień w blokach (cuboid to walec)
    height: 64          # Wysokość ochrony (Y + i Y - od pozycji jaja)
    cost_money: 0       # Koszt ulepszenia (0 dla poziomu startowego)
    cost_items: []
    required_guild_members: 1
  2:
    radius: 25
    height: 64
    cost_money: 2500
    cost_items: ["IRON_BLOCK:16"]
    required_guild_members: 2
  3:
    radius: 35
    height: 80
    cost_money: 5000
    cost_items: ["GOLD_BLOCK:16", "IRON_BLOCK:32"]
    required_guild_members: 3
  4:
    radius: 45
    height: 80
    cost_money: 10000
    cost_items: ["DIAMOND_BLOCK:8", "GOLD_BLOCK:32"]
    required_guild_members: 4
  5:
    radius: 55
    height: 96
    cost_money: 20000
    cost_items: ["DIAMOND_BLOCK:16", "EMERALD_BLOCK:16"]
    required_guild_members: 5
  6:
    radius: 65
    height: 96
    cost_money: 35000
    cost_items: ["DIAMOND_BLOCK:32", "NETHERITE_INGOT:4"]
    required_guild_members: 6
  7:
    radius: 75
    height: 112
    cost_money: 50000
    cost_items: ["NETHERITE_BLOCK:4", "EMERALD_BLOCK:32"]
    required_guild_members: 8
  8:
    radius: 85

```

Konfiguracja modułu `cuboid` jest najobszerniejszym plikiem modułowym ze względu na złożoność systemu ochrony terenów. Dziesięć poziomów zapewnia długoterminową ścieżkę progresji — od skromnej enklawy 15-blokowej na start, po rozległy kompleks 125-blokowy na maksymalnym levelu. System trzech stref (spawn, build, PvP) wprowaduje głęboką taktykę projektowania bazy — centralny bastion, gdzie tylko władca ma dostęp, otoczony strefą budowlaną dla członków, a na peryferiach arena walk. Flagi `leader_toggle` pozwalają liderowi na dostosowanie polityki bezpieczeństwa gildii do aktualnej sytuacji dyplomatycznej.

13.3.3 `modules/ranking.yml` — wzór punktowy, sezony, nagrody

Moduł `ranking` odpowiada za system punktowy obejmujący zarówno gildie, jak i indywidualnych graczy. Plik konfiguracyjny definiuje wzór obliczeniowy, parametry sezonów rankingowych oraz strukturę nagród.


```

# modules/ranking.yml – System Punktowy i Sezonowy
# Wersja schematu: 2.0

enabled: true

# Wzór obliczeniowy punktów – dwa tryby dostępne
formula:
  # Tryb "elo" – system oparty na różnicy punktów (jak w szachach)
  # Tryb "linear" – stałe +/- punkty niezależnie od rankingu przeciwnika
  mode: elo

  # Parametry dla trybu ELO
  elo:
    base_k: 32          # Maksymalna liczba punktów do zdobycia
    k_scaling: true     # Skalowanie K w zależności od punktów
    k_min: 10           # Minimalne K dla wysokopunktowanych
    k_max: 50           # Maksymalne K dla nowych graczy
    min_points: 1       # Minimalne punkty po przegranej (nie spadnie
    # poniżej)
    start_points: 1000   # Punkty startowe

  # Parametry dla trybu liniowego
  linear:
    kill_points: 10      # Punkty za zabójstwo
    death_penalty: 5     # Punkty tracone za śmierć
    assist_points: 3     # Punkty za asystę
    start_points: 0

# System asyst – kto dostaje punkty pomocnicze
assists:
  enabled: true
  time_window_seconds: 15 # Kto zadał obrażenia w ciągu 15s przed śmiercią
  require_damage_percent: 15 # Minimalny % obrażeń dla asysty
  max_assists: 3          # Max 3 graczy dostaje punkty za asystę

# Sezony rankingowe
seasons:
  enabled: true
  # Harmonogram sezonów – można zdefiniować wiele
  schedule:
    - name: "Wiosna 2025"
      start: "2025-03-01 00:00:00"
      end: "2025-05-31 23:59:59"
    - name: "Lato 2025"
      start: "2025-06-01 00:00:00"
      end: "2025-08-31 23:59:59"
  # Co się dzieje po zakończeniu sezonu
  on_season_end:
    reset_points: false # Nie resetuj punktów (trwale na serwerze)
    archive_leaderboard: true # Zachowaj tablicę wyników w historii
    distribute_rewards: true # Rozdaj nagrody

```

System rankingowy z trybem ELO zapewnia sprawiedliwe rozłożenie punktów — pokonanie silniejszego przeciwnika daje więcej punktów niż pokonanie słabszego. Parametr `k_scaling` dynamicznie dostosowuje wartość `K` do doświadczenia gracza, co zapobiega sytuacjom, w których weteran traci ogromną liczbę punktów za jedną pomyłkę. System asyst wraz z progiem 15% obrażeń motywuje graczy do współpracy zespołowej bez nagradzania przypadkowych trafień. Sezony rankingowe z archiwizacją wyników budują długoterminową narrację rywalizacji na serwerze.

13.3.4 `modules/villagers.yml` — typy, oferty, zadania, relacje

Moduł `villagers` wprowadza niestandardowych wieśniaków związanych z gildią — handlarzy oferujących unikalne towary, zleceniodawców dających zadania oraz strażników chroniących teren.

```

# modules/villagers.yml – Wieśniacy Gildii
# Wersja schematu: 2.0

enabled: true

# Typy wieśniaków dostępnych w gildii
types:
  merchant:          # Handlarz – wymiana przedmiotów
    name: "&6Handlarz Gildii"
    skin_texture:
      "eyJ0ZXh0dXJlcYI6eyJTS0l0Ijp7InVybCI6Imh0dHA6Ly90ZXh0dXJlcY5taW5lY3JhZnQubmV0L3RleHR1cmUvN2I1ODUzNDkxZGY5Yjk2M2UxN2Q1N2RmMmMwYmRlZTZkOGM1Y2ZjODk4YzkyYWYxZTA2ZDNkMmM1Zjc0ODg4In19fQ=="
    max_per_guild: 2
    require_guild_level: 3  # Dostępny od levelu gildii 3

  questmaster:       # Zleceniodawca – zadania dla członków
    name: "&9Zleceniodawca"
    skin_texture:
      "eyJ0ZXh0dXJlcYI6eyJTS0l0Ijp7InVybCI6Imh0dHA6Ly90ZXh0dXJlcY5taW5lY3JhZnQubmV0L3RleHR1cmUvZjEyY2U4N2VmOGYzOTUxYjI3ZTQ0MzY3ODJiMTlhY2QxOWZmZGU0MDRlY2Y3MzVlYzk5Y2QyODAzYjcxNyJ9fX0="
    max_per_guild: 1
    require_guild_level: 5

  guard:             # Strażnik – ochrona terenu (AI)
    name: "&cStrażnik"
    skin_texture:
      "eyJ0ZXh0dXJlcYI6eyJTS0l0Ijp7InVybCI6Imh0dHA6Ly90ZXh0dXJlcY5taW5lY3JhZnQubmV0L3RleHR1cmUvOThiN2NhMjg0MDIzZDYxNDk2OGVhYjBlNzU3YWQ4YjY4NDM3YjI2NWU1YzhmYzQ2MTEzMzg2MTUwNmE2Yjk1In19fQ=="
    max_per_guild: 4
    require_guild_level: 7

# Oferty handlarza – rotacyjne (zmieniają się co reset)
merchant_offers:
  reset_interval_hours: 24
  max_offers_at_once: 6  # Ile ofert widocznych jednocześnie

# Pula możliwych ofert – system losuje 6 przy każdym resecie
pool:
  - id: "diamond_trade"
    input: "EMERALD:32"
    output: "DIAMOND:4"
    max_uses: 4
  - id: "obsidian_trade"
    input: "GOLD_INGOT:16"
    output: "OBSIDIAN:32"
    max_uses: 8
  - id: "enchanted_book"
    input: "DIAMOND:8"

```

Moduł wieśniaków wprowadza element gospodarczy i fabularny do życia gildii. Handlarz z rotacyjnym pulą ofert zmusza graczy do regularnego odwiedzania bazy gildii i monitorowania dostępnych transakcji. Zleceniodawca generuje codzienne cele, które integrują mechanikę rankingu z ekonomią — wykonanie zadania daje zarówno walutę, jak i punkty rankingowe. System reputacji nagradza lojalność wobec gildii, tworząc długoterminową relację między graczem a jego organizacją.

13.3.5 modules/gui.yml — menu, podmenu, animacje, dźwięki

Moduł `gui` centralizuje konfigurację wszystkich menu graficznych pluginu — od głównego panelu gildii, przez ekrany zarządzania członkami, aż po interfejsy administracyjne.

```

# modules/gui.yml – System Menu Graficznych (GUI)
# Wersja schematu: 2.0

enabled: true

# Główne menu gildii – otwierane komendą /guild lub kliknięciem NPC
main_menu:
  title: "&8[&b{guild}&8] &7Menu Gildii"
  rows: 5 # Wysokość GUI w wierszach (1-6)

# Wypełnienie tła – dekoracyjne pane szkła
background:
  enabled: true
  material: BLACK_STAINED_GLASS_PANE
  name: " "

# Sloty i ich zawartość
slots:
  10:
    material: PLAYER_HEAD
    name: "&eCzłonkowie &7({members_online}/{members_total})"
    lore:
      - "&7Kliknij aby zarządzać"
      - "&7członkami gildii."
      - ""
      - "&e→ &fKliknij lewym"
    action: OPEN_MEMBERS_GUI
    sound: ENTITY_EXPERIENCE_ORB_PICKUP
  12:
    material: IRON_PICKAXE
    name: "&6Rozbudowa Cuboidu"
    lore:
      - "&7Aktualny level: &e{cuboid_level}"
      - "&7Promień: &e{cuboid_radius} &7bloków"
      - "&7Koszt ulepszenia: &e{upgrade_cost}"
      - ""
      - "&e→ &fKliknij aby ulepszyć"
    action: OPEN_UPGRADE_GUI
    sound: BLOCK_ANVIL_USE
  14:
    material: GOLDEN_SWORD
    name: "&cRanking Gildii"
    lore:
      - "&7Pozycja: &e#{guild_rank}"
      - "&7Punkty: &e{guild_points}"
      - "&7Zabójstwa: &a{kills} &7/ &c{deaths}"
      - ""
      - "&e→ &fKliknij aby zobaczyć topkę"
    action: OPEN_RANKING_GUI
    sound: ENTITY_PLAYER_LEVELUP
  16:

```

Konfiguracja GUI oparta jest na prostym systemie slotów — każdy numer slotu odpowiada pozycji w siatce ekwipunku (0-8 to pierwszy wiersz, 9-17 to drugi, itd.). Dla każdego slotu administrator definiuje materiał ikony, nazwę z kolorami, opis (`lore`) oraz akcję wykonywaną po kliknięciu. System wspiera wszystkie standardowe typy interakcji — otwieranie podmenu, wykonywanie komend, teleportację, przełączanie flag. Sekcja animacji dodaje płynności do interfejsu — zamiast natychmiastowego pojawienia się wszystkich ikon, GUI może "rozmazać się" lub wypełnić losowo, co znacząco podnosi postrzeganą jakość pluginu przez graczy.

13.3.6 `modules/tab.yml` — nagłówek, kolumny, stopka, placeholdery

Moduł `tab` zarządza wyglądem listy graczy (TAB) wyświetlane po wciśnięciu klawisza `Tab` w grze. Konfiguracja obejmuje nagłówek, stopkę, definicje kolumn z danymi oraz częstotliwość odświeżania.

```

# modules/tab.yml – Custom TAB (Lista Graczy)
# Wersja schematu: 2.0

enabled: true

# Nagłówek listy graczy – wyświetlany na górze TAB
header:
  lines:
    - "&8&m"
    - ""
    - "&b&lFRACTION GUILD CLANS"
    - "&7Sezon: &e{season_name} &7| &fTPS: &a{tps} &7| &fOnline: &a{online}&7/&a{max}"
    - ""
    - "&7Twoja gildia: &e{guild_tag} &7| Pozycja: &e#{guild_rank}"
    - "&8&m"
  animation:
    enabled: false # Nagłówek statyczny – bez animacji

# Stopka listy graczy – wyświetlana na dole TAB
footer:
  lines:
    - "&8&m"
    - ""
    - "&7Twój ranking: &e{player_points} pkt &7(&a{kills}&7/&c{deaths}&7)"
    - "&7Ping: &e{ping}ms &7| &7Godzina: &e{time}"
    - ""
    - "&ewww.twojserwer.pl"
    - "&8&m"

# Konfiguracja kolumn
# Minecraft wyświetla TAB w trzech kolumnach: lewa, środkowa, prawa
columns:
  # Lewa kolumna – TOP gildii
  left:
    title: "&e&lTOP GILDIE"
    entries: 15 # Ile pozycji wyświetlić
    content:
      type: GUILD_TOP # Typ danych: ranking gildii
      format: "&7{pos}. &f{guild} &7- &6{points}"
      update_interval_ticks: 200 # Odświeżanie co 10 sekund (20 ticków = 1s)

  # Środkowa kolumna – lista graczy online
  middle:
    title: "&b&lGRACZE &7({online})"
    entries: 40 # Dynamicznie – wszyscy online
    content:
      type: PLAYER_LIST # Lista graczy z dodatkowymi danymi
      format: "{guild_tag}&r {name}"
      # Sortowanie graczy – najpierw wg gildii, potem alfabetycznie
      sort_by:

```

Moduł TAB jest jednym z najbardziej widocznych elementów pluginu dla graczy — lista wyświetlana po wciśnięciu klawisza `Tab` jest stale obecna podczas rozgrywki. Podział na trzy kolumny z rankingiem gildii, listą online i rankingiem graczy tworzy funkcjonalny dashboard serwera. Częstotliwość odświeżania różni się w zależności od kolumny — lista graczy (`middle`) odświeża się co 3 sekundy, ponieważ stan online zmienia się dynamicznie, podczas gdy rankingi (`left`, `right`) odświeżają się co 10 sekund, gdyż zmieniają się znacznie rzadziej. Taka optymalizacja redukuje liczbę aktualizacji pakietów wysyłanych do klientów gry, co ma znaczenie przy dużej liczbie graczy online.

Poniższa tabela podsumowuje parametry techniczne wszystkich sześciu zaprezentowanych modułów — zakres konfiguracji, liczbę kluczy YAML, wymagane zależności oraz typowe przypadki użycia na serwerze produkcyjnym.

Moduł	Klucze konfiguracyjne	Główne sekcje	Zależności	Typowe zastosowanie
<code>egg</code>	~45 kluczy	<code>block</code> , <code>lives</code> , <code>repair</code> , <code>shield</code> , <code>destruction</code> , <code>explosion_resistance</code>	<code>cuboid</code>	Serwery PvP z raidingiem — mechanika jajo-raid jako cel ataków
<code>cuboid</code>	~80+ kluczy	<code>levels</code> (10), <code>flags</code> (9), <code>zones</code> (3), <code>trusted</code>	brak	Każdy serwer gildyjny — ochrona terenu jest fundamentem
<code>ranking</code>	~40 kluczy	<code>formula</code> (<code>elo/linear</code>), <code>assists</code> , <code>seasons</code> , <code>rewards</code> , <code>leaderboard</code>	<code>guild</code>	Serwery z rywalizacją — sezony, turnieje, rankingi
<code>villagers</code>	~50+ kluczy	<code>types</code> (3), <code>merchant_offers</code> , <code>quests</code> (3 typy), <code>reputation</code>	<code>guild</code> , <code>cuboid</code>	Serwery RPG — wieśniacy jako element fabularny i ekonomiczny
<code>gui</code>	~60+ kluczy	<code>main_menu</code> (8 slotów), <code>animations</code> , <code>sounds</code>	<code>guild</code>	Serwery nastawione na UX — graficzne menu zamiast komend
<code>tab</code>	~35 kluczy	<code>header</code> , <code>footer</code> , <code>columns</code> (3), <code>placeholders</code> (~20), <code>advanced</code>	brak	Serwery z dużą liczbą graczy — profesjonalny wygląd TAB

Analiza tabeli pozwala na świadomy wybór aktywnych modułów w zależności od profilu serwera. Minimalna konfiguracja wymaga jedynie modułów `guild` i `cuboid` — zapewniają one rdzeń funkcjonalny (zakładanie gildii i ochronę terenu) bez zbędnych dodatków. Taka konfiguracja jest odpowiednia dla serwerów survival z lekkim akcentem gildijnym. Serwery PvP/raidingowe powinny

aktywować dodatkowo moduły `egg` i `ranking`, które tworzą kompletną pętlę rozgrywkową: zakładasz gildię → budujesz bazę → atakujesz inne gildie → zdobywasz punkty rankingowe. Serwery zorientowane na doświadczenie użytkownika (UX) powinny włączyć moduły `gui` i `tab`, które zastępują tekstowe komendy intuicyjnymi menu i profesjonalnym wyglądem listy graczy. Serwery RPG z fabułą zyskają najwięcej z modułu `villagers`, który wprowadza gospodarkę wewnątrzgildijną i system zadań. Modułarna architektura pozwala na dowolną kombinację tych komponentów bez konieczności modyfikacji kodu źródłowego — wystarczy edycja pliku `config.yml` i przeładowanie komendą `/guild admin module reload`.

12. Komendy i Uprawnienia

Interfejs tekstowy pluginu Fraction Guild Clans oparty jest na dwóch głównych drzewach komend: `/guild` (alias `/g`) dla graczy oraz `/guild admin` dla administratorów serwera. Każda komenda wymaga odpowiednich uprawnień (permissions) zdefiniowanych w systemie zarządzania dostępem — najczęściej LuckPerms lub PermissionsEx. Plugin rejestruje wszystkie komendy w momencie uruchomienia serwera, a ich dostępność dla konkretnego gracza weryfikowana jest na poziomie parsera komend poprzez mapowanie uprawnień na node'y permission. Niezależnie od tego, czy gracz posługuje się interfejsem GUI (rozdział 8), czy komendami tekstowymi, wywoływane są te same metody backendowe, co gwarantuje spójność stanu gildii.

12.1 Komendy Gracza

12.1.1 Podstawowe komendy: aliasy i struktura menu pomocy

Plugin rejestruje dwa aliasy główne dla drzewa komend gracza: `/guild` oraz skróconą formę `/g`. Oba aliasy prowadzą do identycznego parsera komend i nie różnią się funkcjonalnie. Wywołanie komendy bez argumentów (`/guild` lub `/g`) przekierowuje gracza do głównego menu GUI gildii (opisanego szczegółowo w rozdziale 8), pod warunkiem że gracz należy do gildii. Jeśli gracz nie jest członkiem żadnej gildii, wyświetlany jest komunikat informacyjny z propozycją użycia komendy `/guild create` lub `/guild join`.

Komenda `/guild help` wyświetla paginowane menu tekstowe dostępnych komend z uwzględnieniem uprawnień gracza. Oznacza to, że gracz widzi wyłącznie te komendy, do których posiada przypisane uprawnienia — lista jest filtrowana dynamicznie na poziomie serwera, a nie klienta. Struktura menu pomocy dzieli komendy na kategorie: Podstawowe, Zarządzanie, Cuboid, Jajo, Handel i Ranking. Każdy wpis zawiera składnię komendy, krótki opis funkcji oraz wymagane uprawnienie w formacie `permission.node`.

12.1.2 Komendy tworzenia i zarządzania gildią

Komenda `/guild create <nazwa> <tag>` inicjuje proces zakładania gildii. Parametr `nazwa` określa pełną nazwę gildii (od 3 do 32 znaków alfanumerycznych plus spacje), natomiast `tag` to krótki identyfikator wyświetlany przed nickiem gracza (od 2 do 6 znaków, bez spacji). Po wykonaniu komendy plugin weryfikuje uprawnienia gracza (czas gry, poziom, brak cooldownu), sprawdza dostępność nazwy i tagu w bazie danych, a następnie — jeśli skonfigurowano wymagane itemy — otwiera GUI potwierdzenia z listą potrzebnych surowców. Po akceptacji generowany jest cuboid bazowy, jajo obronne umieszczane jest w centrum, a gracz otrzymuje rangę Lidera.

Komenda `/guild disband` przeznaczona jest wyłącznie dla lidera gildii i powoduje trwałe rozwiązanie gildii. Przed wykonaniem plugin wymusza potwierdzenie poprzez wpisanie nazwy gildii lub kliknięcie potwierdzenia w GUI. Po rozwiązaniu wszystkie dane gildii (członkowie, cuboid, jajo, wieśniacy, sojusze) są usuwane z bazy danych, a członkom wysyłany jest komunikat systemowy.

Komenda `/guild invite <nick>` pozwala liderowi lub moderatorowi gildii wysłać zaproszenie do gracza znajdującego się online na serwerze. Zaproszenie jest ważne przez czas skonfigurowany w `config.yml` (domyślnie 5 minut). Zaproszony gracz otrzymuje komunikat z opcją akceptacji (`/guild join <tag>`) lub odrzucenia.

Komenda `/guild kick <nick>` wymaga rangi moderatora lub wyższej. Pozwala na wyrzucenie członka gildii z niższą rangą niż wykonujący komendę. Lidera gildii nie można wyrzucić — konieczne jest najpierw przekazanie przywództwa komendą `/guild leader <nick>`.

Komenda `/guild leave` pozwala członkowi na opuszczenie gildii. Lider nie może opuścić gildii bez wcześniejszego przekazania przywództwa lub rozwiązania gildii. Po opuszczeniu gracz traci dostęp do cuboidu, a jego dane są usuwane z listy członków gildii w bazie danych.

Komenda `/guild info [tag]` wyświetla informacje o gildii. Wywołana bez argumentu pokazuje dane gildii, do której należy gracz. Argument `tag` pozwala sprawdzić informacje o dowolnej gildii — nazwa, pełna nazwa, lider, liczba członków, poziom cuboidu, punkty rankingowe, data założenia oraz status sojuszy.

Komenda `/guild menu` otwiera główne GUI gildii z opisaną w rozdziale 8 strukturą sześciu rzędów. Jest to funkcjonalny odpowiednik wywołania `/guild` bez argumentów.

12.1.3 Komendy cuboidu

Komenda `/guild cuboid info` wyświetla szczegółowe parametry cuboidu gildii, do której należy gracz: aktualny promień, poziom, współrzędne środka (X, Z), powierzchnię w blokach, listę aktywnych flag oraz datę wygaśnięcia ewentualnego rozszerzenia tymczasowego. Komenda dostępna jest dla wszystkich członków gildii.

Komenda `/guild cuboid expand` inicjuje proces awansu cuboidu do następnego poziomu. Plugin weryfikuje, czy gildia spełnia wszystkie wymagania (itemy, waluta, punkty, czas istnienia — szczegóły w rozdziale 5), a następnie wyświetla GUI potwierdzenia z listą kosztów. Po akceptacji cuboid jest rozszerzany, a nowe granice wizualizowane cząsteczkami przez 10 sekund.

Komenda `/guild cuboid setspawn` dostępna dla lidera i moderatorów, ustawia punkt odrodzenia (spawn) gildii wewnątrz jej cuboidu. Punkt ten musi znajdować się w strefie bezpiecznej, co weryfikuje plugin automatycznie. Wywołanie komendy poza granicami cuboidu skutkuje komunikatem błędu.

Komenda `/guild cuboid flags` otwiera GUI zarządzania flagami cuboidu (budowanie, niszczenie, PvP, użycie wiader, wchodzenie obcych). Dostępna dla lidera i moderatorów. Szczegółowa konfiguracja flag opisana została w rozdziale 5.

12.1.4 Komendy jaja gildii

Komenda `/guild egg info` wyświetla status jaja obronnego gildii: aktualny etap integralności (0–3, gdzie 0 to pełna sprawność, a 3 to całkowite zniszczenie), czas do naturalnej regeneracji, dostępność tarczy ochronnej oraz współrzędne fizycznego bloku jaja w świecie.

Komenda `/guild egg repair` inicjuje ręczną naprawę jaja. Plugin sprawdza, czy gracz posiada w ekwipunku wymagane surowce (konfigurowalne w `config.yml`), a następnie przywraca jeden etap integralności jaja. Naprawa podlega cooldownowi globalnemu dla całej gildii (domyślnie 1 godzina).

Komenda `/guild egg shield` aktywuje tymczasową tarczę ochronną na jajo, o ile gildia posiada zakupioną wcześniej tarczę lub gracz ma odpowiednie uprawnienia VIP. Tarcza blokuje możliwość ataku na jajo przez skonfigurowany czas (domyślnie 30 minut).

Komenda `/guild egg teleport` teleportuje gracza do fizycznej lokalizacji jaja w świecie. Komenda dostępna wyłącznie dla członków gildii i działa tylko w obrębie tego samego świata (dimension), w którym znajduje się jajo.

Poniższa tabela zawiera kompletne zestawienie wszystkich komend gracza wraz z ich składnią, wymaganymi uprawnieniami i krótkim opisem funkcjonalnym.

Komenda	Składnia	Uprawnienie	Opis
Główna	<code>/guild</code>	<code>guild.user.men u</code>	Otwiera GUI główne gildii lub wyświetla komunikat dla graczy bez gildii
Alias	<code>/g</code>	<code>guild.user.men u</code>	Skrócona forma <code>/guild</code> , identyczna funkcjonalność

Pomoc	<code>/guild help</code> <code>[strona]</code>	<code>guild.user.help</code>	Wyświetla paginowaną listę dostępnych komend z uwzględnieniem uprawnień
Założ gildię	<code>/guild create</code> <code><nazwa> <tag></code>	<code>guild.user.create</code>	Inicjuje proces zakładania gildii z weryfikacją nazwy, tagu i wymagań
Rozwiąż gildię	<code>/guild disband</code>	<code>guild.user.disband</code>	Rozwiązuje gildię po potwierdzeniu; wymaga rangi lidera
Zaproś gracza	<code>/guild invite</code> <code><nick></code>	<code>guild.user.invite</code>	Wysyła zaproszenie do gracza online; wymaga rangi moderatora+
Wyrzuć gracza	<code>/guild kick</code> <code><nick></code>	<code>guild.user.kick</code>	Usuwa członka z niższą rangą z gildii
Opuść gildię	<code>/guild leave</code>	<code>guild.user.leave</code>	Powoduje opuszczenie gildii przez gracza; lider nie może użyć
Info o gildii	<code>/guild info</code> <code>[tag]</code>	<code>guild.user.info</code>	Wyświetla informacje o własnej lub dowolnej gildii
Menu gildii	<code>/guild menu</code>	<code>guild.user.menu</code>	Otwiera główny interfejs GUI gildii
Info o cuboidzie	<code>/guild cuboid info</code>	<code>guild.user.cuboid.info</code>	Wyświetla parametry cuboidu: promień, poziom, flagi
Rozwiń cuboid	<code>/guild cuboid expand</code>	<code>guild.user.cuboid.expand</code>	Inicjuje awans cuboidu na wyższy poziom po weryfikacji wymagań
Ustaw spawn	<code>/guild cuboid setspawn</code>	<code>guild.user.cuboid.setspawn</code>	Ustawia punkt odrodzenia gildii w strefie bezpiecznej cuboidu
Flagi cuboidu	<code>/guild cuboid flags</code>	<code>guild.user.cuboid.flags</code>	Otwiera GUI zarządzania flagami ochronnymi cuboidu
Info o jaju	<code>/guild egg info</code>	<code>guild.user.egg.info</code>	Wyświetla status integralności, regeneracji i tarczy jaja
Napraw jajo	<code>/guild egg repair</code>	<code>guild.user.egg.repair</code>	Inicjuje ręczną naprawę jaja przy użyciu wymaganych surowców
Tarcza jaja	<code>/guild egg shield</code>	<code>guild.user.egg.shield</code>	Aktywuje tymczasową tarczę ochronną na jajo gildii

Teleport do jaja	<code>/guild egg teleport</code>	<code>guild.user.egg.teleport</code>	Teleportuje gracza do fizycznej lokalizacji jaja w świecie
Ranking	<code>/guild top [kategoria]</code>	<code>guild.user.top</code>	Wyświetla paginowaną listę rankingową gildii

Tabela obejmuje osiemnaście komend podstawowych podzielonych na cztery funkcjonalne kategorie. Komendy `/guild` i `/g` stanowią aliasy tego samego polecenia, co upraszcza nawigację dla doświadczonych graczy przyzwyczajonych do skróconych form komend znanych z innych pluginów gildyjnych. Warto zwrócić uwagę, że większość komend z kategorii zarządzania (`disband`, `invite`, `kick`, `setspawn`, `flags`) posiada wewnętrzne walidatory rangowe — parser sprawdza nie tylko podstawowe uprawnienie `permission`, ale także aktualną rangę gracza w strukturze gildii, co zapobiega sytuacjom, w których zwykły członek z nadanym uprawnieniem `guild.user.kick` mógłby wyrzucić moderatora. Mechanizm ten działa hierarchicznie: Lider > Moderator > Członek > Rekrut.

12.2 Komendy Administratora

12.2.1 Struktura komend administracyjnych

Wszystkie komendy administratora scentralizowane są pod prefiksem `/guild admin <subkomenda>`, co zapobiega przypadkowemu ich wykorzystaniu przez graczy i ułatwia konfigurację uprawnień opartą na prefiksach (prefix matching). Plugin nie rejestruje aliasów dla drzewa administracyjnego — wywołanie `/ga` lub `/gadmin` nie jest obsługiwane, co stanowi dodatkowe zabezpieczenie przed nieautoryzowanym dostępem.

Parser komend administracyjnych posiada rozszerzoną walidację: przed wykonaniem jakiegokolwiek podkomendy sprawdzane jest uprawnienie `guild.admin.*` lub jego specyficzny odpowiednik, a następnie logowane jest wywołanie do pliku `logs/admin-commands.log` wraz z timestampem, nickiem administratora i pełnym argumentem komendy. Mechanizm ten pozwala na pełną audytowalność działań administracyjnych i w razie incydentów umożliwia identyfikację osoby odpowiedzialnej za zmianę stanu gildii.

12.2.2 Komendy moderacji gildii

Komenda `/guild admin delete <tag>` wymusza natychmiastowe rozwiązanie gildii bez konieczności potwierdzenia przez lidera. Wszystkie dane gildii są usuwane z bazy, a członkom wyświetlany jest komunikat systemowy informujący o administracyjnym rozwiązaniu gildii wraz z nickiem administratora. Komenda nie podlega cooldownowi i działa nawet jeśli lider gildii jest offline.

Komenda `/guild admin rename <stary_tag> <nowa_nazwa> <nowy_tag>` pozwala administratorowi na zmianę nazwy i tagu gildii. Plugin automatycznie weryfikuje dostępność nowego tagu,

aktualizuje wszystkie referencje w bazie danych (sojusze, ranking, logi) oraz przeładowuje cache gildii. Zmiana nazwy propagowana jest natychmiast do wszystkich systemów zależnych, w tym mapy HTML i tabulatora.

Komenda `/guild admin setpoints <tag> <wartość>` nadpisuje punktację rankingu gildii na podaną wartość. Akceptowane są wartości całkowite (łącznie z ujemnymi). Komenda przydatna w sytuacjach korekty punktacji po błędzie systemowym lub oszustwie.

Komenda `/guild admin setlevel <tag> <poziom>` zmienia poziom cuboidu gildii na wskazany (zakres 1–10). W przeciwieństwie do naturalnego awansu, zmiana poziomu przez administratora nie wymaga spełnienia warunków ekonomicznych i nie pobiera surowców od gildii. Cuboid jest natychmiast rozszerzany lub zmniejszany do promienia odpowiadającego nowemu poziomowi.

Komenda `/guild admin bypass <tag> [czas]` przełącza tryb omijania ochrony cuboidu dla administratora. W trybie bypass administrator może budować i niszczyć na terenie dowolnego cuboidu niezależnie od ustawień flag. Opcjonalny parametr `czas` (w minutach) ogranicza czas działania bypassu; bez tego parametru tryb aktywny jest do momentu ponownego wywołania komendy lub wylogowania.

12.2.3 Komendy narzędziowe

Komenda `/guild admin verify` przeprowadza pełną weryfikację stanu pluginu: sprawdza spójność bazy danych, poprawność plików konfiguracyjnych, dostępność zależności (Vault, PlaceholderAPI) oraz integralność wszystkich zarejestrowanych cuboidów i jaj. Wynik weryfikacji wyświetlany jest administratorowi w formie raportu tekstowego z podziałem na sekcje (Baza danych, Konfiguracja, Zależności, Obiekty). Komenda ta jest również wywoływana automatycznie podczas uruchamiania serwera.

Komenda `/guild admin reload` przeładowuje wszystkie pliki konfiguracyjne pluginu (`config.yml`, `messages.yml`, `gui.yml`, `tab.yml`) bez konieczności restartu serwera. Zmiany w sekcjach wymagających reinicjalizacji (np. typ bazy danych, aktywne światy) są oznaczane w raporcie jako wymagające restartu i nie są stosowane.

Komenda `/guild admin backup [typ]` tworzy ręczną kopię zapasową danych pluginu. Parametr `typ` może przyjąć wartości: `full` (pełna kopia wszystkich tabel), `guilds` (tylko dane gildii), `players` (tylko dane graczy) lub `config` (tylko pliki konfiguracyjne). Backup zapisywany jest w folderze `plugins/FractionGuildClans/backups/` z timestampem w nazwie pliku.

Komenda `/guild admin migrate <źródło>` uruchamia narzędzie migracji danych z innych pluginów gildyjnych. Obsługiwane źródła: `factions`, `guilds`, `simpleclans`. Proces migracji jest nieodwracalny i przed jego rozpoczęciem plugin automatycznie tworzy pełny backup bieżących danych. Komenda wymaga potwierdzenia wpisaniem `/guild admin migrate <źródło> confirm`.

Komenda `/guild admin debug [moduł] [poziom]` steruje systemem diagnostycznym pluginu. Parametr `moduł` określa obszar logowania (dostępne: `database`, `cuboid`, `egg`, `ranking`, `all`), a

`poziom` — poziom szczegółowości (`OFF`, `ERROR`, `WARN`, `INFO`, `DEBUG`, `TRACE`). Logi diagnostyczne trafiają do pliku `logs/debug.log`. Tryb debugowania resetowany jest po restarcie serwera.

Poniższa tabela zawiera kompletne zestawienie komend administratora.

Subkomenda	Składnia	Uprawnienie	Opis
Usuń gildię	<code>/guild admin delete <tag></code>	<code>guild.admin.delete</code>	Natychmiastowe rozwiązanie gildii bez potwierdzenia lidera
Zmień nazwę	<code>/guild admin rename <tag> <nazwa> <nowy_tag></code>	<code>guild.admin.rename</code>	Zmienia nazwę i tag gildii z aktualizacją referencji
Ustaw punkty	<code>/guild admin setpoints <tag> <wartość></code>	<code>guild.admin.setpoints</code>	Nadpisuje punktację rankingową gildii
Ustaw poziom	<code>/guild admin setlevel <tag> <1-10></code>	<code>guild.admin.setlevel</code>	Zmienia poziom cuboidu bez weryfikacji wymagań
Bypass ochrony	<code>/guild admin bypass [czas]</code>	<code>guild.admin.bypass</code>	Omija ograniczenia cuboidów; opcjonalny limit czasowy
Weryfikacja	<code>/guild admin verify</code>	<code>guild.admin.verify</code>	Pełny audyt stanu pluginu i spójności danych
Przeładowanie	<code>/guild admin reload</code>	<code>guild.admin.reload</code>	Przeładowuje pliki konfiguracyjne bez restartu serwera
Kopia zapasowa	<code>/guild admin backup [typ]</code>	<code>guild.admin.backup</code>	Tworzy ręczny backup danych (full/guilds/players/config)
Migracja	<code>/guild admin migrate <źródło></code>	<code>guild.admin.migrate</code>	Migruje dane z pluginów: factions, guilds, simpleclans
Debug	<code>/guild admin debug [moduł] [poziom]</code>	<code>guild.admin.debug</code>	Steruje szczegółowością logów diagnostycznych

Zestawienie obejmuje dziesięć komend administracyjnych podzielonych na dwie grupy funkcjonalne: moderację gildii (pięć komend) oraz narzędzia techniczne (pięć komend). Wszystkie komendy z kategorii moderacji bezpośrednio modyfikują stan gildii i są nieodwracalne — plugin nie implementuje systemu undo dla operacji administracyjnych, co wynika z prostoty architektury danych i konieczności zapewnienia

natychmiastowej spójności stanu między serwerem a bazą danych. Dlatego przed użyciem komend `delete`, `rename`, `setpoints` lub `setlevel` zaleca się wykonanie backupu. Komendy narzędziowe (`verify`, `reload`, `backup`, `migrate`, `debug`) nie modyfikują bezpośrednio stanu gildii, lecz operują na warstwie infrastrukturalnej pluginu. Szczególną uwagę należy zwrócić na komendę `migrate` — jej wykonanie jest procesem wieloetapowym, podczas którego plugin odczytuje schemat danych źródłowego pluginu, mapuje encje na własny model obiektowy, waliduje spójność referencyjną i zapisuje przekształcone dane do bieżącej bazy. Cały proces jest wykonywany synchronicznie na głównym wątku serwera, dlatego dla dużych zbiorów danych (powyżej 500 gildii) zaleca się wykonanie migracji podczas okna konserwacyjnego, gdy serwer jest pusty.

12.3 System Uprawnień (Permissions)

System uprawnień pluginu Fraction Guild Clans oparty jest na standardowym API Bukkita (`org.bukkit.permissions`). Wszystkie node'y permission rozpoczynają się prefiksem `guild.` i zorganizowane są hierarchicznie przy użyciu notacji kropkowej, co umożliwia efektywne zarządzanie dostępem poprzez wildcard permissions wspierane przez nowoczesne systemy zarządzania uprawnieniami, takie jak LuckPerms.

12.3.1 Uprawnienia gracza: `guild.user.*`

Zestaw uprawnień gracza obejmuje wszystkie komendy niezbędne do pełnego korzystania z funkcjonalności gildii bez możliwości ingerencji w dane innych gildii lub stan systemu. Uprawnienia podzielone są na podgrupy odpowiadające modułom pluginu: podstawowe (`user.*`), cuboid (`user.cuboid.*`) oraz jajo (`user.egg.*`).

Uprawnienie	Domyślna wartość	Opis
<code>guild.user.*</code>	false	Główny węzeł wildcard obejmujący wszystkie uprawnienia gracza
<code>guild.user.menu</code>	true	Dostęp do komendy <code>/guild</code> i <code>/guild menu</code> otwierających GUI
<code>guild.user.help</code>	true	Dostęp do paginowanego menu pomocy <code>/guild help</code>
<code>guild.user.create</code>	true	Możliwość zakładania gildii komendą <code>/guild create</code>
<code>guild.user.disband</code>	true	Możliwość rozwiązania gildii (wymaga rangi lidera)

<code>guild.user.invite</code>	true	Możliwość zapraszania graczy (wymaga rangi moderatora+)
<code>guild.user.kick</code>	true	Możliwość wyrzucania członków z niższą rangą
<code>guild.user.leave</code>	true	Możliwość opuszczenia gildii
<code>guild.user.info</code>	true	Dostęp do informacji o własnej i obcych gildiach
<code>guild.user.top</code>	true	Dostęp do rankingu gildii
<code>guild.user.cuboid.info</code>	true	Podgląd parametrów cuboidu gildii
<code>guild.user.cuboid.expand</code>	true	Inicjowanie awansu poziomu cuboidu
<code>guild.user.cuboid.setspawn</code>	true	Ustawianie punktu odrodzenia gildii
<code>guild.user.cuboid.flags</code>	true	Zarządzanie flagami ochronnymi cuboidu
<code>guild.user.egg.info</code>	true	Podgląd statusu jaja obronnego
<code>guild.user.egg.repair</code>	true	Ręczna naprawa jaja przy użyciu surowców
<code>guild.user.egg.shield</code>	true	Aktywacja tarczy ochronnej jaja
<code>guild.user.egg.teleport</code>	true	Teleportacja do lokalizacji jaja

Tabela przedstawia osiemnaście uprawnień składających się na grupę `guild.user.*`. Domyślne wartości ustawione na `true` oznaczają, że uprawnienia te są przyznawane automatycznie wszystkim graczom na serwerze (grupa `default` w systemach uprawnień), co eliminuje konieczność ręcznej konfiguracji podstawowego dostępu. Administrator może jednak odebrać dowolne z tych uprawnień konkretnemu graczowi lub grupie, co spowoduje ukrycie odpowiednich komend w menu pomocy i zablokowanie dostępu do funkcjonalności. Mechanizm ten pozwala na precyzyjne dostosowanie rozgrywki — na przykład na serwerach hardcore można odebrać uprawnienie `guild.user.egg.shield`, co wyłączy system tarcz ochronnych i zwiększy stawkę ataków na jaja gildii. Notacja kropkowa umożliwia również przyznanie dostępu do całej podgrupy, np. `guild.user.cuboid.*` daje dostęp do wszystkich czterech uprawnień związanych z cuboidem bez konieczności enumeracji pojedynczych node'ów.

12.3.2 Uprawnienia VIP/Premium: `guild.vip.*`

Plugin definiuje dodatkową grupę uprawnień przeznaczoną dla graczy z rangami VIP, Premium lub Sponsor, która rozszerza domyślne limity i odblokowuje dodatkowe funkcjonalności. Uprawnienia VIP nie zastępują uprawnień podstawowych — działają kumulatywnie, co oznacza, że gracz VIP musi posiadać zarówno `guild.user.*`, jak i `guild.vip.*`.

Upewnienie	Efekt
<code>guild.vip.maxmembers.bonus</code>	Zwiększa limit członków gildii o wartość skonfigurowaną w <code>config.yml</code> (domyślnie +3)
<code>guild.vip.cuboid.expand.bypass.time</code>	Pomija wymóg minimalnego czasu istnienia gildii przy awansie cuboidu
<code>guild.vip.egg.repair.cooldown</code>	Redukuje cooldown naprawy jaja o 50%
<code>guild.vip.egg.shield.extended</code>	Wydłuża czas działania tarczy o 50%
<code>guild.vip.color.custom</code>	Odblokowuje dostęp do niestandardowych kolorów prefixu gildii (ponad 16 podstawowych)
<code>guild.vip.join.bypass</code>	Pomija opłatę dołączeniową w systemie pay-to-join
<code>guild.vip.villager.extra</code>	Zwiększa limit wieśniaków handlowych o 1 per poziom gildii

Upewnienia VIP nie są przyznawane domyślnie żadnej grupie i wymagają jawnej konfiguracji w systemie upewnień. Mechanizm działania polega na tym, że plugin przy każdej operacji wymagającej walidacji limitu (np. zaproszenie gracza, awans cuboidu, naprawa jaja) odpytuje permission provider o obecność odpowiedniego node'a VIP i stosuje modyfikator. Na przykład, przy obliczaniu maksymalnej liczby członków, plugin wykonuje sekwencję: pobiera bazowy limit z konfiguracji dla danego poziomu gildii, sprawdza upewnienie `guild.vip.maxmembers.bonus`, a jeśli wynik jest pozytywny, dodaje wartość bonusu. Taki model pozwala administratorowi dowolnie konfigurować wartości bonusów w `config.yml` niezależnie od struktury rang na serwerze.

12.3.3 Upewnienia administratora: `guild.admin.*`

Grupa upewnień administracyjnych zapewnia pełny dostęp do wszystkich funkcji pluginu, w tym do operacji modyfikujących dane innych gildii, konfiguracji systemowej i narzędzi diagnostycznych. Upewnienia te powinny być przypisywane wyłącznie do grup administratorów i właścicieli serwera.

Upewnienie	Domyślna wartość	Opis
<code>guild.admin.*</code>	op	Główny węzeł wildcard obejmujący wszystkie upewnienia administracyjne

<code>guild.admin.delete</code>	op	Rozwiązywanie dowolnych gildii bez ograniczeń
<code>guild.admin.rename</code>	op	Zmiana nazwy i tagu dowolnej gildii
<code>guild.admin.setpoints</code>	op	Nadpisywanie punktacji rankingowej gildii
<code>guild.admin.setlevel</code>	op	Zmiana poziomu cuboidu bez weryfikacji kosztów
<code>guild.admin.bypass</code>	op	Omijanie ochrony wszystkich cuboidów
<code>guild.admin.verify</code>	op	Uruchamianie pełnej weryfikacji stanu pluginu
<code>guild.admin.reload</code>	op	Przeładowywanie plików konfiguracyjnych
<code>guild.admin.backup</code>	op	Tworzenie kopii zapasowych danych
<code>guild.admin.migrate</code>	op	Migracja danych z innych pluginów gildyjnych
<code>guild.admin.debug</code>	op	Sterowanie poziomem logów diagnostycznych
<code>guild.admin.spy</code>	op	Podgląd prywatnych czatów gildyjnych i sojuszniczych
<code>guild.admin.notification</code>	op	Otrzymywanie powiadomień o atakach na jaja wszystkich gildii

Tabela obejmuje trzynaście uprawnień administracyjnych, w tym dwanaście bezpośrednio powiązanych z komendami z sekcji 12.2 oraz dwa dodatkowe uprawnienia specjalne: `guild.admin.spy` i `guild.admin.notification`. Uprawnienie `guild.admin.spy` umożliwia administratorowi podgląd wiadomości wysyłanych na czatach wewnętrznych gildii (`/g <wiadomość>`) oraz czatach sojuszniczych, co jest narzędziem moderacyjnym wykorzystywanym do wykrywania naruszeń regulaminu serwera. Uprawnienie `guild.admin.notification` powoduje, że administrator otrzymuje powiadomienia o każdym ataku na jajo dowolnej gildii na serwerze, co pozwala na aktywny nadzór nad przebiegiem walk i ewentualną interwencję w przypadku oszustw lub nadużyć. Domyślna wartość `op` oznacza, że uprawnienia administracyjne przyznawane są wyłącznie operatorom serwera — w typowej konfiguracji z pluginem LuckPerms należy nadać je grupie `admin` lub `owner` poprzez bezpośrednie przypisanie node'a `guild.admin.*` z włączonym inheritance.

12.3.4 Wildcard permissions i dziedziczenie

Plugin Fraction Guild Clans w pełni wspiera mechanizm wildcard permissions (uprawnień wieloznacznych) zdefiniowany w specyfikacji Bukkit API. Notacja z gwiazdką (`*`) na końcu node'a pozwala na przyznanie dostępu do całej gałęzi drzewa uprawnień. Przykładowo, nadanie graczowi uprawnienia `guild.user.*` jest równoważne z indywidualnym przypisaniem wszystkich node'ów z prefiksem `guild.user.`, w tym tych zdefiniowanych w przyszłych aktualizacjach pluginu. Ta właściwość gwarantuje kompatybilność

wsteczną — dodanie nowej komendy gracza w aktualizacji pluginu nie wymaga modyfikacji konfiguracji uprawnień, pod warunkiem że gracz posiada wildcard.

Struktura dziedziczenia w pluginie uprawnień LuckPerms dla typowego serwera z rangami VIP i Premium powinna wyglądać następująco:

```
# Przykładowa konfiguracja grup w LuckPerms
grupy:
  default:
    permissions:
      guild.user.*: true

  vip:
    parents:
      - default
    permissions:
      guild.vip.*: true

  premium:
    parents:
      - vip
    permissions:
      guild.vip.maxmembers.bonus: true
      guild.vip.color.custom: true

  admin:
    permissions:
      guild.admin.*: true
      guild.user.*: true
      guild.vip.*: true

  owner:
    parents:
      - admin
    permissions:
      "*": true
```

W powyższej konfiguracji grupa `default` otrzymuje wyłącznie podstawowe uprawnienia gracza. Grupa `vip` dziedziczy wszystkie uprawnienia z `default` (mechanizm `parents`) i rozszerza je o zestaw `guild.vip.*`. Grupa `premium` dziedziczy zarówno od `vip`, jak i pośrednio od `default`, co daje jej pełen zestaw uprawnień podstawowych, VIP-owskich oraz dodatkowe uprawnienia ekskluzywne. Grupa `admin` nie dziedziczy z hierarchii gracza, lecz otrzymuje bezpośrednio zarówno `guild.admin.*`, jak i `guild.user.*` z `guild.vip.*`, co gwarantuje administratorowi dostęp do wszystkich funkcji pluginu bez konieczności posiadania rangi gracza. Warto zwrócić uwagę, że grupa `owner` z uprawnieniem globalnym `"*": true` automatycznie posiada dostęp do wszystkich komend pluginu, ponieważ Bukkit API traktuje wildcard globalny jako zastępczy dla wszystkich zarejestrowanych node'ów permission.

Plugin obsługuje również negatywne uprawnienia (negative permissions), co pozwala na precyzyjne odebranie konkretnego dostępu bez konieczności rezygnowania z wildcard. Przykładowo, jeśli administrator chce, aby grupa `vip` posiadała wszystkie uprawnienia VIP z wyjątkiem `guild.vip.join.bypass`, może skonfigurować to następująco:

```
vip:
  permissions:
    guild.vip.*: true
    guild.vip.join.bypass: false
```

W takiej konfiguracji LuckPerms (oraz większość współczesnych systemów uprawnień) przetworzy wyrażenie od lewej do prawej, przyznając najpierw pełen zestaw `guild.vip.*`, a następnie odbierając konkretny node. Plugin Fraction Guild Clans odpytuje system uprawnień poprzez standardową metodę `Player#hasPermission(String)`, która honoruje tę semantykę negacji. Dla serwerów korzystających z PermissionsEx lub innych starszych systemów zaleca się migrację do LuckPerms ze względu na obsługę kontekstów (contexts), co umożliwia przyznanie uprawnień warunkowo — na przykład `guild.user.create` wyłącznie w świecie `world`, a nie w `world_nether`. Taka granularność jest szczególnie przydatna na serwerach multi-world, gdzie mechanika gildii ma obowiązywać selektywnie.

15. Troubleshooting i FAQ

Rozdział ten zawiera katalog najczęściej zgłaszanych problemów wraz z gotowymi rozwiązaniami, zalecenia optymalizacyjne dla dużych instalacji oraz odpowiedzi na pytania powtarzające się najczęściej w procesie wdrażania i eksploatacji pluginu. W wersji 2.0 architektura FractionGuildClans przeszła fundamentalną zmianę — usunięto wszystkie zależności zewnętrzne (Vault, WorldGuard, WorldEdit), co eliminuje całą klasę problemów znanych z wcześniejszych generacji pluginów gildyjnych. Z drugiej strony pojawiły się nowe obszary wymagające diagnostyki: system modułarny, pliki językowe z obsługą MiniMessage oraz opcjonalna integracja z EcoCore.

15.1 Częste Problemy

Poniższa tabela grupuje problemy według komponentu, do którego się odnoszą — instalacji i startu, systemu cuboidów oraz mechaniki jaja gildii. Każdy wiersz zawiera symptomy rozpoznawcze, przyczynę źródłową oraz precyzyjną instrukcję naprawczą. Zaleca się przechodzenie przez kroki diagnostyczne w podanej kolejności.

Kategoria	Problem (symptomy)	Przyczyna	Rozwiązanie
-----------	--------------------	-----------	-------------

Instalacja	Plugin nie ładuje się przy starcie serwera; brak komend <code>/guild</code> w konsoli	Nieprawidłowa wersja Paper/Spigot (poniżej 1.21) lub uszkodzony plik JAR	Zweryfikuj wersję serwera (<code>/version</code> — wymagana 1.21+). Usuń i pobierz ponownie plik JAR. Sprawdź logi startowe pod kątem błędów <code>UnsupportedClassVersionError</code> .
Instalacja	Komendy są niedostępne pomimo prawidłowego startu; komunikat "Unknown command"	Brak uprawnień lub konflikt aliasu z innym pluginem	Sprawdź uprawnienia gracza (<code>guild.command.base</code>). Zweryfikuj w <code>config.yml</code> sekcję <code>commands.aliases</code> — usuń konfliktujący alias lub zmień priorytet w <code>Bukkit (bukkit.yml)</code> .
Instalacja	Błąd <code>NoClassDefFoundError</code> w logach przy starcie	Plugin został skompilowany z brakującą zależnością lub JAR jest częściowo uszkodzony	Pobierz oficjalny build z autoryzowanego źródła. Wersja 2.0 nie wymaga żadnych zewnętrznych bibliotek — błąd ten nie powinien występować przy oficjalnej dystrybucji. Zgłoś incydent deweloperowi.
Instalacja	Komunikat "EcoCore not found, using item economy" przy starcie (INFO, nie ERROR)	EcoCore nie jest zainstalowany lub ma inną nazwę pliku JAR	To oczekiwane zachowanie — plugin pracuje w trybie fallback z itemami jako walutą. Jeśli potrzebujesz zaawansowanej ekonomii, zainstaluj EcoCore w katalogu <code>plugins/</code> . Upewnij się, że nazwa pliku JAR EcoCore nie zawiera spacji ani polskich znaków.
Cuboid	Gracze nie mogą budować na własnym terenie gildii pomimo członkostwa	Moduł <code>cuboid.yml</code> ma nieprawidłowe flagi lub gracz nie jest zapisany w danych gildii	Sprawdź flagi w <code>modules/cuboid.yml</code> : <code>flags.build</code> musi być <code>true</code> dla członków. Wykonaj <code>/guild info</code> i zweryfikuj listę członków. Jeśli gracz jest na liście a problem trwa — wykonaj <code>/guild admin cuboid refresh [tag]</code> aby przebudować cache regionu.
Cuboid	Obcy gracze mogą niszczyć bloki w obrębie czyjegoś cuboidu	System ochrony jest wyłączony lub flaga <code>destroy</code> jest ustawiona na <code>allow</code> dla grupy <code>others</code>	Zweryfikuj stan ochrony: <code>/guild admin cuboid debug [tag]</code> — pokaże efektywne flagi w danej lokalizacji. W <code>modules/cuboid.yml</code> upewnij się, że <code>flags.destroy.others</code> ma wartość <code>deny</code> . Sprawdź czy nie aktywował się tryb "raid" (wojna) — podczas raidingu ochrona jest częściowo wyłączona.

Cuboid	Cuboid nachodzi na spawn lub teren innej gildii	Promień cuboidu jest za duży lub odległość między sercami gildii jest niewystarczająca	W <code>modules/cuboid.yml</code> zmniejsz wartość <code>cuboid.base-radius</code> lub zwiększ <code>guild.min-distance-between-guilds</code> w <code>config.yml</code> . Użyj komendy <code>/guild admin cuboid resize [tag] [nowy-promien]</code> aby dostosować istniejący cuboid bez ponownego zakładania gildii.
Jajo	Komunikat "Nie można postawić jaja" przy zakładaniu gildii	Lokalizacja serca gildii nie spełnia warunków (za blisko granicy świata, w regionie chronionym przez inny plugin, lub na niedozwolonym biomie)	Zmień lokalizację — serce musi być w Overworld, min. 50 bloków od spawnu, na biomie dozwolonym (lista w <code>modules/egg.yml: valid-biomes</code>). Upewnij się, że kolumna Y jest wystarczająco wysoka (brak bedrock bezpośrednio pod blokiem).
Jajo	Jajo nie otrzymuje obrażeń od TNT / gracze nie mogą je uszkodzić	Jajo znajduje się w fazie ochrony początkowej (<code>egg.protection-time > 0</code>) lub jego obsługa jest wyłączona w module	Sprawdź status ochrony komendą <code>/guild egg info</code> . Jeśli chronione — poczekaj na koniec timera lub anuluj ochronę jako admin: <code>/guild admin egg unprotect [tag]</code> . Upewnij się, że moduł <code>egg.yml</code> ma <code>enabled: true</code> .
Jajo	Gildia nie upada po zniszczeniu jaja	Parametr <code>egg.guild-fall-delay</code> jest ustawiony na wartość <code>> 0</code> lub mechanika upadku jest wyłączona	Sprawdź <code>modules/egg.yml: guild-fall-delay</code> określa czas opóźnienia między zniszczeniem jaja a rozwiązaniem gildii (domyślnie 24h). Ustaw na <code>0</code> aby gildia upadała natychmiast. Upewnij się, że <code>destroy-ends-guild</code> ma wartość <code>true</code> .

Analiza tabeli ujawnia wyraźny podział problemów na trzy warstwy. Problemy instalacyjne w v2.0 są zdecydowanie prostsze do zdiagnozowania niż w generacji 1.x — brak zależności od Vault i WorldGuard eliminuje całą kategorię błędów związanych z niezgodnością wersji lub brakiem pluginów ekonomicznych. Komunikat o fallbacku do ekonomii itemowej jest **informacyjny**, nie błędem — administrator powinien traktować go jako potwierdzenie prawidłowego działania mechanizmu degradacji. W warstwie cuboidów najczęstszą przyczyną problemów są nieprawidłowe flagi lub konflikty stanu (np. tryb raidingu tymczasowo modyfikujący uprawnienia) — komenda debugowania `/guild admin cuboid debug` jest kluczowym narzędziem diagnostycznym, które pokazuje efektywne uprawnienia w punkcie bez konieczności przekopywania się przez pliki YAML. W warstwie jaja z kolei dominują problemy konfiguracyjne związane

z ochroną początkową i opóźnieniem upadku — oba parametry istnieją celowo, aby chronić nowe gildie przed natychmiastowym raidingiem, ale administratorzy często interpretują je jako błędy mechaniki.

15.2 Optymalizacja

15.2.1 Zalecenia dla dużych serwerów (100+ graczy)

Serwery o wysokiej liczbie jednoczesnych graczy wymagają świadomego dostrojenia parametrów pluginu. W architekturze v2.0 każdy moduł można niezależnie wyłączyć lub ograniczyć, co dla administratora precyzyjną kontrolę nad zużyciem zasobów.

Baza danych. Dla instalacji 100+ graczy zalecany jest MySQL zamiast domyślnego SQLite. W `config.yml` ustaw `database.type: mysql` i skonfiguruj parametry połączenia. MySQL lepiej radzi sobie z równoległymi zapisami z wielu wątków — szczególnie podczas peaków, gdy wiele gildii jednocześnie dokonuje transakcji ekonomicznych lub modyfikuje cuboidy.

Moduły — selektywne wyłączanie. Przeanalizuj, które funkcje są faktycznie używane przez Twoją społeczność. Każdy nieużywany moduł to zaoszczędzone cykle CPU i pamięć. Aby wyłączyć moduł bez restartu serwera:

```
/guild admin module disable villagers  
/guild admin module disable tab  
/guild admin module reload
```

Po wykonaniu powyższych komend plugin dynamicznie odłącza moduł — nie trzeba restartować serwera ani przeładowywać całej konfiguracji. Moduły bezpieczne do wyłączenia na dużych serwerach PvP (jeśli dana funkcja nie jest używana): `villagers` (wiesniacy handlowi), `tab` (custom tablista — można zastąpić pluginem zewnętrznym), `map` (mapa HTML — generowana synchronicznie). Modułów `cuboid`, `egg` oraz `ranking` nie należy wyłączać, ponieważ stanowią one core mechaniki gildyjnej.

Cuboidy i cache. Zwiększ wartość `cuboid.cache-ttl` w `modules/cuboid.yml` z domyślnych 30 sekund do 120 sekund. Spowoduje to rzadsze odświeżanie cache regionów kosztem nieco dłuższego czasu propagacji zmian uprawnień. W sekcji `cuboid.flags` wyłącz szczegółowe logowanie interakcji (`log-interactions: false`) — na dużym serwerze generuje to gigantyczne ilości wpisów w logach bez wartości diagnostycznej.

Tick rate i asynchroniczność. W `config.yml` ustaw `async-guild-operations: true` aby operacje na gildiach (zapisy, aktualizacje rankingów) były wykonywane poza głównym wątkiem serwera (main thread). Parametr `tick-rate.ranking-update` domyślnie wynosi 1 (co tick) — na serwerze 100+ graczy zaleca się zwiększenie do 5 lub 10, co oznacza aktualizację rankingów co 5-10 ticków zamiast co każdy tick. Spadek responsywności rankingów jest niezauważalny dla graczy, ale redukuje obciążenie TPS (Ticks Per Second) znacząco.

15.3 FAQ

Poniższa tabela zawiera 10 najczęściej zadawanych pytań przez administratorów i graczy. Odpowiedzi zostały sformułowane z uwzględnieniem specyfiki architektury v2.0 — w szczególności braku zależności zewnętrznych oraz systemu modularnego.

#	Pytanie	Odpowiedź
1	Czy do działania pluginu potrzebny jest Vault?	Nie. W wersji 2.0 Vault nie jest wymagany ani obsługiwany. Plugin używa własnego systemu ekonomicznego: jeśli EcoCore jest zainstalowany — korzysta z jego API; jeśli nie — automatycznie przełącza się na ekonomię itemową (walutą są fizyczne itemy z konfiguracji, np. diamenty, szmaragdy). Nie trzeba konfigurować żadnego pluginu ekonomicznego.
2	Czy potrzebny jest WorldGuard lub WorldEdit?	Nie. Plugin posiada wbudowany, niezależny system ochrony terenów oparty na cuboidach z własnymi flagami (build, destroy, pvp, use, bucket, entry, interact). Nie dochodzi do konfliktów z WorldGuard, ponieważ FGC v2.0 nie rejestruje się w API WorldGuard ani nie modyfikuje jego flag. Możesz mieć WorldGuard na serwerze dla innych światów — nie będzie to kolidować.
3	Jak dodać własny język do pluginu?	Utwórz nowy plik w katalogu <code>plugins/FractionGuildClans/lang/</code> o nazwie zgodnej z konwencją <code>ll_cc.yml</code> (np. <code>fr_FR.yml</code> dla francuskiego). Skopiuj zawartość <code>en_US.yml</code> jako szablon i przetłumacz wszystkie wartości. W <code>config.yml</code> dodaj kod języka do listy <code>lang.available-languages</code> . Ustaw <code>lang.default: fr_FR</code> jeśli ma to być język domyślny.
4	Czy można przeładować pliki językowe bez restartu serwera?	Tak. Użyj komendy <code>/guild admin lang reload</code> — przeładuje wszystkie pliki z katalogu <code>lang/</code> i zastosuje zmiany w czasie rzeczywistym. Gracze zobaczą nowe tłumaczenia natychmiast. Komenda ta jest dostępna tylko dla administratorów (uprawnienie <code>guild.admin.lang.reload</code>).
5	Jak wyłączyć konkretny moduł bez restartu serwera?	Użyj komendy <code>/guild admin module disable [nazwa]</code> , gdzie <code>[nazwa]</code> to nazwa pliku w <code>modules/</code> bez rozszerzenia <code>.yml</code> (np. <code>villagers</code> , <code>tab</code> , <code>map</code>). Następnie wykonaj <code>/guild admin module reload</code> aby zastosować zmiany. Moduł zostanie odłączony natychmiast — nie wpływa to na inne funkcje. Aby włączyć ponownie: <code>/guild admin module enable [nazwa]</code> .

6	Co się stanie jeśli wyłączę moduł <code>cuboid</code> lub <code>egg</code> ?	Moduły <code>cuboid</code> i <code>egg</code> należą do core mechaniki i nie powinny być wyłączane na serwerze produkcyjnym. Wyłączenie <code>cuboid</code> spowoduje całkowity brak ochrony terenów — każdy gracz będzie mógł budować i niszczyć wszędzie. Wyłączenie <code>egg</code> uniemożliwi zakładanie gildii (jajo jest wymagane jako serce gildii) i zatrzyma mechanikę raidingu. Moduły te istnieją jako osobne pliki wyłącznie dla celów deweloperskich i debugowania.
7	Czy plugin współpracuje z PlaceholderAPI?	Tak, opcjonalnie. Integracja z PlaceholderAPI jest dostępna i włączana automatycznie gdy PlaceholderAPI jest wykryte w katalogu <code>plugins/</code> . Jeśli PlaceholderAPI nie jest zainstalowane — plugin działa w pełni funkcjonalnie, po prostu placeholders nie są rozwijane w wiadomościach innych pluginów. Lista dostępnych placeholderów: <code>%guild_name%</code> , <code>%guild_tag%</code> , <code>%guild_level%</code> , <code>%guild_points%</code> , <code>%guild_online%</code> , <code>%guild_members%</code> , <code>%guild_rank%</code> .
8	Jak zmienić walutę w trybie <code>item economy</code> (bez EcoCore)?	W <code>modules/economy.yml</code> (używany jako fallback) znajdź sekcję <code>item-economy.currency-item</code> . Zmień wartość na dowolny prawidłowy Material z API Bukkit (np. <code>EMERALD</code> , <code>NETHERITE_INGOT</code> , <code>GOLD_BLOCK</code>). Opcjonalnie ustaw <code>item-economy.currency-name</code> dla wyświetlanej nazwy waluty. Zmiany wymagają przeładowania modułu: <code>/guild admin module reload economy</code> .
9	Gdzie znajdują się dane gildii i czy można je backupować?	Dane przechowywane są w <code>plugins/FractionGuildClans/data/</code> — podkatalogi <code>guilds/</code> (dane gildii), <code>players/</code> (dane graczy) oraz <code>backups/</code> (automatyczne kopie zapasowe). Plugin tworzy automatyczny backup co 6 godzin (konfigurowalne w <code>config.yml: backup.interval</code>). Ręczny backup: <code>/guild admin backup create</code> . Przywracanie: <code>/guild admin backup restore [nazwa-pliku]</code> . Przy pełnej migracji między serwerami wystarczy skopiować cały katalog <code>data/</code> .
10	Czy można jednocześnie używać MiniMessage i legacy kodów kolorów?	Tak. System Lang Files akceptuje oba formaty jednocześnie w tym samym pliku. Możesz użyć <code><green>witaj</green></code> (MiniMessage) obok <code>&awitaj</code> (legacy) w tej samej linii. Plugin automatycznie parsuje oba formaty do komponentów Adventure. Zalecany jest MiniMessage dla nowych tłumaczeń, ponieważ oferuje zaawansowane efekty: gradienty (<code><gradient:red:blue>tekst</gradient></code>), klikalne linki oraz hover events.

Tabela FAQ odzwierciedla fundamentalne zmiany filozofii w v2.0: pytania 1 i 2 (Vault i WorldGuard) są obecnie najczęstszym źródłem wątpliwości administratorów przychodzących ze starszych pluginów, ponieważ cała kategoria problemów z integracją zewnętrzną przestała istnieć. Pytania 3, 4 i 5 odnoszą się do nowych możliwości architektury v2.0 — systemu Lang Files oraz modularności — które wymagają od

administratora przyzwyczajenia się do nowych komend i przepływów pracy. Pytanie 6 jest istotne z punktu widzenia bezpieczeństwa: choć technicznie można wyłączyć moduły core, robi się to wyłącznie w kontrolowanym środowisku deweloperskim. Pytania 7-10 pokrywają operacyjne aspekty codziennej eksploatacji: integracja opcjonalna (PlaceholderAPI), konfiguracja waluty w trybie fallback, zarządzanie danymi oraz formatowanie wiadomości. Razem tworzą one kompletny przewodnik pozwalający administratorowi samodzielnie rozwiązać większość problemów bez konieczności sięgania do zewnętrznej dokumentacji lub wsparcia technicznego.